



UNIVERSITY OF BAMBERG

Applied Computer Science Degree Program in the
Faculty of Information Systems and Applied Computer Sciences

MASTER'S THESIS

Extensionality and Instance-based Methods in Tableau-based Higher-Order Automated Theorem Proving

by

Johannes Schuster

Supervisor

Prof. Dr. Christoph Benzmüller

Chair for AI Systems Engineering

August 2026

The thesis is written as a set of executable Livebook notebooks. This document is a typeset rendering of them, and both are held in the archived record <https://doi.org/10.5281/zenodo.22045933>.

Abstract

Tableau calculi for higher-order logic decompose a problem without clausifying it, at the cost of one global dependency: a free variable is rigid, so a value closing one branch commits every branch. This thesis presents SHOT, a prover organised around that dependency. Every branch-level rule of its calculus over Church's simple type theory is binding-free, recording the condition under which a branch would close rather than choosing a value. Boolean extensionality is reached by renaming an opaque propositional argument onto the branch and instantiating it over the two truth values, the functional principles by expanding equalities at functional type, and the search rests on instance-based methods: general bindings, instance-based quantifier expansion, finite expansion of quantifiers over purely propositional types, and ordered demodulation under a term order stable under substitution. Closing the tableau is then a constraint satisfaction problem over the closure evidence of all open branches, solved by one process, while the branch expansion feeding it runs on a pool of BEAM workers over shared hash-consed terms. An untuned ablation over the TPTP higher-order corpus and a study on a structured problem set locate two limits: global closure produces about half of all refutations and does not receive the parallelism, so it is at present too weak for the architecture built around it, and the calculus decides most of the problems whose validity needs only conversion and Boolean extensionality and few of those where a functional principle is required, which locates a missing rule rather than a general weakness.

Keywords: Automated Theorem Proving · Higher-Order Logic · Semantic Tableaux · Parallel Proof Search

Acknowledgements

I thank Christoph Benzmüller and David Fuenmayor for the frequent and stimulating discussions during the preparation of this work. Their feedback and pointers to literature and concepts for higher-order proof search and adjacent topics was invaluable for both the design and the implementation of the calculus.

Contents

List of Figures	ix
List of Tables	xi
1 Introduction	1
1.1 Higher-Order Proof Search	1
1.2 Analytic Tableaux in Higher-Order Proving	2
1.3 Rigidity, Locality and Parallel Search	3
1.4 Agent-Based and Cooperative Proof Search	4
1.5 The BEAM as Platform for Concurrency	5
1.6 Executable Presentation	6
1.7 Contributions	7
1.8 Scope and Non-Goals	8
1.9 Outline	9
2 Preliminaries	11
2.1 Notation	11
2.2 Types	11
2.3 Terms	14
2.4 Typing	15
2.5 Shifting and Substitution	18
2.6 Reduction and Normal Forms	18
2.7 Parameter Terms	19
2.8 Syntactic Equality and Unification	19
2.9 Connectives, Quantifiers and Equality	20
2.10 Frames and Interpretations	20
2.11 Denotation and Models	21
2.12 Extensionality and Leibniz Equality	22
2.13 Satisfiability and Entailment	22
3 Higher-Order Tableaux	25
3.1 Global and Local Tableau States	25
3.2 Propositional Rules	26
3.3 Quantifier Rules	27
3.4 Primitive Substitution	29
3.5 Boolean Extensionality: Renaming and Instantiation	29
3.6 Equality Rules	31
3.7 Closure and Discharge	34
3.8 Literal Processing	34
3.9 Preprocessing and BDDs	37

3.10	Search Bounds	39
3.11	Soundness and Completeness	40
3.12	Rule Priority and Implementation	41
4	Higher-Order Unification	43
4.1	The Shape of the Problem	43
4.2	Rigid-Rigid: Decomposition	44
4.3	Flex-Rigid: Imitation and Projection	44
4.4	Flex-Flex: Deferral	45
4.5	The Procedure	45
4.6	A Worked Enumeration	46
4.7	Depth, Completeness and Decidable Fragments	47
5	Term Ordering	49
5.1	Overview	49
5.2	Requirements for an Orientation Order	49
5.3	The Type Order	50
5.4	The Ordering Parameters	50
5.5	The Order	52
5.6	No Transitivity: One Pair at a Time	54
5.7	The Total Prover-Side Wrapper	54
5.8	Summary and Forward Pointers	56
6	A Concurrent Actor-Based Architecture	57
6.1	Overview	57
6.2	The Rigid Variable Problem	57
6.3	The Supervision Tree	58
6.4	Pure Branch Logic, Stateful Shell	60
6.5	The Evidence Stream	61
6.6	Global Closure as Constraint Satisfaction	61
6.7	The Message Flow	62
6.8	Iterative Deepening	62
6.9	The Peer Agents	64
6.10	A Proof, End to End	65
6.11	Discussion	66
7	Peer Agents and Proof Reconstruction	69
7.1	Overview	69
7.2	The Suggestion Agent	69
7.3	The Model Agent	71
7.4	Proof Reconstruction	73
7.5	Discussion	74
8	Evaluation	77
8.1	Overview	77
8.2	Method and Scoring	77
8.3	Baseline	78
8.4	Ablating the Agents, the Calculus and the Search Control	80
8.5	Scaling with the Worker Pool	81

8.6	Closure Search under the Two Pool Sizes	82
8.7	The Structured Problem Set	84
8.8	Coverage by Model Class	85
8.9	Threats to Validity	86
9	Conclusion	89
9.1	Summary	89
9.2	Design Decisions and Assumptions	89
9.3	Measurement Results	91
9.4	Limitations	92
9.5	Future Work	93
	Bibliography	97
A	Examples	103
A.1	Overview	103
A.2	The Propositional and Quantifier Rules	103
A.3	Equality	106
A.4	Leibniz and Primitive Equality	110
A.5	Extensionality: The Trivial Directions	110
A.6	Boolean Extensionality: The Non-Trivial Direction	111
A.7	De Morgan by Degrees	114
A.8	Primitive Substitution and Finite Domains	117
A.9	Higher-Order Unification	119
A.10	Cantor	119
A.11	Countermodels	120
A.12	Bounds, Timeouts and the Unknown Verdict	123
	Declaration of Independent Authorship	125

List of Figures

4.1	Enumeration trace for $X(a) \stackrel{?}{=} f(a, a)$	48
6.1	The session supervision tree.	59
6.2	The message sequence of one proof session.	63
6.3	The reconstructed proof of $(\forall X. p(X)) \supset p(a)$	66
8.1	Refutations gained and lost against the baseline.	79
8.2	Coverage on the structured problem set. Reference provers at a 10 s limit; SHOT at 5 s.	84
A.1	The proof of $p \supset p$	104
A.2	The proof of the distribution of a universal quantifier over a conjunction.	105
A.3	The proof of $(\forall l. p(1)) \supset (\exists l. p(1))$	106
A.4	The proof of reflexivity.	107
A.5	The proof of symmetry.	108
A.6	The proof of transitivity.	109
A.7	The proof of Example 9.	111
A.8	The proof of Example 10.	112
A.9	The proof of Example 17, $\neg(a = \neg a)$	113
A.10	The proof of Example 18.	114
A.11	De Morgan's law at the first strength.	115
A.12	De Morgan's law at the second strength.	116
A.13	De Morgan's law at the third strength.	117
A.14	The proof of Example 21.	118
A.15	The proof of Example 12.	119
A.16	The proof of Example 31, the surjective Cantor theorem.	120
A.17	The saturated branch behind the propositional countermodel.	121
A.18	The saturated branch behind the countermodel of Example 21.	122

List of Tables

3.1	The sixteen members of $\text{gen}((o, o) \rightarrow o)$	28
3.2	The BDD oracle applied to four propositional formulae under :deep simplification.	39
3.3	Rule priority costs. Lower cost means higher priority.	41
3.4	Classification of $a \equiv b$ under the two equivalence-expansion strategies.	42
3.5	Rule family and queue cost for three representative formulae.	42
5.1	Type-order queries under the precedence $\iota < o$	51
5.2	Orientability of the two differentiation rewrite rules under three status assignments.	51
5.3	The accessible-subterm rule.	53
5.4	The precedence rule, with $g \succ_{\mathcal{F}} h \succ_{\mathcal{F}} c$	53
5.5	Comparing primitive equality at ι against its Leibniz encoding, under two precedences.	54
8.1	Baseline outcomes over 5109 problems.	78
8.2	Sixteen configurations, refutations and deltas against the baseline.	79
8.3	worker_pool_size :auto against 1.	82
8.4	ContradictionAgent counters, 5031 runs per configuration.	83
8.5	The 130 problems by the class in which they are valid.	85

Introduction

1.1 Higher-Order Proof Search

Classical higher-order logic, in the form of Church’s simple theory of types [Chu40] under Henkin’s general model semantics [Hen50, And02], gains substantial expressiveness over first-order logic from a small extension of the syntax. Quantification over predicates and functions makes induction schemes, set-theoretic constructions, program semantics and the shallow embeddings of non-classical logics statable as single formulae rather than as first-order axiom schemes. The additional difficulty this creates for proof search has three independent sources.

The first is unification. Deciding whether two simply typed terms have a common instance is undecidable already at second order [Gol81], and Huet’s pre-unification procedure [Hue75] replaces the decision with an enumeration that may be infinite and infinitely branching. Every calculus for higher-order logic therefore invokes an unbounded subprocedure from its most frequent operation, and every implementation has to bound that subprocedure somewhere and recover completeness by deepening. Decidable fragments exist (patterns [Mil91] most notably) and are worth detecting, but they do not cover the problems that motivate the logic.

The second is quantifier instantiation at predicate types. A witness for a variable of type $\bar{\tau}_n \rightarrow o$ is a formula, and no finite set of instances suffices in general. Andrews’ primitive substitution [And89] enumerates approximations to such witnesses by their outermost logical symbol, which is a large branching factor added on top of an already unbounded one.

The third is extensionality. Henkin completeness demands rules for Boolean and functional extensionality, and what a calculus may assume about them is itself stratified into a hierarchy of model classes [BBK04]. Formulae occur as arguments, equality at function types is not decided by structure, and the ordinary first-order division of a problem into terms and formulae stops holding. Calculi differ in how much of this they build in. The principles can be supplied as axioms over the input, or as inference rules directed by the goal, which is the treatment developed for higher-order resolution in [BK98, Ben99a] and carried to equality and paramodulation in [Ben99b], with the alternatives compared in [Ben02].

The systems that address these difficulties are surveyed by Benzmüller and Miller [BM14] and are classified here by their calculus. TPS searches for expansion proofs by mating search [And81, Mil87, ABB00]. LEO-II uses extensional higher-order resolution, in which the extensionality principles are supplied as inference rules and unification constraints are carried as literals rather than solved on the spot [BK98, BPTF08, BSPT15]; Leo-III extends that line to extensional higher-order paramodulation [SB21]. Satallax and its reimplementations Lash use a ground tableau calculus with a SAT solver as the propositional decision procedure [Bro12, BK22]. Zipperposition and the higher-order modes of Vampire and E use superposition

extended to λ -terms [BBT⁺21, VBB⁺21]. The competition record favours the last of these. In the THF division of CASC-J13, held at IJCAR 2026, the entrants were Vampire, Zipperposition, E, Leo-III and LEO-II; Vampire solved 354 of 400 problems and Zipperposition 347, and no tableau-based system entered [Sut26].

1.2 Analytic Tableaux in Higher-Order Proving

A thesis that builds a higher-order tableau prover has to address that record, so the case against the method is stated first and without qualification. Satallax led higher-order proving through the 2010s and was then overtaken by superposition, by a wide margin and on the same benchmark set [VBB⁺21]. The reasons are structural rather than incidental. A saturation-based prover maintains a global clause set and a redundancy criterion, so every derived clause may simplify and subsume every other one; a tableau maintains a tree of branches, for which no comparable notion of one branch rendering another redundant is available. Free-variable tableaux are destructive, in that a closing substitution rewrites the whole tableau, so the standard proof procedure backtracks over instantiations and repeats work [Fit96, Häh01]. Equational reasoning, which superposition was designed around, has to be retrofitted. None of this is disputed here.

Four properties nevertheless argue for the method at higher order. The first of them is conceded by the superposition literature itself.

The input structure is preserved. Clausification is a lossy translation, and at higher order it is lossy in ways that first-order experience does not predict. Formulae occur inside terms, Boolean-typed arguments have to be lifted out, and an axiomatic encoding of the Boolean type degrades saturation performance measurably; the response has been to interleave clausification with the derivation instead of running it as a preprocessing step [NBTv21]. Vukmirović et al. put the comparison plainly: “One of the strengths of tableau provers is that they do not clausify the input problem”, and “delaying the clausification through the use of calculus rules that act on formulas achieves the same effect in superposition” [VBB⁺21]. A tableau calculus obtains that property by construction. Its rules act in place on the connective at the root of a formula, so a subformula to which no rule has been applied is unchanged from the input. For a prover whose secondary aim is presentation, this property is relevant twice over.

The proof object is legible. A closed tableau is a tree whose nodes carry formulae in the vocabulary of the problem, and whose closure is witnessed by a pair of complementary literals and one substitution. It can be read without knowledge of the calculus, displayed as a tree, and verified by hand for small problems. Expansion proofs and matings offer a comparable representation [Mil87]. A refutation obtained by superposition after clausification, skolemisation, Boolean hoisting and simplification relates far less directly to the input formulae.

Rules compose additively. Higher-order tableau calculi have been extended by the addition of rules. Kohlhasse’s $\mathcal{HTE}$ added extensionality rules to a Smullyan-style base [Koh95, Smu68]; Konrad’s HOT reworked that calculus for implementation [Kon98]; Backes and Brown added a choice operator, with completeness relative to Henkin models preserved [BB11]. Each addition is a case in a rule dispatch and a case in the corresponding soundness argument. The same is true of the additions made here: primitive substitution, ordered demodulation and the definition-unfolding

and BDD preprocessing of Chapter 3 are separate rules with separate costs, each independently switchable, so that the ablation study of Chapter 8 can turn one off without disturbing the rest.

Failure is informative. A saturated branch that does not close is a candidate description of a model, which makes the method bidirectional: a failed proof attempt yields the material for a countermodel search. Satallax exploits this [Bro12], and external model finders accept the same material [BN10]. The model agent of Chapter 7 is based on it, and the third verdict this prover can return, that a branch is saturated but not decidably open, follows from treating the countermodel reading as a soundness obligation.

Against those four properties is one that is usually listed as a defect and is treated here as the central design constraint. The free variables introduced by γ -expansion are *rigid*: a variable denotes the same term on every branch it occurs on, and a closed tableau requires a single substitution closing all open branches simultaneously. Branch expansion is local, closure is global. In the standard proof procedure this global dependency is what forces backtracking, and it is why parallelising a tableau naively is unsound: two branches that each close under incompatible bindings have not closed the tableau.

The first-order literature already contains the reformulation that removes the backtracking. Giese’s incremental closure keeps, for each branch, the set of substitutions that would close it, and searches for a common solution incrementally rather than committing to one instantiation and undoing it later [Gie01]. Read as an architecture rather than as a search strategy, this decomposes the tableau into many independent local computations and one shared constraint problem. The present work transfers that decomposition to higher order, where the local computations are more expensive, the constraints are pre-unifiers rather than unifiers [Hue75], and the enumeration underlying each constraint is unbounded. The deferral itself has a higher-order precedent on the resolution side, where unification constraints are carried as literals inside a clause and discharged only when a refutation depends on them [BK98, BPTF08]. The step taken here is to collect the deferred constraints of all open branches and solve them as one problem in a single process.

The claim made here is therefore not that tableaux outperform superposition on higher-order problems; the CASC record indicates otherwise and this thesis presents no evidence to the contrary. However, the tableau method admits a decomposition into independent local work and a single global constraint, which saturation-based calculi do not, since a clause set is shared by construction and mutual simplification makes every inference depend on every other [Bon00]. That decomposition has been exploited concurrently at first order and has not been exploited at higher order on a runtime designed for such computations. Whether doing so is profitable is an empirical question, addressed in Chapter 8.

1.3 Rigidity, Locality and Parallel Search

Parallel deduction has an established classification. Bonacina classifies strategies by granularity, by whether one process directs the others or the processes are peers, and by whether the parallelism subdivides a single search or runs several searches at once [Bon00]. In those terms the architecture of Chapter 6 is coarse-grained, peer-based, and a distributed search: branches are the unit of parallel work, no

process directs another, and the search performed by a population of workers differs from the sequential search it derives from rather than reordering it.

Prior work that parallelises the tableau itself is represented by Goéland, which spawns a lightweight process per branch, communicates closing substitutions over channels, and reconciles them centrally [CRD⁺22]. Goéland is first-order, and the process-per-branch mapping it uses is the one design this thesis considered and did not adopt, for the reasons given in Section 1.5. A second line distributes the reasoning over a population of agents rather than parallelising the tableau, and it is the direct predecessor of the architecture built here.

1.4 Agent-Based and Cooperative Proof Search

Agent-based proof search begins with Ω -Ants, where concurrent agents compute possible arguments for the inference rules applicable to the current partial proof and post them to a blackboard from which a proof planner selects [BS98, Sor01]. Its mature form, OANTS, arranges the agents in two layers over blackboards, admits external reasoning systems as agents alongside the internal ones, and is resource-adaptive: computation is reallocated between the agents according to what each has been observed to contribute, and agents that cannot contribute to the current goal are deactivated [BSJK08]. Konrad’s HOT carries the blackboard into higher-order proving, running an extensional higher-order tableau calculus on a concurrent blackboard architecture [Kon98]; its concurrency serves the blackboard rather than branch expansion.

A second strand couples reasoners of different kinds. LEO-II is built as a cooperative prover: it performs extensional higher-order resolution itself and passes the essentially first-order part of a problem to an external first-order system, whose answer it integrates [BPTF08, BSPT15]. Whether such a pairing is worth its overhead, and what has to hold of the translation between the two logics for it to be, is examined directly in [BSJK05]; [WB16] asks the same question of agent populations in general. LeoPARD supplies the term data structures and the agent infrastructure on which the later systems of this line are implemented [WSB15]. The model agent of Chapter 7 belongs to this strand: it consults an external system about a single branch, and what differs is the direction of the question, satisfiability of a saturated branch rather than refutation of a subgoal.

Leo-III inherits from both strands. Its agent framework has a scheduler that collects the read and write sets of the tasks its agents propose and selects a maximal non-conflicting set of them by an approximation algorithm for combinatorial auctions [SWB16]. The framework has two modes, and the fine-grained one, in which each inference rule is an agent, was reported as “not competitive and still experimental”; the configuration Leo-III competes in runs a sequential loop inside a single agent, with parallelism confined to unification, redundancy checks and calls to external provers [BSW17, SB21]. That experience bears on the design question from one side: when every inference has to be arbitrated by a scheduler, the arbitration is the cost. The architecture of Chapter 6 arbitrates only closure, and leaves expansion unarbitrated.

Two things separate the design developed here from these systems. The first is the direction of control. No scheduler is advised by agents: workers expand branches and publish closure evidence; a single contradiction agent consumes that

evidence and solves the global constraint; further agents subscribe to the same stream and may publish hints that a worker may ignore. An agent is added by subscribing and removed by not starting it, so each agent can be disabled without modifying the components that remain. The second is the level at which computation is allocated. OANTS allocates it between agents explicitly, on evidence of what each has contributed [BSJK08]. Here no component allocates it at all: preemption is counted in reductions by the runtime, and the worker pool is of fixed size, so what an agent receives is decided by the scheduler of the virtual machine rather than by the prover.

1.5 The BEAM as Platform for Concurrency

The choice of Elixir on the BEAM follows from the structure of the computation described above. The unit of parallel work is the branch, and the number of branches grows at every β -expansion, which suggests mapping each branch to its own process. That is not the design used here. Branches are created faster than they are closed, each retains its full formula set for as long as it lives, and each would publish closure evidence to a single contradiction agent whose mailbox and cartesian product both grow with the number of live branches. Memory consumption would then track the size of the tableau rather than the size of the pool, and the schedulers would be occupied by processes that are for the most part not running.

Branches are therefore values rather than processes. A branch is a data structure held in a public ETS table, and a fixed pool of long-lived worker processes takes branches from that table, applies a bounded number of rules to each, and returns them. The pool defaults to one worker per online scheduler, so the degree of parallelism follows the hardware while the number of branches in flight follows the problem. The runtime is used here for a work-stealing pool over shared structures that many processes read and write at once, rather than for a process per unit of work.

Four properties of the runtime are relevant [Arm03]. Scheduling is preemptive and reduction-counted, so a worker that has entered a long pre-unification enumeration cannot starve the workers that would have closed the tableau sooner; the bound on that enumeration is still needed for completeness, but it is no longer needed to keep the system responsive. Each process has its own heap, so garbage collection is per-process and collecting one worker does not suspend the others. Schedulers are per core, and the same program runs on one core and on many without a change in the code, so `worker_pool_size` sweeps a single implementation rather than comparing two in Chapter 8. ETS provides tables with per-slot read and write concurrency, which is what allows the branch queue, the traces, the provenance records and the published hints to be shared between workers without a coordinating process on the frequently executed path.

Message passing between isolated processes also enforces the calculus's central discipline through the primitives of the language rather than by convention. A worker cannot mutate another worker's state, since it holds no reference to it; the only shared mutable objects in a session are the ETS tables owned by a single process, and all global search state resides in them. Wherever the code accesses a shared table instead of sending a message, the local/global partition of Chapter 6 is apparent at the call site.

Supervision has a further consequence. A pre-unification stream that exhausts

memory, or a rule application that raises on a malformed term, terminates one worker rather than the session; its supervisor restarts it and the remaining workers continue. The branch that worker held is not returned to the queue, since checkout removes it from the shared table, so the cost of a worker crash is one abandoned branch. Failure isolation therefore bounds the damage a single malformed term can do, and the defensive checks a single-process prover performs inline are replaced by process termination.

The disadvantages are stated with the same directness. The BEAM is not a fast substrate for the inner loops of a theorem prover. Term traversal, unification and order comparisons are allocation-intensive symbolic operations, and an OCaml or C implementation of the same algorithms performs more of them per second than an Elixir one; Satallax, Lash and Zipperposition are single-threaded systems with substantially better constant factors than are attainable here [Bro12, BK22, BBT⁺21]. Hash-consing of terms and ETS-resident caches reduce that difference, and the speedup from parallelism must compensate for the remainder. No part of this thesis assumes that it does.

It does not. On the structured set of higher-order problems of [BB05], the prover is about three times slower at the median than the fastest reference system on each problem it proves, and two orders of magnitude slower at the tail. A worker pool of one process per online scheduler multiplies inference by a factor of three over a single worker within the same budget and leaves the number of problems solved unchanged, because the parallelism reaches branch expansion rather than the global closure search that produces most of the refutations. Both measurements are reported in Chapter 8.

Two secondary considerations support the choice without being decisive for it. Elixir retains the developers who adopt it, ranking third among the most admired languages in the 2025 Stack Overflow developer survey behind Rust and Gleam [SO25], which is relevant for a research prototype intended to be read and extended by others. Its notebook environment, Livebook, is the medium of this thesis and is the subject of the next section.

1.6 Executable Presentation

Teaching and presentation are secondary aims of the SHOT system, and they influenced the calculus only where no other property had to be given up. The prover's formatters print types and terms in the conventional annotated style rather than in the uncurried notation of Chapter 2, derivations are rendered as trees, and every rule carries a name a reader can look up. Since the tableau method was already the most readable of the candidate calculi, no substantial conflict between the two aims arose.

The thesis itself serves as the proof of concept for that aim. It is written as a set of Livebook notebooks [Liv25] following the principles of literate programming [Knu84]: the chapters are the source of truth, the LaTeX build reads them, and the executable cells run the prover being described. A rule classification, an enumeration of pre-unifiers, an orientation decision and a rendered proof object are computed when the chapter's cells are executed rather than asserted in prose. Three registers are kept typographically distinct throughout, as set out in Chapter 2, so that a claim about the mathematics and a claim about the implementation are never confused for one another. The notebooks and this typeset rendering are archived under a

single identifier, <https://doi.org/10.5281/zenodo.22045933>. A reader who follows a claim to the cell that settles it obtains the code and the stored outputs that produced the figures reported here. The identifier resolves to this version of the notebooks; later versions are separate entries in the same record.

1.7 Contributions

The thesis makes the following contributions.

- A higher-order free-variable tableau calculus over Church’s simple type theory with primitive equality, extensional rules, primitive substitution, and equational reasoning by ordered demodulation, together with the preprocessing that precedes it (Chapter 3).
- A binding-free formulation of every branch-level rule. No rule commits to a value for a free variable; each records a closure condition as evidence. The consequences are traced through the equality rules, which admit only rewrites with an empty matcher, and through the term order, which must be stable under a substitution that has not yet been computed (Chapter 3, Chapter 5).
- A treatment of global closure as a constraint satisfaction problem over the closure evidence of all open branches, solved once by a dedicated process, giving a higher-order counterpart to incremental closure [Gie01] and removing backtracking over instantiations from the search (Chapter 6).
- A concurrent actor architecture realising that partition on the BEAM: a supervision tree per proof session, a fixed pool of worker processes stealing branches from a shared queue, shared state confined to tables owned by a single process, and closure evidence distributed by broadcast (Chapter 6). The measurements qualify the design: coverage is bounded by the rate at which global closure is attempted rather than by the rate at which branches are expanded, so enlarging the worker pool does not solve more problems at the budget measured (Chapter 8).
- Three peer agents built as consumers of that broadcast, for cross-branch instantiation hints, for satisfiability probing of saturated branches, and for global closure, each independently switchable; and reconstruction of an ordinary tableau proof from the derivation traces recorded by the concurrent search (Chapter 7).
- An orientation order for higher-order terms in η -long β -normal form, with the stability and typing properties the deferred-substitution discipline requires (Chapter 5).
- A set of concurrent data structures with explicit memory sharing, on which the pool design depends. Terms are hash-consed into a global pool, constructed once and thereafter referred to by identifier, so that a branch is cheap to write to a shared table and cheap for another worker to take from it; intermediate terms are built in process-local scratchpads and committed to the global pool only when a result survives, so that failed enumerations are discarded without

adding to it. The session’s search state is held in public ETS tables with read and write concurrency, addressed by branch prefix so that closure, tombstoning and hint lookup are single operations (Chapter 2, Chapter 6).

- The SHOT ecosystem: four independently usable Hex packages, `shot_ds` (v1.3.1) for terms, types and the semantics of Church’s simple type theory, `shot_un` (v0.2.1) for pre-unification, `shot_to` (v0.2.0) for the term order, and `shot_tx` (v0.1.0) for the calculus, the architecture and proof objects. Each carries its own documentation and test suite, and the first three are usable without the prover.
- An open-source implementation presented as executable notebooks in which the described behaviour is run rather than paraphrased.
- An ablation study over the agents, the calculus rules and the search control, with parameter sweeps and a scaling measurement over the worker pool, across the TPTP higher-order corpus (Chapter 8). Global closure through the contradiction agent accounts for about half of all refutations; iterative deepening, demodulation, finite α -quantification and atom decomposition contribute measurably; the suggestion agent, primitive substitution, instance-based γ -expansion and the size of the worker pool do not, at the budget measured.
- A qualitative evaluation on the structured set of [BB05] against five reference systems, characterising the coverage by the model class in which each problem is valid: 57 of the 70 problems valid without a functional principle are proved on every run, against five of the 35 that require one (Chapter 8, worked example by example in Appendix A).

1.8 Scope and Non-Goals

The system is a research prototype and is not tuned for competition. No claim is made that it is faster than, or solves more problems than, any of the systems named above, and the evaluation is designed to measure the contribution of its own components rather than to place it in a ranking.

Soundness of the calculus is argued rule by rule, and the obligations are collected in Section 3.11 of Chapter 3. Completeness is not established. That section states as a conjecture what a complete extension of the calculus would look like and names the rule the calculus lacks: the functional extensionality principles are reachable only from a formula that is already an equation, so the calculus as given is expected to be incomplete. The bounded pre-unification of Chapter 4 is recovered by iterative deepening, the interaction of demodulation with the extensionality rules is not analysed, and the semantic layering of [BBK04] is used to state what is assumed rather than to prove what is achieved.

The choice operator is not part of the calculus, so the fragment covered is narrower than that of [BB11]. Concurrency is confined to a single BEAM node; the runtime’s distribution facilities are not used and multi-node scaling is left to future work. Integration with an interactive prover as a hammer-style backend is out of scope.

The logic reasoned about is monomorphic. Input is read in the TPTP typed higher-order dialects [Sut17, SB10], and rank-1 polymorphic TH1 problems [KSR16] are

monomorphised during parsing, so a polymorphic symbol is presented to the calculus as a family of monomorphic ones. Problems whose proofs require an instance not produced by the monomorphisation are therefore not provable, and no reasoning about type variables takes place inside the calculus.

1.9 Outline

Notation and semantics are fixed in Chapter 2, which develops Church’s simple theory of types with an uncurried, η -long β -normal presentation and Henkin general models, and defines the type-level vocabulary the later chapters depend on. The notational conventions follow those of Niederhauser and Middeldorp [NM26].

The calculus is given in Chapter 3: the tableau objects and their global and local states, the propositional and quantifier rules, primitive substitution, the extensionality rules for Boolean and functional equality, ordered demodulation, closure and discharge, literal processing, BDD-based preprocessing, the search bounds and the rule priorities that order the whole.

Two term-level chapters follow. Huet-style pre-unification, its three cases and the decidable fragments worth detecting are covered in Chapter 4. The orientation order used to gate rewriting, its parameters and the properties demanded of it by the deferred-substitution discipline are covered in Chapter 5.

The architecture is the subject of Chapter 6: the rigid variable problem and the local/global partition it forces, the supervision tree, the separation of pure branch logic from stateful shell, the evidence stream, global closure as constraint satisfaction, the message flow and iterative deepening. Its extensions, the two optional peer agents and proof reconstruction, are the subject of Chapter 7.

Measurements are reported in Chapter 8: the benchmark method and the scoring conventions, the baseline configuration, ablations of the agents, of the calculus rules and of the search control, scaling with the worker pool, the behaviour of the closure search under the two pool sizes, a second study on a structured problem set with the coverage it gives by model class, and threats to validity. Worked examples are collected in Appendix A, and Chapter 9 summarises, states the limitations that survive the evaluation, and sets out future work.

The principal findings, for a reader of this chapter alone: global closure through the contradiction agent is what the prover rests on, and removing it costs about half the refutations; the concurrent search multiplies inference without multiplying closure, so the worker pool reduces latency without extending coverage; and the calculus decides most of the problems whose validity needs only conversion and Boolean extensionality and few of those needing a functional principle, which locates the missing rule rather than a general weakness.

Preliminaries

2.1 Notation

We abbreviate a sequence $a_1, \dots, a_n$ by $\bar{a}_n$ where $n \geq 0$, write $()$ for the empty sequence, and write $\bar{a}, \bar{b}$ for concatenation. We drop the length subscript and write $\bar{a}$ when it is irrelevant or clear from context, and write $|\bar{a}|$ for the length of $\bar{a}$. $\bar{n}$ is the sequence $1, \dots, n$ and $\{\bar{a}\}$ is the set $\{a_1, \dots, a_n\}$. The reverse of a sequence $\bar{a}_n$ is denoted $\bar{a}_n^{\text{rev}}$. We denote the sequence $a_i, \dots, a_j$ by $\bar{a}_{i..j}$ which we identify with $()$ when $i \geq j$.

Under $\lambda\bar{\tau}_n$, index $n + 1 - i$ denotes τ_i , so a head applied to its own binders takes $\bar{n}$.

Relations and functions on sequences are interpreted pointwise, so $\bar{a}_n R \bar{b}_n$ stands for $a_1 R b_1, \dots, a_n R b_n$ and $f(\bar{a}_n)$ for $f(a_1), \dots, f(a_n)$.

Note that parsing uses TPTP syntax [Sut17], in the typed higher-order dialects TH0 [SB10] and TH1 [KSR16], which differs from the notation established in this chapter. The higher-order TPTP dialects use *currying* and an *explicit application* symbol $@$ where we will use conventional application notation $(f(x, y))$ instead of $f @ x @ y$. Everything developed here is a more direct representation of the prover's internal data structures. The same goes for the formatting API of the prover, which uses more conventional type annotation (you may see $f_{\iota \rightarrow \iota \rightarrow \iota}$ instead of $f : (\iota, \iota) \rightarrow \iota$ in the interactive outputs). This is a deliberate choice as this thesis is a system description more than it is developing a logic or calculus. The prover's output has a different aim, which is for teaching or visualisation of classical higher-order logic.

Three registers run through every chapter and are kept typographically distinct. **Definition** and **Proposition** blocks are the mathematics: implementation-independent, and the only material a soundness or completeness argument may appeal to. Ordinary prose motivates and explains, and carries no logical weight. Ruled paragraphs are *implementation notes*, recording what the SHOT packages do, which module does it and which parameter governs it; a note may narrow a definition without the definition changing. Executable cells and quoted code belong to the third register as well: where the code is the precise statement, it is run rather than paraphrased.

Sigils ($\sim f$, $\sim g$, $\sim t$, $\sim e$) enable quick parsing of TPTP notation into our internal data structures.

2.2 Types

Simple Type Theory (STT), in the formulation of Church [Chu40], is a language of *terms* and defines the semantics of the transformation of higher-order terms. It defines computation on a single reduction principle, β -reduction. An efficient implementation of STT is hence most relevant for the performance of a prover for

higher-order logic. Its type layer is fixed in this section and its term layer in the sections that follow.

Definition 2.1 (Base Type, Type). Let $\mathcal{S} = \{\iota, o\}$ be the set of *sorts* and let $\mathcal{A}$ be a countably infinite set of *type variables* (α). The set of *base types* (ς) is $\mathcal{B} = \mathcal{S} \uplus \mathcal{A}$. The set of *types* $\mathcal{T}$ (τ, ν, ρ), is defined inductively: if $\bar{\tau}_n \in \mathcal{T}$ with $n \geq 0$ and $\varsigma \in \mathcal{B}$, then $\bar{\tau}_n \rightarrow \varsigma \in \mathcal{T}$. We identify $() \rightarrow \varsigma$ with ς .

Proposition 2.2 (Canonical Decomposition). *Every $\tau \in \mathcal{T}$ is of the form $\bar{\tau}_n \rightarrow \varsigma$ for a unique $n \geq 0$, a unique sequence $\bar{\tau}_n$ and a unique $\varsigma \in \mathcal{B}$.*

Proof. The type grammar has a single production, whose goal component is required to lie in $\mathcal{B}$, so a type is an argument sequence paired with a base type and n is the length of that sequence. Distinct pairs are distinct types, and the identification of $() \rightarrow \varsigma$ with ς places each base type at $n = 0$ rather than giving it a second decomposition. $\square$

Definition 2.3 (Arity, Goal). For $\tau = \bar{\nu}_n \rightarrow \varsigma$ we call n the *arity* $\text{ar}(\tau)$ of τ and ς its *goal* $\text{gl}(\tau)$.

Definition 2.4 (*o*-Type, Pure *o*-Type). A type τ is an *o*-type if $\text{gl}(\tau) = o$, so that a term of type τ is a formula once it is fully applied. The set $\mathcal{O} \subseteq \mathcal{T}$ of *pure o*-types is the least set with $o \in \mathcal{O}$ and $\bar{\tau}_n \rightarrow o \in \mathcal{O}$ whenever every $\tau_i \in \mathcal{O}$.

Proposition 2.5 (*o*-Types and Pure *o*-Types). *Every pure o-type is an o-type, and the inclusion is strict: $\iota \rightarrow o$ is an o-type and is not pure.*

Proof. Induction on membership in $\mathcal{O}$: both clauses produce a type whose goal is o . For strictness, $\text{gl}(\iota \rightarrow o) = o$, so $\iota \rightarrow o$ is an *o*-type, and it lies in $\mathcal{O}$ only if ι does, whereas every element of $\mathcal{O}$ has goal o and $\text{gl}(\iota) = \iota$. $\square$

The two notions do different work later and are not interchangeable. The weaker one selects the quantifiers a primitive substitution may be scheduled for (Section 3.4), where all that is required is that instantiating the variable eventually yields something of type o ; the argument types are unconstrained. The pure one selects the types whose domain is finite and enumerable (Section 3.3), which is what allows quantification over them to expand linearly and what bounds the case analysis of the Boolean extensionality rules.

Definition 2.6 (Type Substitution). A *type substitution* ϑ is a map from a finite subset of $\mathcal{A}$ to $\mathcal{T}$. Its application $\tau[\vartheta]$ is defined by recursion on τ :

$$\varsigma[\vartheta] = \begin{cases} \theta(\varsigma) & \varsigma \in \text{dom}(\vartheta) \\ \varsigma & \text{otherwise} \end{cases}$$

$$(\bar{\tau}_n \rightarrow \varsigma)[\vartheta] = \bar{\tau}_n[\vartheta], \bar{\nu} \rightarrow \varsigma' \quad \text{where } \varsigma[\vartheta] = \bar{\nu} \rightarrow \varsigma' \text{ and } n \geq 1$$

We write $[\bar{\alpha}_n \mapsto \bar{\rho}_n]$ for the substitution mapping α_i to ρ_i .

Proposition 2.7 (Instantiation and Arity). *$\text{ar}(\tau[\vartheta]) \geq \text{ar}(\tau)$, with equality if and only if $\text{gl}(\tau)[\vartheta] \in \mathcal{B}$.*

```

1  defmodule Ch2.Type do
2    @enforce_keys [:goal]
3    defstruct [:goal, args: []]
4    @type type_id() :: concrete_id() | variable_id()
5
6    @type concrete_id() :: atom()
7    @type variable_id() :: reference()
8
9    @type t :: %__MODULE__{goal: type_id(), args: [t()]}
10 end

```

Listing 2.1: The Type data structure

```

1  defmodule Ch2.TypeScheme do
2    @enforce_keys [:body]
3    defstruct [:body, vars: []]
4
5    @type t :: %__MODULE__{vars: [Ch2.Type.variable_id()], body: Ch2.Type.t()}
6  end

```

Listing 2.2: The TypeScheme data structure

Proof. Write $\tau = \bar{\tau}_n \rightarrow \varsigma$ by Canonical Decomposition, so that $\text{ar}(\tau) = n$ and $\text{gl}(\tau) = \varsigma$. The substitution clause appends the argument sequence of $\varsigma[\vartheta]$ to $\bar{\tau}_n[\vartheta]$, giving $\text{ar}(\tau[\vartheta]) = n + \text{ar}(\varsigma[\vartheta])$, and the same identity holds at $n = 0$, where $\tau[\vartheta] = \varsigma[\vartheta]$. Arities are non-negative, which gives the inequality, and equality holds exactly when $\text{ar}(\varsigma[\vartheta]) = 0$, by Canonical Decomposition exactly when $\varsigma[\vartheta] \in \mathcal{B}$. $\square$

Our implementation defines the recursive data structure for a Type of Listing 2.1. We use a unique Erlang reference as identifier for variables.

Definition 2.8 (Type Scheme). A *type scheme* $\sigma \in \mathfrak{T}$ has the form $[\bar{\alpha}].\tau$ with $n \geq 0$, where $\bar{\alpha}$ are pairwise distinct and every type variable occurring in τ is among $\bar{\alpha}$. Quantification occurs only at the outermost position. A type ρ is an *instance* of σ if $\rho = \tau[\bar{\alpha}_n \mapsto \bar{\rho}_n]$ for some $\bar{\rho}_n \in \mathcal{T}$.

A TypeScheme is implemented as a wrapper around a Type, explicitly pinning down quantified type variables, as Listing 2.2 shows.

For example, the type $(\iota, (\iota \rightarrow o)) \rightarrow o$ is represented as follows:

```

%Type{args: [%Type{goal: :i, args: []}, %Type{goal: :o, args: [%Type{goal: :i, args: []}]}], goal: :o}

```

which is a direct, nested mirror of the mathematical decomposition: the outer struct's goal is the type's goal o , and its args list holds one child struct per argument type, here ι and $\iota \rightarrow o$ in order.

2.3 Terms

Definition 2.9 (Signature). A *signature* Σ is a finite set of declarations $c : \sigma$ with $\sigma \in \mathfrak{Z}$, in which every constant symbol c is declared at most once.

Definition 2.10 (Free Variable, Parameter). Let $\mathcal{V} = \bigsqcup_{\tau \in \mathcal{T}} \mathcal{V}_\tau$ and $\mathcal{P} = \bigsqcup_{\tau \in \mathcal{T}} \mathcal{P}_\tau$ be families of countably infinite, pairwise disjoint sets with $\mathcal{V} \cap \mathcal{P} = \emptyset$. Elements of $\mathcal{V}$ are *free variables* (X, Y); elements of $\mathcal{P}$ are *parameters* (p, q).

Definition 2.11 (Term). Given a signature Σ , the set of *terms* $\Lambda(\Sigma)$ (s, t, u, v) is defined by

$$\Lambda(\Sigma) \ni s, t := c(\bar{\rho}) \mid X \mid p \mid i \mid \lambda \bar{t}. t \mid s(\bar{t})$$

where $c \in \Sigma$ is a constant symbol, $\bar{\rho}, \bar{t} \in \mathcal{T}$ with $n \geq 0$, $X \in \mathcal{V}$, $p \in \mathcal{P}$, and $i \in \mathbb{N}_{>0}$ is a de Bruijn index [dB72]. We identify $\lambda().t$ with t and $s()$ with s , and further identify $\lambda \bar{t}. \lambda \bar{v}. t$ with $\lambda \bar{t}. \bar{v}. t$.

Constants, variables and parameters are all implemented as `Declaration`. Note that we merge the definitions of constants and parameters and give parameters a unique reference as name. We also make the name of bound variables explicit and refer to them by their local index. Listing 2.3 gives the structure.

Terms as the main data structure require more thought in their representation to optimise both space and time efficiency. We opt for *hash-consed directed acyclic graphs* (DAGs) [FC06] and rely on the Erlang term storage (ETS), a heavily optimised global hash table, for efficient and concurrency-safe lookup. Our term representation, in Listing 2.4, features 5 data fields and 4 metadata fields, optimised for caching and minimising recursive checks.

A constructed term shows the representation directly. Note that `:args` holds term IDs rather than nested terms, so the DAG is flat in the struct and unfolds only through the factory: for $t = (p(fa)) \wedge (q(fa))$, the top-level struct carries `id: 7426`, head the declaration for $\wedge$, and `args: [6594, 7298]` — two term *identifiers*, not two nested term structs. The left conjunct, term 6594, is itself $p(f(a))$: it carries head the declaration for p and `args: [6466]`, again an identifier rather than a nested term.

Both conjuncts reference the same node for $f(a)$, which is memoised once: comparing the argument lists of the left and right conjuncts, `argsleft = argsright`, evaluates to `true`.

Terms are constructed globally, via the `ShotDs.Stt.TermFactory` module in a single ETS table (`:term_pool`). Term IDs may be *global* or *local*. Global term IDs are stored in the `:term_pool` while local IDs only persist in a process-local ETS table, the *scratchpad*. Garbage collection is implemented so that result terms (and transitively, their argument terms) are committed to the `:term_pool` while the scratchpad with all its entries is deleted. Garbage collection can be enabled by wrapping the code block in `with_scratchpad(fn -> ... end)`. The `:term_pool` table holds two shapes of row, interactively browsable during development: a *signature* row keyed on a term's structural signature (its head declaration, bound variables, argument IDs and type, packed into a tuple) mapping to that term's ID, and a *lookup* row keyed on a

```

1  defmodule Ch2.Declaration do
2    @enforce_keys [:kind, :name, :type]
3    defstruct [:kind, :name, :type]
4
5    @type t :: free_var_t() | bound_var_t() | const_t()
6
7    @type free_var_t :: %__MODULE__{
8      kind: :fv,
9      name: var_name_t(),
10     type: Ch2.Type.t()
11   }
12   @type bound_var_t :: %__MODULE__{
13     kind: :bv,
14     name: pos_integer(),
15     type: Ch2.Type.t()
16   }
17   @type const_t :: %__MODULE__{
18     kind: :co,
19     name: const_name_t(),
20     type: Ch2.Type.t()
21   }
22
23   @type var_name_t :: String.t() | reference() | pos_integer()
24   @type const_name_t :: String.t() | reference()
25 end

```

Listing 2.3: The Declaration data structure

term ID mapping to the term struct itself. Of its 36 entries at this point in the chapter, two representative rows are

```

{6466, %Term{id: 6466, head: f, args: [6338], type: %Type{goal: :i, args:
  []}, ...}}
{7426, %Term{id: 7426, head: ^, args: [6594, 7298], type: %Type{goal: :o,
  args: []}, ...}}

```

the first the lookup row for $f(a)$ (term 6466, the shared node above), the second the lookup row for t itself.

Treating terms as global DAGs lets us write efficient canonical higher-order functions such as `map_term` or `fold_term` using term IDs as keys for caching, as in Listing 2.5.

2.4 Typing

Definition 2.12 (Context). A *context* Γ is a finite sequence of types $\bar{\tau}$, listed from outermost to innermost binding. Lookup $\Gamma(i)$ is defined by

$$(\Gamma, \tau)(1) = \tau \qquad (\Gamma, \tau)(i + 1) = \Gamma(i)$$

```

1  defmodule Ch2.Term do
2    @enforce_keys [:id, :head, :type]
3    defstruct [
4      # Data fields
5      :id,
6      :head,
7      :type,
8      bvars: [],
9      args: [],
10     # Metadata fields
11     fvars: MapSet.new(),
12     consts: MapSet.new(),
13     tvars: MapSet.new(),
14     max_num: 0
15   ]
16
17   @type global_term_id :: pos_integer()
18   @type local_term_id :: neg_integer()
19   @type dummy_term_id :: 0
20
21   @type term_id :: global_term_id() | local_term_id() | dummy_term_id()
22
23   @type t :: %__MODULE__{
24     # Data fields
25     id: term_id(),
26     bvars: [Ch2.Declaration.t()],
27     head: Ch2.Declaration.t(),
28     args: [term_id()],
29     # Metadata fields
30     type: Ch2.Type.t(),
31     fvars: MapSet.t(Ch2.Declaration.t()),
32     consts: MapSet.t(Ch2.Declaration.const_t()),
33     tvars: MapSet.t(Ch2.Type.variable_id()),
34     max_num: non_neg_integer()
35   }
36 end

```

Listing 2.4: The Term data structure

Definition 2.13 (Typing). The typing judgement $\Gamma \vdash_{\Sigma} t : \tau$ is defined by the following rules; we drop the subscript Σ when it is fixed.

$$\begin{array}{c}
\frac{c : [\tilde{\alpha}_n]. \tau \in \Sigma \quad \bar{\rho}_n \in \mathcal{T}}{\Gamma \vdash c(\bar{\rho}_n) : \tau[\tilde{\alpha}_n \mapsto \bar{\rho}_n]} \quad
\frac{X \in \mathcal{V}_{\tau}}{\Gamma \vdash X : \tau} \quad
\frac{p \in \mathcal{P}_{\tau}}{\Gamma \vdash p : \tau} \quad
\frac{\Gamma(i) = \tau}{\Gamma \vdash i : \tau} \\
\\
\frac{\Gamma, \bar{\tau} \vdash t : \bar{v} \rightarrow \zeta}{\Gamma \vdash \lambda \bar{\tau}. t : (\bar{\tau}, \bar{v}) \rightarrow \zeta} \quad
\frac{\Gamma \vdash s : \bar{\tau}_n \rightarrow \zeta \quad \Gamma \vdash \bar{t}_n : \bar{\tau}_n}{\Gamma \vdash s(\bar{t}_n) : \zeta}
\end{array}$$

```

1  defmodule Ch2.TermTraversal do
2    @spec map_term!(
3      term_id :: Ch2.Term.term_id(),
4      env :: a,
5      update_env :: (Ch2.Term.t(), a -> a),
6      transform :: (Ch2.Term.t(), [Ch2.Term.term_id()], a, map() ->
7        ↪ {Ch2.Term.term_id(), map()}),
8      short_circuit :: (Ch2.Term.t(), a -> boolean()),
9      cache :: map()
10   ) :: {Ch2.Term.term_id(), map()}
11   when a: var
12   def map_term!(
13     term_id,
14     env,
15     update_env,
16     transform,
17     short_circuit \\ fn _, _ -> false end,
18     cache \\ %{}
19   ) do
20     case Map.fetch(cache, {term_id, env}) do
21       {:ok, cached_id} -> {cached_id, cache}
22       :error ->
23         term = StubTF.get_term!(term_id)
24         if short_circuit.(term, env) do
25           {term_id, Map.put(cache, {term_id, env}, term_id)}
26         else
27           new_env = update_env.(term, env)
28           arg_map_fn = &map_term!(&1, new_env, update_env, transform,
29             ↪ short_circuit, &2)
30           {new_args, cache} = Enum.map_reduce(term.args, cache, arg_map_fn)
31           {new_id, cache} = transform.(term, new_args, new_env, cache)
32           {new_id, Map.put(cache, {term_id, env}, new_id)}
33         end
34     end
35   end
36 end

```

Listing 2.5: A caching term traversal combinator

A term t is *well-typed* in Γ if $\Gamma \vdash t : \tau$ holds for some τ . From here on, we only consider well-typed terms. We write $t : \tau$ for the term t for which $\Gamma \vdash t : \tau$ for some global context Γ .

Proposition 2.14 (Unique Typing). *If $\Gamma \vdash t : \tau$ and $\Gamma \vdash t : \tau'$, then $\tau = \tau'$.*

Proof. Induction on t . Each typing rule is determined by the shape of t , so at most one applies to a given term. For a head the conclusion is read off data fixed independently of the derivation: the signature entry together with the type arguments for $c\langle\bar{\rho}_n\rangle$, the

type index of $\mathcal{V}$ or $\mathcal{P}$ for X and p , and the context lookup for i . For $\lambda\bar{t}.t$ and $s(\bar{t}_n)$ the conclusion is a function of the premise types, unique by the induction hypothesis, and of the split of the premise type into arguments and goal, unique by Canonical Decomposition. $\square$

2.5 Shifting and Substitution

Definition 2.15 (Shifting). For $d \geq 0$ and cutoff $k \geq 0$, the *shift* $t \uparrow_k^d$ is defined by

$$i \uparrow_k^d = \begin{cases} i + d & i > k \\ i & i \leq k \end{cases} \quad (\lambda\bar{t}.t) \uparrow_k^d = \lambda\bar{t}.t \uparrow_{k+n}^d \quad (s(\bar{t})) \uparrow_k^d = (s \uparrow_k^d)(\bar{t} \uparrow_k^d)$$

together with $c(\bar{\rho}) \uparrow_k^d = c(\bar{\rho})$, $X \uparrow_k^d = X$ and $p \uparrow_k^d = p$. We write $t \uparrow^d$ for $t \uparrow_0^d$.

Definition 2.16 (Substitution). The simultaneous substitution of a block $\bar{u}_n$ for the innermost n indices, $t[\bar{u}_n]$, is $t[\bar{u}_n]_0$ where

$$i[\bar{u}_n]_k = \begin{cases} u_{n+1-(i-k)} \uparrow^k & k < i \leq k+n \\ i-n & i > k+n \\ i & i \leq k \end{cases}$$

$$(\lambda\bar{t}_m.t)[\bar{u}_n]_k = \lambda\bar{t}_m.t[\bar{u}_n]_{k+m}$$

together with $(s(\bar{t}_m))[\bar{u}_n]_k = (s[\bar{u}_n]_k)(\bar{t}_m[\bar{u}_n]_k)$, and with constants, free variables and parameters left unchanged.

Definition 2.17 (Free-Variable Substitution). A *free-variable substitution* θ is a map from a finite subset of $\mathcal{V}$ to $\Lambda(\Sigma)$ such that $\theta(X)$ has type τ for every $X \in \mathcal{V}_\tau \cap \text{dom}(\theta)$. Its application $t\theta$ is $t\theta_0$ where

$$X\theta_k = \begin{cases} \theta(X) \uparrow^k & X \in \text{dom}(\theta) \\ X & \text{otherwise} \end{cases} \quad (\lambda\bar{t}_n.t)\theta_k = \lambda\bar{t}_n.t\theta_{k+n}$$

together with $(s(\bar{t}))\theta_k = (s\theta_k)(\bar{t}\theta_k)$, and with constants, parameters and indices left unchanged.

Shifting and substitution are efficiently implemented on top of the `map_term` combinator. Both make use of the `env` field to store the current index depth and use a term's `max_num` and `fvars` fields to skip unnecessary recursive calls (as `short_circuit`).

2.6 Reduction and Normal Forms

Definition 2.18 (β -Reduction). $(\lambda\bar{t}_n.t)(\bar{u}_n, \bar{v}_m) \rightarrow_\beta t[\bar{u}_n](\bar{v}_m)$

Definition 2.19 (Head). A *head* (h) is a term of the form $c(\bar{\rho}_n)$, X , p or i .

Definition 2.20 (Long Normal Form). A well-typed term t with $\Gamma \vdash t : \tau$ is in η -long β -normal form if $t = \lambda\bar{u}_n.h(\bar{u}_m)$ where $n = \text{ar}(\tau)$ and each u_j is in η -long β -normal form. In the following, we only consider terms that are in η -long β -normal form and assume that they are always β -reduced and η -expanded after each transformation.

The internals of the `TermFactory` and `Semantics` API ensure terms to always be well-typed and in η -long β -normal form.

2.7 Parameter Terms

Definition 2.21 (Parameter Term). Let $\bar{X} \in \mathcal{V}$ be pairwise distinct with $\bar{X} \in \mathcal{V}_{\bar{\tau}}$, and let $p \in \mathcal{P}_{(\bar{\tau}, \bar{v}_n) \rightarrow \zeta}$. The *parameter term* of p over $\bar{X}$ is

$$p\{\bar{X}\} = \lambda \bar{v}_n. p(\bar{X}, \bar{n})$$

We call $\bar{X}$ the *dependencies* of $p\{\bar{X}\}$, and p *fresh* for a set of terms T if $p \notin \text{par}(T)$.

Proposition 2.22 (Parameter Term). $\Gamma \vdash p\{\bar{X}\} : \bar{v} \rightarrow \zeta$, and $p\{\bar{X}\}$ is in η -long β -normal form.

Proof. Under the context $\Gamma, \bar{v}_n$ the parameter rule gives $p : (\bar{\tau}, \bar{v}_n) \rightarrow \zeta$, the variable rule gives $X_i : \tau_i$, and lookup gives the reversed indices $\bar{n}$ the types $\bar{v}_n$, the context being listed outermost to innermost. The application rule yields $\Gamma, \bar{v}_n \vdash p(\bar{X}, \bar{n}) : \zeta$ and the abstraction rule $\Gamma \vdash \lambda \bar{v}_n. p(\bar{X}, \bar{n}) : \bar{v}_n \rightarrow \zeta$. The term is in η -long β -normal form because its head is the parameter p , its arguments are in that form by the standing convention on terms, and the number of binders is $n = \text{ar}(\bar{v}_n \rightarrow \zeta)$. $\square$

A parameter is created by `make_fresh_const_term/1`, which builds a `Declaration` carrying a unique reference as its name at type $(\bar{\tau}_n, \bar{v}_m) \rightarrow \zeta$. Since `make_term/1` η -expands any declaration of arrow type on construction, the abstraction $\lambda \bar{v}_m$ is produced by the term factory rather than written by the caller; applying the result to the dependencies $\bar{X}_n$ then β -reduces the outer n binders away. No shift on $\bar{X}_n$ is needed, as shifting acts as the identity on free variables. For $m = 0$ the parameter term collapses to $p(\bar{X}_n)$, and for $n = 0$ to the η -long form of p itself.

2.8 Syntactic Equality and Unification

Definition 2.23 (Syntactic Equality). Two terms s and t are *syntactically equal* if they are the same element of $\Lambda(\Sigma)$.

Because terms are hash-consed into a single global pool, syntactic equality is decided by comparing term IDs. Together with de Bruijn indices and the η -long β -normal invariant, this makes α -, β - and η -equivalence all coincide with identity of IDs.

Definition 2.24 (Flex, Rigid). A term is *flex* if its head is a free variable and *rigid* otherwise. A *unification problem* is a finite set of equations $\{\bar{s}_n \stackrel{?}{=} \bar{t}_n\}$; an equation is *flex-flex* if both of its sides are flex. A *unifier* of a unification problem is a free-variable substitution θ such that $s_i\theta$ and $t_i\theta$ are syntactically equal for every i .

Definition 2.25 (Pre-Unification Problem). A *pre-unifier* [Hue75] of a unification problem E is a pair $\Theta = (\theta, C)$, where θ is a free-variable substitution and C is a finite set of flex-flex equations, such that $\theta\theta'$ is a unifier of E for every unifier θ' of C . The *pre-unification problem* for E is the task of enumerating pre-unifiers of E .

A pre-unifier is represented as `%ShotUn.UnifSolution{substitutions: theta, flex_pairs: C}` and pre-unifiers are enumerated lazily as a stream. Flex-flex equations are always solvable, so a pre-unifier witnesses solvability of E without committing to a solution for C .

2.9 Connectives, Quantifiers and Equality

Classical higher-order logic (HOL) is defined as an extension of STT and is the language we will be working with. Textbook treatments are given by Andrews [And02], and the model classes lying between Henkin and standard semantics are laid out by Benzmüller, Brown and Kohlhase [BBK04]. The signature, the semantics and the notion of entailment the prover decides are fixed in this section and the four that follow it.

Definition 2.26 (Signature of HOL). Σ^{HOL} contains the following symbols:

$$\begin{array}{lll} \top, \perp : o & \neg : o \rightarrow o & \vee, \wedge, \supset, \equiv : (o, o) \rightarrow o \\ \forall, \exists : [\alpha]. (\alpha \rightarrow o) & = : [\alpha]. (\alpha, \alpha) \rightarrow o & \end{array}$$

We write $\varphi * \psi$ instead of $*(\varphi, \psi)$ for $* \in \{\wedge, \vee, \supset, \equiv, =\}$ and $Q\bar{\tau}_n.\varphi$ instead of $Q(\lambda\tau_1. \dots Q(\lambda\tau_n. \varphi))$ for $Q \in \{\forall, \exists\}$. We call terms from $\Lambda(\Sigma^{\text{HOL}})$ *HOL terms*, those of type o *formulae* (φ, ψ, χ) , and those of type $\tau \rightarrow o$ for $\tau \in \mathcal{T}$ *properties* (Φ, Ψ) .

Definition 2.27 (Complement). The *complement* $\sim\varphi$ of a formula φ is χ if $\varphi = \neg\chi$, and $\neg\varphi$ otherwise.

2.10 Frames and Interpretations

Classical HOL is typically discussed for *curried* terms, i.e. $f(x, y)$ represented as $f x y$. To discuss the semantics of HOL in the notation introduced in this chapter, we borrow the notion of *sections* from set theory. This allows us to describe *partial application*.

Definition 2.28 (Frame). A *frame* $\mathcal{D} = (\mathcal{D}_\tau)_{\tau \in \mathcal{T}}$ is a collection of nonempty sets $\mathcal{D}_\tau$ such that $\mathcal{D}_o = \{\text{T}, \text{F}\}$, each $\mathcal{D}_{\bar{\tau}_n \rightarrow \zeta}$ is a set of functions mapping $\overline{\mathcal{D}_{\bar{\tau}_n}}$ to $\mathcal{D}_\zeta$, and the collection is closed under *sections*: for $f \in \mathcal{D}_{(\bar{\tau}_n, \bar{v}) \rightarrow \zeta}$ and $\bar{a}_n \in \overline{\mathcal{D}_{\bar{\tau}_n}}$, the section $f_{\bar{a}_n}$ given by $f_{\bar{a}_n}(\bar{b}) = f(\bar{a}_n, \bar{b})$ lies in $\mathcal{D}_{\bar{v} \rightarrow \zeta}$. The frame is *standard* if every $\mathcal{D}_{\bar{\tau}_n \rightarrow \zeta}$ is the full space of such functions.

Sections are what make partial application meaningful here. Types are uncurried with a base goal, so an element of $\mathcal{D}_{(\tau, \bar{v}) \rightarrow \zeta}$ is a function of $1 + |\bar{v}|$ arguments and supplying one of them yields nothing in the frame unless the frame is asked to contain the result. Where no confusion arises we allow $s(a)$ as a synonym for s_a , and correspondingly read a term $s(t)$ whose head is under-applied as the η -long form $\lambda\bar{v}. s(t, \bar{v})$.

Definition 2.29 (Interpretation). An *interpretation* $\mathcal{I}$ over a frame $\mathcal{D}$ and a signature $\Sigma \supseteq \Sigma^{\text{HOL}}$ maps every type instance $c\langle\bar{p}\rangle : v$ of a constant $c \in \Sigma$, and every parameter $p \in \mathcal{P}_v$, to an element of $\mathcal{D}_v$, subject to the conditions below on the logical constants.

Each condition constrains $\mathcal{J}$ and at the same time demands of $\mathcal{D}$ that the required element be present in it.

- $\mathcal{J}(\top) = \text{T}$ and $\mathcal{J}(\perp) = \text{F}$; $\mathcal{J}(\neg)$ maps T to F and F to T; and $\mathcal{J}(\wedge)$, $\mathcal{J}(\vee)$, $\mathcal{J}(\supset)$, $\mathcal{J}(\equiv)$ are the corresponding Boolean functions on $\{\text{T}, \text{F}\}$.
- *Equality*. For every $\tau \in \mathcal{T}$ and $a, b \in \mathcal{D}_\tau$, $\mathcal{J}(=\langle \tau \rangle)(a, b) = \text{T}$ if and only if a and b are the same element of $\mathcal{D}_\tau$.
- *Quantifiers*. For every $\tau \in \mathcal{T}$ and every $f \in \mathcal{D}_{\tau \rightarrow o}$, $\mathcal{J}(\forall \langle \tau \rangle)(f) = \text{T}$ if and only if $f(a) = \text{T}$ for every $a \in \mathcal{D}_\tau$, and $\mathcal{J}(\exists \langle \tau \rangle)(f) = \text{T}$ if and only if $f(a) = \text{T}$ for some $a \in \mathcal{D}_\tau$.

Equality is fixed to identity at every type, which is the property that makes an interpretation *logical* in the sense of Benz Müller, Brown and Kohlhasse [BBK04]. The two extensionality principles a higher-order calculus needs are then consequences rather than stipulations, and are stated as such below; the one principle that is *not* a consequence, Leibniz equality, is separated out with the condition it actually requires.

2.11 Denotation and Models

Definition 2.30 (Variable Assignment). A *variable assignment* g maps every free variable $X \in \mathcal{V}_\tau$ to an element $g(X) \in \mathcal{D}_\tau$.

Definition 2.31 (Denotation). Let $\mathcal{D}$ be a frame, $\mathcal{J}$ an interpretation over it, $\mathcal{M}$ the pair $(\mathcal{D}, \mathcal{J})$, g a variable assignment and $\tilde{\xi}_k$ a sequence of elements of $\mathcal{D}$ valuing the outermost k de Bruijn indices. The *denotation* of a term $t : \tau$, written $\llbracket t \rrbracket(\mathcal{M}, g, \tilde{\xi}_k)$, is the partially defined element of $\mathcal{D}_\tau$ given by

- $\llbracket c \langle \bar{\rho} \rangle \rrbracket(\mathcal{M}, g, \tilde{\xi}_k) = \mathcal{J}(c \langle \bar{\rho} \rangle)$ for constant symbols $c \in \Sigma$
- $\llbracket p \rrbracket(\mathcal{M}, g, \tilde{\xi}_k) = \mathcal{J}(p)$ for parameters $p \in \mathcal{P}$
- $\llbracket X \rrbracket(\mathcal{M}, g, \tilde{\xi}_k) = g(X)$ for free variables $X \in \mathcal{V}$
- $\llbracket i \rrbracket(\mathcal{M}, g, \tilde{\xi}_k) = \xi_i$ for de Bruijn indices $1 \leq i \leq k$
- $\llbracket s(\bar{t}_n) \rrbracket(\mathcal{M}, g, \tilde{\xi}_k) = (\llbracket s \rrbracket(\mathcal{M}, g, \tilde{\xi}_k)) \left(\overline{\llbracket t_i \rrbracket(\mathcal{M}, g, \tilde{\xi}_k)_n} \right)$
- $\llbracket \lambda \bar{t}_n. t \rrbracket(\mathcal{M}, g, \tilde{\xi}_k)$, for $t : \bar{v} \rightarrow \varsigma$, is the unique $f \in \mathcal{D}_{(\bar{t}_n, \bar{v}) \rightarrow \varsigma}$ whose sections satisfy $f_{\bar{d}_n} = \llbracket t \rrbracket(\mathcal{M}, g, (\bar{d}_n, \tilde{\xi}_k))$ for all $\bar{d}_n \in \overline{\mathcal{D}_{\bar{t}_n}}$, and is undefined when $\mathcal{D}$ contains no such f

Definition 2.32 (Henkin Model, Standard Model). A *Henkin model* [Hen50, And02] is a pair $\mathcal{M} = (\mathcal{D}, \mathcal{J})$ of a frame and an interpretation over it whose denotation is *total*, meaning that $\llbracket t \rrbracket(\mathcal{M}, g, \tilde{\xi}_k)$ is defined for every well-typed term t , every variable assignment g and every $\tilde{\xi}_k$ valuing t 's context. $\mathcal{M}$ is a *standard model* if $\mathcal{D}$ is standard.

Totality is the substantive condition in this definition. A frame may omit functions, so the demand is that it omit none whose absence would leave a term without a value; every standard model is a Henkin model, and the converse fails.

2.12 Extensionality and Leibniz Equality

Proposition 2.33 (Boolean Extensionality). $\mathcal{I}(=\langle o \rangle) = \mathcal{I}(\equiv)$.

Proof. $\mathcal{D}_o = \{T, F\}$, so two elements are the same element exactly when both are T or both are F, which is the graph of $\mathcal{I}(\equiv)$. $\square$

Proposition 2.34 (Functional Extensionality). *Let $s, t \in \mathcal{D}_{(\tau, \bar{v}) \rightarrow \zeta}$. Then $\mathcal{I}(=\langle (\tau, \bar{v}) \rightarrow \zeta \rangle)(s, t) = T$ if and only if $\mathcal{I}(=\langle \bar{v} \rightarrow \zeta \rangle)(s_a, t_a) = T$ for every $a \in \mathcal{D}_\tau$.*

Proof. Elements of a frame at arrow type are functions, and two functions are the same function exactly when all their sections agree. Both sides therefore say that s and t are the same element. $\square$

Definition 2.35 (Separation). A frame *separates at τ* if for all distinct $a, b \in \mathcal{D}_\tau$ there is a $P \in \mathcal{D}_{\tau \rightarrow o}$ with $P(a) \neq P(b)$. It *separates* if it separates at every type.

Proposition 2.36 (Leibniz Equality). *Let $a, b \in \mathcal{D}_\tau$. If $\mathcal{I}(=\langle \tau \rangle)(a, b) = T$ then $P(a) = P(b)$ for every $P \in \mathcal{D}_{\tau \rightarrow o}$. The converse holds for all $a, b \in \mathcal{D}_\tau$ if and only if $\mathcal{D}$ separates at τ . Every standard frame separates.*

Proof. The first claim is substitution of identicals. The second is the definition of separation, contraposed. For the third, the characteristic function of $\{a\}$ is a member of a standard $\mathcal{D}_{\tau \rightarrow o}$ and separates a from every other element. $\square$

The three propositions are what Section 3.6's three equality-expansion rules appeal to, one each, and they do not all cost the same. Expansion at o and expansion at an arrow type are sound in every Henkin model, because they restate identity in a frame whose arrow components are functions. Leibniz expansion at a base type is sound in the direction that reads an equation as agreement of all properties, and its converse direction, the one a refutation needs when it expands $\neg(s = \langle \zeta \rangle t)$, holds only where the frame separates at ζ . Separation is therefore a condition on the model class rather than a consequence of the semantics, and the negative rule of that section is what makes it operational: it Skolemises the separating property into a fresh parameter rather than requiring the frame to supply it.

2.13 Satisfiability and Entailment

Definition 2.37 (Satisfiability). A formula φ is *satisfiable* iff there exists a Henkin model $\mathcal{M}$ and a variable assignment g , such that $\llbracket \varphi \rrbracket(\mathcal{M}, g, ()) = T$. A set of formulae is *satisfiable* if all its elements are jointly satisfiable under the same Henkin model and variable assignment. Conversely, a formula (or a set of formulae) is *unsatisfiable* if no such model and variable assignment exists.

Definition 2.38 (Entailment and Validity). A formula ψ is *entailed* by a set of formulae Φ , written $\Phi \models \psi$, if $\llbracket \psi \rrbracket(\mathcal{M}, g, ()) = T$ for every Henkin model $\mathcal{M}$ and variable assignment g under which every element of Φ denotes T. A formula φ is *valid*, written $\models \varphi$, iff $\emptyset \models \varphi$.

Proposition 2.39 (Refutational Reformulation). $\Phi \models \psi$ if and only if $\Phi \cup \{\neg\psi\}$ is *unsatisfiable*.

Proof. $\mathcal{I}(\neg)$ is Boolean complement and $\mathcal{D}_o$ has two elements, so under a fixed $\mathcal{M}$ and g exactly one of ψ and $\neg\psi$ denotes T. $\square$

The reformulation is the form the prover decides. A conjecture is refuted rather than proved: the negated conclusion joins the assumptions and the tableau of Chapter 3 attempts to show the resulting set unsatisfiable. The turnstile $\vdash$ is reserved for the typing judgement of this chapter and is not used for entailment.

Higher-Order Tableaux

3.1 Global and Local Tableau States

Definition 3.1 (Free Variable Tableau). A *free variable tableau* [Fit96, Häh01] for a finite formula set Φ (from here on just *tableau*) is a finitely branching tree whose nodes are labelled with formulae, built from a root path labelled by Φ by finitely many applications of the expansion rules of this chapter. A *branch* $\mathfrak{B}$ is a maximal path from the root; we identify $\mathfrak{B}$ with the set of formulae labelling it, read as the conjunction $\bigwedge \mathfrak{B}$. The tableau denotes $\bigvee_{\mathfrak{B}} \bigwedge \mathfrak{B}$ over its branches. Free variables introduced by γ -rules are *rigid*: they range over the whole tableau, so a substitution applies to every branch simultaneously.

Definition 3.2 (Literal). A formula on a branch is a *literal* if it is atomic or the negation of an atomic formula, where a formula is *atomic* when no expansion rule of this chapter applies to it. $\text{Lit}(\mathfrak{B})$ denotes the literals of $\mathfrak{B}$.

Definition 3.3 (Position, Replacement). Several rules below act at a distinguished position inside a term rather than at its root. The set of *positions* of a term u is given by $\varepsilon \in \text{Pos}(u)$ together with $i.\pi \in \text{Pos}(\lambda\bar{\tau}_n.h(\bar{u}_m))$ whenever $1 \leq i \leq m$ and $\pi \in \text{Pos}(u_i)$. The subterm $u|_{\pi}$ and the *replacement* $u[t]_{\pi}$ are defined by recursion on π , with $u|_{\varepsilon} = u$ and $u[t]_{\varepsilon} = t$. Replacement preserves well-typedness and η -long β -normal form whenever t has the type of $u|_{\pi}$; on the hash-consed representation it is a rebuild of the spine from π to the root, sharing everything else.

Definition 3.4 (Clash, Disagreement). Let φ and ψ be literals on a branch. They *clash* if $\sim\varphi$ and ψ are syntactically equal. They *disagree* if at least one of the unification problems $\{\sim\varphi \stackrel{?}{=} \psi\}$ and $\{\varphi \stackrel{?}{=} \sim\psi\}$ admits a pre-unifier.

A clash is decided by term-ID membership, so it is a hash-set lookup. Disagreement is decided by running `ShotUn.unify/2` on both orientations and testing whether the resulting stream is non-empty. Since that enumeration is bounded by `unification_depth` (8), disagreement as implemented is *bounded* pre-unifiability: a pair may be unifiable and still not be reported at a given depth. Iterative deepening (Section 3.10) is what recovers the missed pairs.

Definition 3.5 (Closure). A branch B is *closed under* θ , for a free-variable substitution θ , if some pair $\varphi, \psi \in \text{Lit}(\mathfrak{B})$ satisfies $(\sim\varphi)\theta = \psi\theta$. It is *closed unconditionally* (*ground closure*) if this holds for $\theta = \varepsilon$, and *conditionally closed* otherwise. A tableau $\mathfrak{T}$ is *closed* if there is a single θ such that every branch of $\mathfrak{T}$ is closed under θ ; rigidity of free variables is the requirement that a single θ close all branches.

Definition 3.6 (Saturation, Openness). A branch is *saturated* if no expansion rule applies to any of its formulae. A saturated branch is *open* if it is not closed under any pre-unifier. A tableau is *open* if it has an open branch.

Proposition 3.7 (Tableau-Proof). *If a tableau for Φ is closed, then Φ is unsatisfiable.*

Proof. By induction on the construction of the tableau, from the per-rule obligations collected in Section 3.11 and Definition (Closure), under which every branch of a closed tableau is unsatisfiable under the substitution of the closing pre-unifier. $\square$

The converse, that an open tableau witnesses satisfiability of Φ , is assumed rather than proved. An open branch is intended to describe a Henkin model, and the model-existence arguments for tableau calculi obtain one by extending a saturated open branch to a Hintikka set and reading a model off it [Fit96], in the higher-order setting over the model classes of [Koh95, BBK04]. That argument is not carried out here, so an open tableau for Φ is treated as evidence of satisfiability and not as a proof of it. The implementation narrows the claim further, as the note below records.

Ground closure terminates a branch immediately and locally. A disagreement, by contrast, only produces a *candidate* closing pair; the substitution that would realise it must be reconciled against the candidate pairs of every other open branch. That reconciliation is global and is deferred to a separate component, described in Chapter 6. Everything in this chapter is branch-local.

Saturation is decidable (the rule queue is empty) but openness is not, so the implementation reports only a decidable under-approximation of it. `Branch.model_certain?/2` calls a saturated branch open when every same-head, opposite-polarity literal pair on it is separated at some argument position by two distinct rigid base-type terms, the configuration a free term model satisfies by interpreting those terms as distinct elements. A pair such as $P(\lambda \dots)$ and $\neg P(\lambda \dots)$ with extensionally equal but syntactically different arguments fails that test: SHOT decides term equality only inside the fragment its equational machinery covers, namely identity, bounded unifiability, and the oriented equations of Section 3.6, so reporting such a branch as a countermodel would be unsound. A saturated branch failing the test is *exhausted*, a third outcome alongside open and closed, and the session answers :unknown for it.

3.2 Propositional Rules

Following Smullyan’s uniform notation [Smu68, Fit96], we distinguish between α -, β -, γ - and δ -rules for expansion.

Definition 3.8 (α -Rules). α -rules are expansion rules for linear decomposition. Applying one yields a list of (implicitly conjuncted) terms on the branch.

$$\begin{array}{cccc} \frac{\neg\neg\varphi}{\varphi}(\neg\neg) & \frac{\varphi \wedge \psi}{\varphi}(\wedge) & \frac{\neg(\varphi \vee \psi)}{\neg\varphi}(\neg\vee) & \frac{\neg(\varphi \supset \psi)}{\varphi}(\neg\supset) \\ & \psi & \neg\psi & \neg\psi \end{array}$$

Definition 3.9 (β -Rules). β -rules are branching expansion rules reading as disjunctions.

$$\begin{array}{ccc} \frac{\varphi \vee \psi}{\varphi \mid \psi}(\vee) & \frac{\neg(\varphi \wedge \psi)}{\neg\varphi \mid \neg\psi}(\neg\wedge) & \frac{\varphi \supset \psi}{\neg\varphi \mid \psi}(\supset) \end{array}$$

A variant of β -rules may be used which corresponds to the equivalence of $\varphi \vee \psi$ and $\varphi \vee (\psi \wedge \neg\varphi)$ ¹. When β -rules are of the shape:

$$\frac{\beta}{\beta_1 \mid \beta_2}(\beta)$$

they may also be processed as:

$$\frac{\beta}{\beta_1 \mid \begin{array}{c} \beta_2 \\ \neg\beta_1 \end{array}}(\beta\text{-var})$$

Definition 3.10 (Equivalence Expansion). Equivalence admits two expansion strategies. Under *same-polarity* expansion:

$$\frac{\varphi \equiv \psi}{\begin{array}{c} \varphi \mid \neg\psi \\ \psi \mid \neg\varphi \end{array}}(\equiv_{\text{sp}}) \qquad \frac{\neg(\varphi \equiv \psi)}{\begin{array}{c} \neg\varphi \mid \neg\psi \\ \psi \mid \varphi \end{array}}(\neg\equiv_{\text{sp}})$$

Under *bidirectional-implication* expansion:

$$\frac{\varphi \equiv \psi}{\begin{array}{c} \varphi \supset \psi \\ \psi \supset \varphi \end{array}}(\equiv_{\text{bi}}) \qquad \frac{\neg(\varphi \equiv \psi)}{\neg(\varphi \supset \psi) \mid \neg(\psi \supset \varphi)}(\neg\equiv_{\text{bi}})$$

The modes differ in rule class as well as in shape. Same-polarity expansion makes positive $\equiv$ a β -rule; bidirectional-implication makes it an α -rule and defers the split to the produced disjunctions. The prover's default is bidirectional implication. `Rules.classify_formula/3` carries its own default of same-polarity for direct calls, so a classification performed outside the prover will not agree with the prover unless the mode is passed explicitly.

3.3 Quantifier Rules

Definition 3.11 (γ -Rules). γ -rules instantiate universally quantified formulae with arbitrary terms of that type. In free-variable tableaux, these may also contain free variables ("holes") which defer instantiation to be found by unification. The rule is not consumed: each application produces an instance and reinserts the rule with its instantiation counter incremented.

$$\frac{\forall\tau. \Phi}{\Phi(X : \tau)}(\forall) \qquad \frac{\neg(\exists\tau. \Phi)}{\neg(\Phi(X : \tau))}(\neg\exists)$$

$$\forall\tau. \Phi \qquad \neg(\exists\tau. \Phi)$$

where X is fresh for the branch.

Two refinements sit on top of this. Under *instance-based* γ (`instance_based_gamma`, `on`), the first application of a rule additionally emits instances $\Phi(t)$ for ground terms $t : \tau$ already registered on the branch, and generates unit-set prim-subst bindings for the branch's constants. The per-firing ground fan-out is capped by `instance_based_`

¹Consider $\varphi \vee (\psi \wedge \neg\varphi) \Leftrightarrow (\varphi \vee \psi) \wedge (\varphi \vee \neg\varphi) \Leftrightarrow \varphi \vee \psi$

Table 3.1: The sixteen members of $\text{gen}((o, o) \rightarrow o)$, in the order `Generation.gen_o/1` enumerates them. Under $\lambda o, o$ the index 2 denotes the first argument and 1 the second.

1. $\lambda o, o. 2 \wedge 1$	5. $\lambda o, o. \neg(2 \wedge 1)$	9. $\lambda o, o. 2 \wedge \neg 1$	13. $\lambda o, o. 2$
2. $\lambda o, o. 2 \vee 1$	6. $\lambda o, o. \neg(2 \vee 1)$	10. $\lambda o, o. \neg 2 \wedge 1$	14. $\lambda o, o. 1$
3. $\lambda o, o. 2 \supset 1$	7. $\lambda o, o. 1 \supset 2$	11. $\lambda o, o. \top$	15. $\lambda o, o. \neg 2$
4. $\lambda o, o. 2 \equiv 1$	8. $\lambda o, o. \neg(2 \equiv 1)$	12. $\lambda o, o. \perp$	16. $\lambda o, o. \neg 1$

`gamma_limit`, whose default `0` suppresses it: nested quantifiers multiply branch constants combinatorially, and the cheap first- γ cost makes every IBG-derived rule fire ahead of β . Ground terms discovered later are picked up the next time the same rule fires under deepening. When the variable's type is an o -type, reaching `prim_subst_after` (1) instantiations schedules a primitive substitution rule (Section 3.4) at the same source, unless `primitive_substitution` is off.

Definition 3.12 (Propositional Domain). Let $\mathcal{O}$ be the pure o -types of Chapter 2. For $\tau \in \mathcal{O}$ the frame component $\mathcal{D}_\tau$ is finite, and in a standard frame

$$|\mathcal{D}_o| = 2 \quad |\mathcal{D}_{\bar{\tau}_n \rightarrow o}| = 2 \prod_{i=1}^n |\mathcal{D}_{\bar{\tau}_i}|$$

We write $\text{gen}(\tau) = \{b_1, \dots, b_k\}$ for a fixed enumeration of closed terms realising the elements of the standard $\mathcal{D}_\tau$. A Henkin frame's $\mathcal{D}_\tau$ is a subset of the standard one, so $\text{gen}(\tau)$ covers it in every model, which is what the rules below require of it.

Definition 3.13 (γ -Rules over Finite Domains). For $\tau \in \mathcal{O}$ with $\text{gen}(\tau) = \{b_1, \dots, b_k\}$, quantification expands linearly rather than through a free variable:

$$\frac{\forall \tau. \Phi}{\Phi(b_1)} (\forall_{\text{fin}}) \quad \frac{\neg(\exists \tau. \Phi)}{\neg \Phi(b_1)} (\neg \exists_{\text{fin}})$$

$$\vdots \quad \vdots$$

$$\Phi(b_k) \quad \neg \Phi(b_k)$$

`Generation.gen_o/1` performs the enumeration: 2 truth values at o , 4 unary connectives at $o \rightarrow o$, 16 binary connectives at $(o, o) \rightarrow o$, and higher arities by if-then-else expansion over the domain of the first argument. Results are memoised per process. Because the domain is exhausted in one step, the rule is consumed rather than reinserted, and the instances land on a single branch rather than splitting it. The routing is governed by `finite_o_quantification`; with it off, a pure- o quantifier is instantiated by the ordinary γ -rule like any other type. Table 3.1 lists the sixteen at $(o, o) \rightarrow o$.

Definition 3.14 (δ -Rules). δ -rules instantiate existentially quantified formulae with a parameter term depending on the free variables of the matrix (Skolemisation), in the liberalised form that lets the parameter depend on those free variables alone [HS94].

$$\frac{\exists \tau. \Phi}{\Phi(p\{\text{FV}(\Phi)\})} (\exists) \quad \frac{\neg(\forall \tau. \Phi)}{\neg(\Phi(p\{\text{FV}(\Phi)\}))} (\neg \forall)$$

where, for $\text{FV}(\Phi) = \bar{X}_n$ with $\bar{X}_n \in \mathcal{V}_{\bar{\tau}_n}$ and $\tau = \bar{v}_m \rightarrow \zeta$, the parameter $p \in \mathcal{P}_{(\bar{\tau}_n, \bar{v}_m) \rightarrow \zeta}$ is fresh, so that $p\{\bar{X}_n\} : \tau$.

This is the parameter term of Chapter 2 with its dependencies fixed to the free variables of the matrix. Dependence on the free variables rather than on the enclosing λ -context is what keeps the parameter sound under the global reconciliation of Chapter 6: a later substitution for some X_i propagates into the parameter's argument positions instead of invalidating it.

3.4 Primitive Substitution

The rule below instantiates the matrix of a γ -rule with *general bindings* over a fixed family of heads, in the sense of Andrews [And89].

Definition 3.15 (Primitive Substitution). Let Φ be the matrix of a γ -rule whose variable type is an o -type in the sense of Chapter 2, that is, a type of goal o with arbitrary argument types. A *general binding* for argument types $\bar{\alpha}$ is a term $\lambda \bar{\tau}_n. \mathcal{H}(H_1(\bar{n}), \dots, H_k(\bar{n}))$ where $\bar{H}_k$ are fresh free variables and $\mathcal{H}$ is drawn from a family of admissible heads. The *primitive substitution* rule instantiates Φ with each general binding in turn:

$$\frac{\forall \tau. \Phi}{\Phi(G_1) \mid \dots \mid \Phi(G_j)} (\text{prim})$$

Three head families are generated. The *propositional* family fixes a logical connective: $\top$, $\perp$, $\neg H$, and the four binary connectives over two holes; at depth ≥ 2 it is extended with five composed forms such as $\neg(H_1 \wedge H_2)$ and $\neg H_1 \wedge H_2$. The *polymorphic* family fixes $=\langle \tau \rangle$ over the branch's type universe, extended at depth ≥ 2 with composed equality heads and with $\forall/\exists$ heads, which are ordered last so proof search reaches them late. The *unit-set* family produces $\lambda \bar{\tau}_n. (H(\bar{n})) = \langle \tau \rangle c$ for each parameter c on the branch. The H holes are created in the global term pool rather than in a scratchpad, because a unifier found later must be traceable to the rule that introduced them.

3.5 Boolean Extensionality: Renaming and Instantiation

The remaining higher-order principle is *Boolean extensionality* [BBK04]: in a Henkin model $\mathcal{D}_o = \{\text{T}, \text{F}\}$, so two formulae denoting the same truth value are interchangeable in every context. A tableau built only from the rules above cannot use this. The rules decompose a formula's *logical skeleton*, but an atom $P(t)$ whose argument t has type o is opaque to them: t sits in an argument position, no connective governs it, and no rule looks inside. Yet t denotes T or F, and $P(\top)$ together with $P(\perp)$ decides $P(t)$ outright.

The two rules of this section are the mechanism that recovers exactly that. They work as a pair: *renaming* moves an opaque o -typed argument out of the argument position and onto the branch as an equation, where the equality rules can reach it; *instantiation* eliminates the parameter left behind by case analysis over its (finite) domain. Together they make the propositional part of a Henkin model's function space accessible to a calculus whose remaining rules act only on heads. Supplying the extensionality principles as goal-directed inference rules, rather than as axioms added to the input, is the design Benzmlüller and Kohlhasse adopted for higher-order

resolution [BK98] and developed in [Ben99a]; the two rules of this section and the equality rules of the next are its tableau counterpart.

Definition 3.16 (Primitive Term, Parameter Head). A term is *primitive* if it is the η -long form of a single head, i.e. of the shape $\lambda \bar{t}_n. h(n, \dots, 1)$; equivalently, its arguments are exactly its own binders in order. A head is a *parameter head* if it is a parameter introduced by the calculus (δ -Skolemisation, renaming, lambda lifting), which by Chapter 2's representation carries a reference as its name; otherwise it is a *named head*, coming from Σ^{HOL} or the problem's initial parameters.

Definition 3.17 (Extensionality Candidate). Let φ be an atom whose head is a constant or parameter and let π be a position inside one of φ 's arguments, $t = \varphi|_\pi$ of type τ . Then π is a *candidate* if

- (C1) $\tau \in \mathcal{O}$ and the head of t is a constant;
- (C2) $t \notin \text{gen}(\tau)$, so that t is not already one of the closed propositional terms, which would make the rule idempotent;
- (C3) t contains no loose de Bruijn index, so that replacing t by a closed term captures nothing and the companion equation is well-formed.

A candidate is a *renaming* candidate if t is non-primitive with a named head, and an *instantiation* candidate if t is primitive with a parameter head. The two are disjoint, and the remaining two combinations are no candidate at all: a primitive term with a named head admits no further decomposition, and a non-primitive term with a parameter head has had its head introduced by a previous renaming.

Definition 3.18 (Renaming). Let π be a renaming candidate of φ with $t = \varphi|_\pi$ of type τ . Then

$$\frac{\varphi}{\varphi[c]_\pi}(\text{rename})$$

$$c = \langle \tau \rangle t$$

where $c = p\{\text{FV}(t)\}$ is a fresh parameter term of type τ

The rule is an α -rule: both conclusions land on the same branch.

Definition 3.19 (Instantiation). Let π be an instantiation candidate of φ , with $t = \varphi|_\pi$ headed by the parameter $c : \tau$, and let $\text{gen}(\tau) = \{b_1, \dots, b_k\}$. Then

$$\frac{\varphi}{\varphi[b_1]_\pi \mid \dots \mid \varphi[b_k]_\pi}(\text{inst})$$

and the j -th branch additionally records the definition $c := b_j$, so every later occurrence of c on that branch unfolds consistently.

Proposition 3.20 (Boolean Extensionality). (rename) and (inst) are sound, and composed they reduce an occurrence of an opaque o-typed argument t to a finite case split over $\mathcal{D}_\tau$.

- Proof.*
1. (rename) is sound because c is fresh, so $\varphi \equiv \exists \tau. (\varphi[1]_\pi \wedge 1 = t)$ holds in every Henkin model, and the δ -style witness is t itself.
 2. (inst) is sound because $\text{gen}(\tau)$ exhausts $\mathcal{D}_\tau$, so the disjunction of the k conclusions is entailed by the premise, and the case analysis on c is exhaustive because no element of $\mathcal{D}_\tau$ is omitted.
 3. Renaming replaces t by a parameter and produces $c = \langle \tau \rangle t$, which the equality rules expand to $c \equiv t$ at type o (and pointwise above it), placing t under the propositional rules; instantiation then discharges c by case analysis over $\text{gen}(\tau)$. This is the syntactic content of $\mathcal{D}_o = \{T, F\}$.

□

Joint completeness for the propositional fragment of the function space is not claimed. The rules fire only on atoms with a rigid head and only at argument positions satisfying (C1) to (C3); no argument is given here that every occurrence a refutation requires is reachable under those restrictions.

Definition 3.21 (Top-Level Fallback). If φ has no candidate at any inner position, but some top-level argument t satisfies (C1)-(C3) and additionally has a head whose *own* type lies in $\mathcal{O}$, then (inst) is applied at that argument, over gen of the head's type. If neither this nor any inner candidate applies, φ is *atomic* and joins the branch's literal set.

Both rules fire only on an atom whose head is a rigid constant, and only one candidate is taken per application, the first found by a bottom-up fold over the argument DAG, after which the formula is reclassified and the next candidate, if any, is taken on the following step. Condition (C2) is decided by membership in the memoised $\text{gen_o}(\tau)$ list, (C3) by a bottom-up loose-index computation rather than by the term's fvars field, which tracks free variables only and says nothing about indices reaching a binder further out.

The fallback's extra condition on the *head's* type is not redundant. The definition recorded by (inst) pairs the head declaration with the instance, so the instance must inhabit the head's type, not the candidate's: $p(a)$ with $\Gamma \vdash p : ((o, o) \rightarrow o) \rightarrow o$ has type o , and pairing $p := \perp$ would make a later $p(a)$ unfold to $\perp(a)$. The rule therefore draws from gen of the head's type, which requires that type to be enumerable, not implied by the candidate's own type being pure- o , since $p : \iota \rightarrow o$ applied to $a : \iota$ also has type o .

The branch count k that the cost table charges for (inst) is computed as $2^{\#_o(\tau)}$, where $\#_o$ counts the o -occurrences of a pure o -type. This agrees with $|\mathcal{D}_\tau|$ at arities 0 and 1 and under-estimates it above; it prices the rule and does not bound the enumeration, which is the full lazy gen_o stream. The whole mechanism is behind the `atom_decomposition` flag, which when off routes every atom straight to the literal set (the ablation baseline for “prover without Boolean extensionality”).

3.6 Equality Rules

The rules below expand an equation at a type into statements at smaller types, until the heads that the closure test compares are reached. On the resolution side the same expansion is carried out by extensional paramodulation and RUE-resolution

[Ben99b], and the trade-offs between that treatment of equality and the alternatives are set out in [Ben02].

Definition 3.22 (Equality Expansion). In the spirit of Kohlhase’s higher-order tableaux [Koh95], we extend α -rules to expansion rules for equality, where $\varsigma \in \mathcal{S} \setminus \{o\}$:

$$\begin{array}{c} \frac{\varphi = \langle o \rangle \quad \psi}{\varphi \equiv \psi} (= \langle o \rangle) \qquad \frac{s = \langle \varsigma \rangle t}{\forall \varsigma \rightarrow o. 1(s) \equiv 1(t)} (= \langle \varsigma \rangle) \\[10pt] \frac{s = \langle (\tau, \bar{v}) \rightarrow \varsigma \rangle t}{\forall \tau. s(1) = \langle \bar{v} \rightarrow \varsigma \rangle t(1)} (= \langle (\tau, \bar{v}) \rightarrow \varsigma \rangle) \\[10pt] \frac{\neg(\varphi = \langle o \rangle \quad \psi)}{\neg(\varphi \equiv \psi)} (\neg = \langle o \rangle) \qquad \frac{\neg(s = \langle \varsigma \rangle t)}{\neg(\forall \varsigma \rightarrow o. 1(s) \equiv 1(t))} (\neg = \langle \varsigma \rangle) \\[10pt] \frac{\neg(s = \langle (\tau, \bar{v}) \rightarrow \varsigma \rangle t)}{\neg(\forall \tau. s(1) = \langle \bar{v} \rightarrow \varsigma \rangle t(1))} (\neg = \langle (\tau, \bar{v}) \rightarrow \varsigma \rangle) \end{array}$$

Definition 3.23 (Negative Extensionality). Let s, t be closed of type $(\tau, \bar{v}) \rightarrow \varsigma$. Then

$$\frac{\neg(s = \langle (\tau, \bar{v}) \rightarrow \varsigma \rangle t)}{\neg(s(c) = \langle \bar{v} \rightarrow \varsigma \rangle t(c))} (\neg =_{cl})$$

where $c = p\{\}$ is a fresh parameter of type τ .

The general negative extensionality rule unfolds the definition and then needs a γ - and a δ -step to reach the same conclusion. When both sides are closed there is no free variable for the witness to depend on, so the δ -step can be performed immediately and the three rule firings collapse into one. The three expansion kinds are tagged separately because they are priced separately: $= \langle o \rangle$ is cheap, extensional expansion is an order of magnitude dearer, and Leibniz expansion dearer still.

Demodulation, and the discipline of rewriting only in the direction a reduction order permits, is taken over from superposition-based equational reasoning [Der82, BG94, BN98], whose higher-order extension is developed by Bentkamp and others [BBT⁺21].

Demodulation is the one rule in this chapter that acts on a branch’s formulae *globally* rather than decomposing a single formula, and is therefore stated in full. Three ingredients are needed on top of the positions of Section 3.1: the set of equations that qualify as rewrite rules, the one-step relation they induce, and the normal form that relation reaches.

Definition 3.24 (Rewrite Rules of a Branch). Let $\succ$ be a reduction order (Chapter 5) and let $\text{Eq}(B)$ be the equations $\ell = \langle \tau \rangle r$ occurring as literals on branch B . The *rewrite rules* of B are

$$\mathcal{E}(B) = \{ \ell \rightarrow r \mid \ell = \langle \tau \rangle r \in \text{Eq}(B), \text{FV}(\ell) \cup \text{FV}(r) = \emptyset, \ell \succ r \}$$

Two side conditions therefore filter $\text{Eq}(B)$ independently. *Groundness* discards any equation mentioning a free variable. *Orientability* discards any pair the order does not strictly separate; since $\succ$ is partial, an equation may be ground and still contribute no rule. A discarded equation remains a literal of B and is still available to every other rule.

Definition 3.25 (Demodulation Step). For formulae u, u' write $u \rightarrow_{\mathcal{E}(B)} u'$ if there is a position $\pi \in \text{Pos}(u)$ such that one of the following holds:

$$(D1) \quad \ell \rightarrow r \in \mathcal{E}(B), \quad u|_{\pi} = \ell, \quad u' = u[r]_{\pi}$$

$$(D2) \quad u|_{\pi} = h(\bar{t}_n) \text{ with } n > 0, \quad (\lambda \bar{v}_n. h(\bar{n})) \rightarrow r \in \mathcal{E}(B), \quad u' = u[r(\bar{t}_n)]_{\pi}$$

(D1) rewrites a whole subterm, (D2) rewrites a head declaration in place, keeping the arguments it was applied to. In both, the equality $u|_{\pi} = \ell$ is *syntactic identity*, not matching modulo a substitution.

Definition 3.26 (Normal Form, (demod)). Let $\rightarrow_{\mathcal{E}(B)}$ be the reflexive-transitive closure of $\rightarrow_{\mathcal{E}(B)}$. A formula u is in *normal form* if no u' with $u \rightarrow_{\mathcal{E}(B)} u'$ exists, and $u \downarrow_{\mathcal{E}(B)}$ denotes a normal form reached from u . The rule replaces a branch formula by its normal form:

$$\frac{\mathcal{E}(B) \quad u}{u \downarrow_{\mathcal{E}(B)}} (\text{demod})$$

Proposition 3.27 (Binding-Freeness). *If $u \rightarrow_{\mathcal{E}(B)} u'$ then $\text{FV}(u') \subseteq \text{FV}(u)$ and no free variable of u is instantiated.*

Proof sketch. Both (D1) and (D2) match by identity of term IDs, so the matcher is the empty substitution; the replacement introduces r , which is ground by the groundness condition on $\mathcal{E}(B)$. $\square$

Proposition 3.28 (Termination). *$\rightarrow_{\mathcal{E}(B)}$ is terminating, so $u \downarrow_{\mathcal{E}(B)}$ exists for every u .*

Proof sketch. Each step replaces $u|_{\pi}$ by a strictly $\succ$ -smaller term; monotonicity of the order's companion relation lifts this to $u \succ u'$, and $\succ$ is well-founded. $\square$

Proposition 3.29 (Soundness). *If $u \rightarrow_{\mathcal{E}(B)} u'$ then $B \models u \leftrightarrow u'$.*

Proof sketch. Substitutivity of equality applied at π , with the equation $\ell = r$ available on B by construction of $\mathcal{E}(B)$. $\square$

Both cases are implemented as matching, never unification. (D1) is decided by term-ID equality, which under hash-consing is exactly structural equality on $\beta\eta$ -long normal forms; (D2) compares head declarations and re-applies the right-hand side with `fold_apply!`. Classical demodulation instead matches modulo a substitution σ and applies it, and a σ chosen on branch-local evidence would bind free variables that the global reconciliation of Chapter 6 must remain free to bind otherwise. That is why the classical case, rewriting $f(sk)$ by $f(X) = a$ under $\sigma = [X \mapsto sk]$, is not performed here: such a rewrite becomes available only once γ has produced the concrete instance. Unifying paramodulation is unavailable in this calculus for the same reason.

The ground restriction is a second, independent condition, and it is a refutational one rather than a soundness one. An equation carrying a rigid γ -variable is recorded

on the branch; the openness test consults it and the Leibniz/extensional expansion still fires, but it never rewrites. Rewriting with it would destroy the very literal pair the branch needs in order to close: the surjective Cantor theorem yields $c(d(Y)) = Y$ with Y rigid, whose left side is a primitive η -expansion, so head rewriting collapses the pair $\{c((d(Y)), X), Y(X)\}$ to $\{Y(X)\}$ and leaves nothing for the global substitution $[Y \mapsto \lambda u. \neg(c(1, 1))]$ to contradict. Ground equations have the opposite effect: they *create* clashes, as when $a, \neg b$ and $b = a$ close only because $\neg b$ normalises to $\neg a$; and this is the case the rule is intended for.

Rewriting runs in both directions. A formula entering the branch is normalised against the equations already present; an equation entering the branch triggers renormalisation of the literals already present, and any literal whose form changes is discarded in favour of its normal form. Orientation is decided by the term ordering of Chapter 5, falling back to a total heuristic extension when the two sides are incomparable, purely so that storage is deterministic.

The rules of this section reach the functional principles only from a formula that is already an equation. No rule turns a pair of literals with the same rigid head and opposite polarity into an equation between their arguments, so a branch that can be separated only in that way saturates. This is the gap stated in Section 3.11 and measured in Chapter 8.

3.7 Closure and Discharge

Not every classification produces an expansion. Six formula shapes terminate or discharge without adding to the branch.

$$\begin{array}{ccc} \frac{\perp}{\times}(\perp) & \frac{\neg\top}{\times}(\neg\top) & \frac{\neg(a=a)}{\times}(\neg\text{refl}) \\ \frac{\top}{\neg}(\top) & \frac{\neg\perp}{\neg}(\neg\perp) & \frac{a=a}{\text{refl}} \end{array}$$

The first group closes the branch unconditionally; the second discharges the formula as carrying no information. A formula matching no expansion rule is *atomic* and is added to the branch's literal set, where it participates in clash and disagreement detection.

Reflexivity is decided by term identity on both sides, so it costs a single comparison rather than an equality expansion. An atom whose head is a defined constant is unfolded first and re-classified; only genuinely opaque atoms reach the literal set.

3.8 Literal Processing

$\text{Lit}(B)$ is the branch's only interface to closure. Every rule of this chapter contributes to it, and everything the branch communicates to the rest of the prover, whether the candidate closing pairs of Section 3.1 or the countermodel of Chapter 7, is a statement about its contents. It is treated separately here, because admission to $\text{Lit}(B)$ takes several steps: a formula is normalised twice, may be unfolded, is compared against every literal already present, and can later be replaced.

Every branch is *initialised* with $\{\top, \neg\perp\} \subseteq \text{Lit}(B)$. The two discharge rules of the previous section remove $\top$ and $\neg\perp$ as *premises*, because as premises they carry

no information; retaining them as *literals* costs two entries once and makes both elements of $\mathcal{D}_o$ permanently available as closing partners. The benefit concerns flex literals. A literal $H(\bar{y})$ produced by primitive substitution, or any literal whose head is still a free variable, is tested against the two seeds like any other pair, so it can close a branch by disagreeing with a truth constant instead of waiting for a general binding to fix its head first. Both seeds are filtered out again wherever a model or a proof is reported, so they never appear as atoms of a countermodel.

A formula reaches $\text{Lit}(B)$ by exactly two routes: the atomic rule admits a formula that classification has found no expansion rule for, and dual instantiation (below) admits the source of an (inst) application alongside the case split it triggers. Nothing else writes to $\text{Lit}(B)$; the only further modification is replacement under (demod).

Classification reaches the atomic verdict only after the decomposition attempts of Section 3.5 have failed. An atom is a candidate for (rename) or (inst) only if its head is a rigid constant applied to at least one argument; anything else, such as a bare constant, a variable-headed term or an abstraction, is atomic immediately. Under a negation the classification recurses on the atom and re-wraps the verdict, so $\neg\phi$ and ϕ take the same route. With `atom_decomposition` off, both decomposition verdicts are re-dispatched as the atomic rule on the unchanged source, which is why that flag is the ablation baseline for “prover without Boolean extensionality”: the atoms still reach $\text{Lit}(B)$, undecomposed.

Admission is not immediate. An atom is *demodulated again* on its way in, having already been normalised once when it entered the queue, and is then *unfolded* if its head is defined.

The second demodulation is not redundant, and its absence is unsound. A formula is normalised and queued under the equations that existed at insertion, but equations keep arriving while it waits, and an α -rule can ingest an equation produced by the very rule that queued the atom. Normalising only at insertion therefore lets an atom enter $\text{Lit}(B)$ in a form that is no longer normal; a clash is decided by identity, so it is missed, the branch is reported saturated, and the prover answers `CounterSatisfiable` for a theorem. If the rewrite fires, the normal form is registered on the branch and a demodulation entry is recorded, so proof reconstruction can replay it.

When a defined head is unfolded is the `unfold_defs` flag. Under `:eager`, definitions are expanded on the way into the branch, so a defined head never survives classification. Under `:lazy` (the default) the folded atom is classified and queued as itself, and the expansion is deferred to exactly this point: it happens only once no expansion rule applies, and the unfolded form then *replaces* its source, consuming it and advancing the frontier rather than entering $\text{Lit}(B)$. `eager_unfold_defs` is the corresponding ablation row.

Admission ends in a comparison against every literal already present, and the two outcomes are the *clash* and *disagreement* of Section 3.1, tried in that order. An admitted ϕ is first tested for a clash, which is membership of $\sim\phi$ in $\text{Lit}(B)$ and closes B by ground closure; failing that, ϕ is tested for disagreement with each $\psi \in \text{Lit}(B)$, and every disagreeing pair is retained as a candidate closing pair. In either case ϕ joins $\text{Lit}(B)$, including the closing case, so a closed branch’s literal set still contains the literal that closed it, which is what proof reconstruction needs.

Both orientations of the disagreement test are formed because negation is stored syntactically, so only one of the two shapes presents the unifier with the right pair. Only the non-emptiness of the solution stream is used; the pre-unifiers themselves are discarded, since a branch may not commit to a value for any free variable. The disagreeing pairs are reduced to the partner literals before publication, and the branch records its `last_clash` as either a ground or a unification clash for the proof object. Publishing those pairs is the branch's *only* outbound message about closure, and Chapter 6 is where they are reconciled.

A cheaper test runs earlier and independently of the literal set. When *any* formula is inserted, its complement is looked up among the formulae the branch has already seen, and a hit is remembered as a pending closure that the next step consumes before touching the queue. This catches syntactic contradictions between arbitrary formulae rather than only between literals, and costs one set lookup.

Definition 3.30 (Dual Instantiation). Let φ be an instantiation candidate, so that (inst) splits B into $\varphi[b_1]_\pi \mid \cdots \mid \varphi[b_k]_\pi$. Before the split is performed, φ itself is admitted to $\text{Lit}(B)$. Each of the k children therefore inherits the undecomposed source in its literal set in addition to receiving its own instance.

The rule applies both treatments of φ : it is retained as an opaque literal and decomposed by case analysis. The name is TPS's [BA98, ABB00], where the same discipline is applied to definitions: keeping the undecomposed and the decomposed form simultaneously available means the search cannot fail because the wrong one was chosen, and the reconstruction can discard whichever the closure did not use.

Two consequences follow from the ordering. Admitting the source first means its comparison runs first, so if it closes the branch by ground closure the k -fold split is never built at all, since the rule reports closure and the enumeration is never forced. When the candidate type enumerates no instances, the rule degenerates to the atomic case: the source is still admitted and the step behaves as a plain atomic rule.

Candidate closing *pairs* found at this point are not published; only ground closure is acted on. The reason is structural rather than an economy measure: a step returning a fan-out of children carries no effect channel in the branch's result type, so there is nowhere to attach the notification. The pairs are rediscovered in the children, which do have one.

Each (inst) child extends the branch's definition map with its own $c := b_j$, and must re-expand what it inherited under the enlarged map, both its literals *and* its equations.

The equations are the part that is easy to miss, and missing them is again unsound. (rename) may have abbreviated $a \wedge b$ as a fresh parameter c and recorded $a \wedge b = \langle o \rangle c$; a child choosing $a := \top$, $b := \top$, $c := \perp$ then holds a definition set that its own equation refutes, while every literal in it stays clash-free. It would report itself saturated and be published as a countermodel. Re-inserting the unfolded equation routes $\top = \langle o \rangle \perp$ back through classification, which closes it.

It is already stated in Section 3.6 that an equation arriving on the branch renormalises the literals already present. Two details of that sweep belong to the literal set rather than to the rewrite relation.

The sweep runs under the whole of $\mathcal{E}(B)$, not under the newly arrived rule alone. Rewriting with the new rule can expose a redex for an older one, and the alternative assumption, that every literal is already in normal form under the previous $\mathcal{E}(B)$, is an invariant the branch does not maintain, because literals also enter directly through the atomic rule, whose own normalisation happened at a different moment. Using the whole set makes $\text{Lit}(B)$ independent of the order in which equations arrived. When a literal's normal form is already present, the non-normal form is dropped rather than replaced, so the sweep never enlarges $\text{Lit}(B)$.

When the queue empties, $\text{Lit}(B)$ is read one last time, to decide between the two outcomes of Section 3.1: the under-approximation of openness described there, or exhaustion.

Two disjuncts of the openness test discharge a literal pair without inspecting arguments at all, and account for the test being cheap in the common case: two distinct rigid heads can never be identified, whatever their arguments denote, and neither can two atoms of different arity. Only when both fail does the test search for a separating argument position, and there it additionally requires that neither of the two terms is the left-hand side of an equation of B , since a term the branch can still rewrite is not yet evidence that the pair stays apart.

3.9 Preprocessing and BDDs

Definition 3.31 (Lambda Lifting). Let φ contain a closed abstraction $\lambda\bar{y}.b$ occurring as an argument of a non-logical rigid constant. *Lambda lifting* replaces it with a fresh constant f and records the axiom $f = \lambda\bar{y}.b$.

Lambda lifting [Joh85] runs once per input formula at branch creation and is recorded in the history under its own tag, so the proof object can present the original formula rather than the lifted one. The emitted axiom is an ordinary branch formula: the equality-expansion rule decomposes it extensionally or by Leibniz, so subsequent reasoning proceeds by the ordinary rules. Lift constants are kept out of the branch's definition map by design, so definition unfolding cannot undo a lift, and η -expansions of named constants are skipped, since they add a level of indirection without adding structure.

The second preprocessing step is the one design decision in this chapter that departs from ordinary automated-theorem-prover practice. Every formula in SHOT is passed through a complete decision procedure for its propositional structure before any tableau rule is chosen for it. The procedure is a *reduced ordered binary decision diagram*, built and read back for each formula.

Provers in this family normally handle propositional redundancy with local rewrite rules, such as a table of tautologies, unit simplification and subsumption on clauses, because a full propositional decision procedure is thought too expensive to run inside the search loop. SHOT takes the opposite position. In a higher-order tableau, the expensive rules are γ and primitive substitution, and their cost exceeds that of the propositional layer by orders of magnitude, so the cost of preprocessing that prevents a formula from reaching them is small relative to the cost it avoids. A propositional tautology reaching the queue would be decomposed by α/β rules

into literals that enlarge the set scanned by clash detection, and a propositionally unsatisfiable formula would be split by β into branches that each have to be closed separately. The BDD reduces both to a single node before the rule loop reaches them.

Definition 3.32 (Propositional Abstraction). A formula φ of type o is read as a propositional formula over *atoms*: every maximal subterm of φ whose head is not one of $\top, \perp, \neg, \vee, \wedge, \supset, \equiv$ is an atom. Because terms are hash-consed, an atom is identified with its term ID, and syntactically equal atoms are the same propositional variable.

Definition 3.33 (ROBDD). A *binary decision diagram* [Bry86] over a totally ordered set of variables is $\top, \perp$, or a node $\langle v, h, l \rangle$ with h, l diagrams whose variables all exceed v . It denotes $(v \wedge h) \vee (\neg v \wedge l)$. It is *reduced* if no node has $h = l$, and *ordered* by construction. The variable order used is the ascending order of atom term IDs, so it is total, stable across a proof, and free to compute.

The construction is the textbook one, with the reduction rule folded into the node constructor:

$$\text{mk}(v, h, h) = h \qquad \text{mk}(v, h, l) = \langle v, h, l \rangle \ (h \neq l)$$

Connectives are interpreted by structural recursion: $\supset$ as $\neg\varphi \vee \psi$ and $\equiv$ as $(\varphi \wedge \psi) \vee (\neg\varphi \wedge \neg\psi)$, so only $\neg, \wedge$ and $\vee$ need diagram operations; and an atom a becomes the node $\langle a, \top, \perp \rangle$. Negation complements both children recursively. Conjunction and disjunction are Shannon expansions on the smaller of the two root variables, with absorbing and neutral cases short-circuited:

$$\begin{aligned} \langle v, h_1, l_1 \rangle \star \langle v, h_2, l_2 \rangle &= \text{mk}(v, h_1 \star h_2, l_1 \star l_2) \\ \langle v_1, h_1, l_1 \rangle \star B &= \text{mk}(v_1, h_1 \star B, l_1 \star B) \quad (v_1 < \text{var}(B)) \end{aligned}$$

Definition 3.34 (Readback). The terminals map to $\top$ and $\perp$, and a node is expanded by Shannon expansion under six short-circuit identities that suppress the redundant connectives, with the general case as the fallback:

$$\begin{aligned} \langle v, \top, \perp \rangle &= v & \langle v, \perp, \top \rangle &= \neg v & \langle v, \top, l \rangle &= v \vee l \\ \langle v, h, \perp \rangle &= v \wedge h & \langle v, \perp, l \rangle &= \neg v \wedge l & \langle v, h, \top \rangle &= \neg v \vee h \\ \langle v, h, l \rangle &= (v \wedge h) \vee (\neg v \wedge l) \end{aligned}$$

Proposition 3.35 (Decision). Under a fixed variable order the reduced ordered diagram is a canonical form for the propositional abstraction of φ . In particular φ is a propositional tautology iff its diagram is $\top$, propositionally unsatisfiable iff its diagram is $\perp$, and two formulae are propositionally equivalent iff their diagrams are identical.

Proof. Canonicity of reduced ordered diagrams under a fixed variable order is due to Bryant [Bry86]. A diagram denotes the Boolean function obtained by reading $\langle v, h, l \rangle$ as above, and the construction preserves that denotation, so the diagram of φ denotes the propositional abstraction of φ . The constant functions T and F have the diagrams $\top$ and $\perp$, which by canonicity are the diagrams of exactly the tautologies and of exactly the unsatisfiable formulae, and identity of canonical forms is identity of the denoted functions. $\square$

Table 3.2: The BDD oracle applied to four propositional formulae under `:deep` simplification.

Input	Simplified
$(a \wedge b) \vee (a \wedge \neg b)$	a
$a \vee \neg a$	$\top$
$((a \supset b) \wedge a) \wedge \neg b$	$\perp$
$\neg(a \wedge b) \equiv (\neg a \vee \neg b)$	$\top$

Canonicity is what makes the two extreme verdicts free; it does not make the readback small. A diagram can be exponentially larger than the formula that produced it, and the readback is linear in the diagram, so simplification guarantees nothing about size. What it guarantees concerns content: the readback mentions an atom only if the propositional function genuinely depends on it, so every connective it does emit is one the tableau would have had to decompose anyway.

The two extreme verdicts are handled directly by the rule table. A formula whose diagram is $\top$ is discharged by the $(\top)$ rule; one whose diagram is $\perp$ closes the branch by the $(\perp)$ rule at cost 0. Neither outcome requires a single α -, β - or γ -step. Everything in between arrives at the classifier in a normalised form whose atoms are exactly the propositionally relevant ones: the a of $(a \wedge b) \vee (a \wedge \neg b)$ survives, the b does not, and the β -rule that would have split on it is never generated, as Table 3.2 shows.

`ShotTx.Util.PropSimplify` is a self-contained ROBDD over term IDs: nodes are `:bdd_top`, `:bdd_bot`, or `{var, high, low}`, `mk/3` is the reduction rule, and `bdd_` and `bdd_or/bdd_neg` are the operations above. `simplify/2` runs the whole construction inside a term-factory scratchpad, so the intermediate terms built during readback are discarded with it and only the result is committed. The `simplification` parameter selects the depth: `:none` disables the oracle, `:shallow` normalises only the formula's top-level propositional structure (treating quantifier and abstraction bodies as opaque atoms), and `:deep` (the default) folds bottom-up over the term DAG and simplifies every pure σ -type subformula, including the bodies of quantifiers and of abstractions of σ -type.

The oracle runs at the moment a formula is popped from the priority queue, before it is classified. When the simplified term differs from the original, it is recorded as a `:bdd_oracle` history entry sourced at the original formula, added to the branch's term set, and classified in place of the original, so the proof object shows the simplification as an explicit step rather than silently presenting a formula the user never wrote.

3.10 Search Bounds

Three rules do not terminate on their own: γ can be reapplied indefinitely, primitive substitution can be nested indefinitely, and pre-unification is only semi-decidable. Each is therefore bounded, and the bounds are raised together when the search saturates.

- A γ -rule carries an instantiation counter. When it reaches the current limit, the rule is *parked* rather than discarded: it is moved to a sleeping set and removed

from the processed-rule set, so raising the limit reactivates it exactly where it left off.

- A primitive substitution rule carries a depth. On exhausting the bindings at one depth it advances to the next, and parks in the same sleeping set when the depth limit is reached.
- Pre-unification is bounded by a depth counter decremented only on binding steps, so decomposition is free and only imitation and projection consume budget.

Because parking preserves rule state, deepening is not a restart. This is what makes the iterative deepening of Chapter 6 affordable: raising the limits wakes the parked rules and resumes, rather than re-deriving the branch from the input formulae.

3.11 Soundness and Completeness

Soundness is argued rule by rule rather than in one place, and the obligations are collected here. The α , β , γ and δ rules of Section 3.2 and Section 3.3 are the standard ones [Smu68, Fit96] read over the semantics of Chapter 2, and each preserves satisfiability of the branch under every free-variable substitution. Primitive substitution instantiates a γ -formula and is sound for the same reason as (γ) . (rename) and (inst) are covered by Proposition (Boolean Extensionality), (demod) by Propositions (Binding-Freeness), (Termination) and (Soundness), and the preprocessing of Section 3.9 replaces a formula by one with the same propositional abstraction, canonical by Proposition (Decision). Proposition (Tableau-Proof) assembles these.

Completeness is not established. One gap in the rule set is identified here. The extensionality rules reach the functional principles only from a formula that is already an equation: $(=\langle(\tau, \bar{v}) \rightarrow \zeta\rangle)$ and Definition (Negative Extensionality) both act on an equality or on its negation. Two literals with the same rigid head and opposite polarity whose arguments are extensionally equal without being unifiable are therefore acted on by no rule, and a branch carrying such a pair and nothing else saturates. The rule that is absent is the one that turns such a pair into the disequation it entails.

Conjecture 3.36 (Henkin Completeness). *Let (ext) be the branching rule*

$$\frac{\begin{array}{c} P(\bar{s}_n) \\ \vdots \\ \neg P(\bar{t}_n) \end{array}}{\neg(s_1 = \langle \tau_1 \rangle t_1) \mid \cdots \mid \neg(s_n = \langle \tau_n \rangle t_n)}^{(\text{ext})}$$

where $P(\bar{s}_n)$ and $\neg P(\bar{t}_n)$ are literals of the branch and the head P is rigid, and let $\mathcal{C}^+$ be the calculus of this chapter extended by (ext) . If the pre-unification bound and the bounds of Section 3.10 are lifted by iterative deepening, then $\mathcal{C}^+$ is complete for Henkin semantics: every unsatisfiable Φ admits a closed $\mathcal{C}^+$ -tableau.

(ext) is not part of the calculus and is not implemented. It is stated so that the conjecture quantifies over a calculus that could satisfy it: the calculus as given is expected to fail the conjecture for the reason above, and Chapter 8 measures where the

Table 3.3: Rule priority costs. Lower cost means higher priority.

Rule	Cost
closure	0
discharge, atomic	1
$\alpha, \delta, =\langle o \rangle$, suggested instantiation	2
instantiation over k instances	$2 + k$
γ over a finite domain, renaming	3
γ after c instantiations	$3 + 2c$
β	4
γ from instance-based generation	$5 + 2c$
extensional expansion	10
Leibniz expansion	15
primitive substitution at depth d , offset c	$20 + 5d + 2c$

resulting boundary falls. Two obligations remain open even for $\mathcal{C}^+$. The interaction of demodulation with the extensionality rules is not analysed, so normalising under $\mathcal{E}(B)$ is not shown to preserve the existence of a closed tableau. The bounds are recovered one at a time by deepening and are not shown to be recoverable jointly. A proof would take the shape of the model-existence arguments of [Fit96, Koh95] over the model classes of [BBK04].

3.12 Rule Priority and Implementation

Rules are drawn from a priority queue rather than a stack, so the order in which a branch is developed is determined by a cost function rather than by the order of insertion. Lower cost means higher priority.

Table 3.3 is the search heuristic. Closure is checked before anything else; linear decomposition precedes branching; the two rules that generate unbounded work (γ and primitive substitution) are priced so that a branch is decomposed as far as possible before either fires, and γ becomes progressively more expensive as its counter rises. Leibniz expansion is priced above extensional expansion because it introduces a quantifier over a predicate type and so feeds the most expensive rule in the table. Two named alternatives are available for ablation: a uniform cost, which degenerates the queue to insertion order, and a depth-first cost, which pushes every branching rule far back.

Classification is a single function from a term to a rule. It is polarity-aware and takes the two parameters that change the rule assignment: whether quantification over pure o -types is routed to the finite γ -rule, and which equivalence expansion is used.

Classifying $a \wedge (b \supset f(c))$ yields:

```

a  $\wedge$  (b  $\supset$  (f c))
----- (:alpha)
a
b  $\supset$  (f c)
```


Table 3.4: Classification of $a \equiv b$ under the two equivalence-expansion strategies.

Strategy	Family	Produced
:same_polarity	β	$a \wedge b$; $\neg b \wedge \neg a$
:bidirectional_imp	α	$a \supset b$; $b \supset a$

Table 3.5: Rule family and queue cost for three representative formulae.

Formula	Family	Cost
$a \wedge b$	:alpha	2
$a \vee b$	:beta	4
$\forall l. p(1)$	:gamma	3

The equivalence mode changes the rule class rather than only the produced formulae, as Table 3.4 shows for $a \equiv b$ under each strategy.

The cost function is what the queue orders by, so it can be inspected directly, as Table 3.5 shows for three representative formulae.

Higher-Order Unification

4.1 The Shape of the Problem

Branch closure was reduced in Chapter 3 to a question about terms: two literals *disagree* when one of the orientations $\{\sim\varphi \stackrel{?}{=} \psi\}$ or $\{\varphi \stackrel{?}{=} \sim\psi\}$ admits a pre-unifier. This chapter develops the procedure that answers that question. It is the term-level procedure on which the calculus rests: every conditional closure, every γ -instantiation discharged by a later substitution, and every clash reconciled globally in Chapter 6 invokes it.

The problem is harder than its first-order counterpart in three ways, each of which shapes the algorithm. Unifiers need no longer be unique, so the procedure enumerates a *set* of solutions rather than returning one. A most general unifier need not exist, so the notion of “solved” is weakened to *pre-unification*. Solvability is moreover only semi-decidable (indeed already undecidable at second order [Gol81]) so the search is bounded by a depth budget and completeness is recovered only in the limit, through the iterative deepening of Chapter 3’s search bounds.

We treat *pre-unification* (Huet’s procedure [Hue75], surveyed by Dowek [Dow01]) as the primary object of study. Two decidable fragments, Miller patterns and second-order matching, are available as faster paths and are described only briefly at the end; they are optimisations rather than the core.

The vocabulary was already fixed in Chapter 2. A term is *flex* if its head is a free variable and *rigid* otherwise; a *unification problem* is a finite set of equations $\mathcal{E} = \{\bar{s}_n \stackrel{?}{=} \bar{t}_n\}$; a *unifier* is a free-variable substitution θ making $s_i\theta$ and $t_i\theta$ syntactically equal for every i . Recall also the two definitions the tableau depends on directly:

- a *pre-unifier* of $\mathcal{E}$ is a pair $\Theta = (\theta, C)$ with C a finite set of flex-flex equations, such that $\theta\theta'$ unifies $\mathcal{E}$ for every unifier θ' of C ;
- the *pre-unification problem* is to enumerate the pre-unifiers of $\mathcal{E}$.

An individual equation falls into one of three configurations according to its two heads. This trichotomy is the entire case analysis of the procedure.

Definition 4.1 (Equation Configuration). An equation $s \stackrel{?}{=} t$ is *rigid-rigid* if both heads are rigid, *flex-rigid* if exactly one head is flex, and *flex-flex* if both heads are flex.

A pre-unifier is represented as `%ShotUn.UnifSolution{substitutions: theta, flex_pairs: C}`, exactly the pair (θ, C) of the definition. The three configurations are dispatched by `ShotUn’s evaluate_pair/4` on the `head.kind` fields (`:co/bv` rigid, `:fv` flex) of the two terms, after they are fetched from the global pool. Because terms are kept in η -long β -normal form, every term has a head and an argument list accessible, so the configuration is read off without any normalisation.

4.2 Rigid-Rigid: Decomposition

When both heads are rigid there is nothing to guess. A unifier must make the heads coincide, so if they differ the equation has no solution; if they agree, the equation reduces to the componentwise unification of the arguments.

Definition 4.2 (Decomposition). Let $s = \lambda \bar{x}. h(\bar{s}_n)$ and $t = \lambda \bar{x}. h'(\bar{t}_m)$ be rigid. If $h \neq h'$ or $n \neq m$, the equation $s \stackrel{?}{=} t$ *clashes* and has no unifier. Otherwise it reduces to the equations $\{\lambda \bar{x}. s_i \stackrel{?}{=} \lambda \bar{x}. t_i \mid 1 \leq i \leq n\}$.

ShotUn.Internal.decompose/2 zips the two argument lists, failing when the lengths differ, and wraps each pair of arguments back under the parent's binders with wrap_in_bvars/2 so the produced equations are well-typed at the top level. The rigid heads themselves come in two kinds and are checked separately: two constants must be the identical declaration (equal reference or name), while two bound-variable heads must denote the same binder slot, which same_bound_slot?/4 decides modulo the independent renaming of the surrounding λ 's. A length or head mismatch is a :rigid_clash and prunes the branch.

4.3 Flex-Rigid: Imitation and Projection

The flex-rigid case is where the search branches and where non-termination enters. A flex term $\lambda \bar{x}. X(\bar{u})$ must be unified with a rigid $\lambda \bar{x}. h(\bar{v})$. The head X is unknown, and a unifier must choose a value for it whose own head, after β -reduction, is h . There are two ways to arrange that.

Definition 4.3 (General Binding). Let $X : \bar{\alpha}_n \rightarrow \varsigma$ be flex and let the opposing rigid head be h . A *general binding* for X has the form

$$\lambda \bar{x}_n. \mathcal{H}(H_1(\bar{n}), \dots, H_k(\bar{n}))$$

where the $\bar{H}_k$ are fresh free variables of the appropriate types and the head $\mathcal{H}$ is one of:

- the rigid head h itself, giving an *imitation* binding; or
- a projection $\bar{y}_i$ onto one of X 's own arguments whose goal type matches ς , giving a *projection* binding.

The imitation binding copies the opposing rigid head into the solution; the projection binding discards it and promotes one of X 's arguments to the head position, on the assumption that the argument already carries the required structure. Each fresh H_j is applied to all of X 's bound variables $\bar{y}_n$ so that no solution is excluded a priori, since the H_j may still depend on anything in scope. The equation is then replaced by the binding applied against the rigid side, and the search recurses.

ShotUn.Bindings.generic_binding/3 builds these. Given the flex head, the rigid head and the requested binding kinds [:imitation, :projection], it forms the candidate heads (the rigid head for imitation, each of X 's argument variables for projection), keeps only those whose goal type agrees with that of the opposing side, and for each surviving head constructs the matrix $\mathcal{H}(H_1(\bar{y}), \dots, H_k(\bar{y}))$ by allocating

the fresh holes H_j at the enclosing type, applying them to $\tilde{y}_j$ and re-abstracting over $\tilde{y}$. The result is a list of `Substitution` structs, one per admissible binding, each mapping X to a general binding.

The configuration determines which kinds are offered. Against a rigid *constant* head both imitation and projection are candidates; against a rigid *bound variable* only projection makes sense, since there is no constant to imitate. `ShotUn` special-cases these: `:fv::co` and `:co::fv` request `[:imitation, :projection]`, while `:fv::bv` and `:bv::fv` request `[:projection]` alone. A rigid-flex pair is flipped to flex-rigid first so the binding is always generated for the variable side.

One special case applies before the general bindings are built. When the flex side is a *bare* variable X applied to nothing (no arguments, no binders) and X does not occur in the opposing term, the only general binding is imitation of the whole term, so the procedure binds X directly to the rigid side rather than enumerating. This is the ordinary first-order variable-elimination step, recovered as a degenerate case. The occurs-check that guards it is what makes this step sound.

4.4 Flex-Flex: Deferral

When both heads are flex, a common instance always exists, which is to bind both variables to a constant function returning a shared fresh variable; but there are infinitely many, and committing to any particular one risks discarding solutions that a later equation would have forced. Huet’s insight [Hue75] is that one need not choose: a flex-flex equation is always solvable, so it can be carried as a *constraint* rather than solved.

Definition 4.4 (Flex-Flex Deferral). A flex-flex equation is removed from the active problem and accumulated into the constraint set C . The procedure never attempts to solve it.

`ShotUn`’s flex-flex case pushes the pair onto the state’s `flex` list and drops it from the work-list; when the work-list empties, the accumulated `flex` list becomes the `flex_pairs` field of the `UnifSolution`. A pre-unifier thus witnesses solvability of E without committing to any solution for C , which is what the tableau needs: Chapter 6’s global reconciliation may later constrain those same variables from another branch, and a premature local commitment could exclude the solution it requires. It is the term-level analogue of the branch-level rigid-variable discipline.

4.5 The Procedure

The three cases assemble into a single search. The state is a work-list of equations still to process, the substitution θ accumulated so far, the deferred flex-flex set C , and a remaining depth budget. One step inspects the first equation and either solves it trivially, decomposes it, defers it, binds a variable, or *branches* into one successor state per general binding.

Definition 4.5 (Pre-Unification Step). Given a state $(\mathcal{E}, \theta, C, d)$ with $\mathcal{E} = \{e\} \uplus \mathcal{E}'$:

- if $d = 0$, the branch *fails* (budget exhausted);
- if e is $s = ?$, drop it: continue with $(\mathcal{E}', \theta, C, d)$;

- if e is rigid-rigid, decompose it or fail;
- if e is flex-flex, defer it: continue with $(\mathcal{E}', \theta, C \cup \{e\}, d)$;
- if e is a bare-variable binding $X \stackrel{?}{=} t$ with $X \notin \text{FV}(t)$, apply $[X \mapsto t]$ and continue with the same budget;
- if e is flex-rigid, branch into one successor per general binding, each with budget $d - 1$.

A state with $\mathcal{E} = \emptyset$ is a *solution*: it emits the pre-unifier (θ, C) .

The budget is spent only where the search actually branches. In `ShotUn`, `step/1` decrements depth exclusively on the general-binding successors (`build_binding_branch/5` does the `depth - 1`); decomposition, deferral, trivial elimination and the bare-variable bind all pass the budget through unchanged. So “depth” counts imitation/projection commitments (the only source of unbounded work) and agrees with the tableau’s `unification_depth` parameter, which Chapter 3 described as decremented “only on binding steps.” Applying a substitution eagerly rewrites the remaining work-list and re-tests each deferred flex-flex pair, since a binding may turn a pair rigid and pull it back out of C (`apply_substitution/3`).

The successor states are explored depth-first with backtracking, and each solution is yielded before the next is sought, so the whole enumeration is a lazy stream.

`ShotUn.unify/3` wraps the search in a `Stream.resource/3` over a term-factory scratchpad: solutions are produced one at a time, and each is committed from the process-local scratchpad to the global pool only as it leaves the stream (`commit_solution/1`), while intermediate terms from failed branches are discarded with the scratchpad. The depth argument defaults to 10. Because the stream is lazy, a caller that only needs to know whether a pre-unifier *exists* (e.g. the tableau’s disagreement test) takes one element and stops, computing a single solution rather than the whole enumeration.

4.6 A Worked Enumeration

We set up a small flex-rigid problem and enumerate its pre-unifiers. The context declares a binary constant f , a constant a , and leaves X as a free variable.

$$fXa \stackrel{?}{=} faa$$

The heads agree (f vs f), so the equation decomposes into $X \stackrel{?}{=} a$ and $a \stackrel{?}{=} a$. The second is trivial; the first is a bare-variable bind. We expect the single pre-unifier $[X \mapsto a]$ with no residual flex-flex constraints, which is exactly what the procedure returns:

```
substitutions: [X ↦ a]; remaining flex-flex pairs: []
```

Now a genuinely higher-order problem, where imitation and projection diverge. We unify $X(a) \stackrel{?}{=} f(a, a)$ with $X : \iota \mapsto \iota$ flex. Imitation guesses $X \mapsto \lambda y. f(H_1 y, H_2 y)$ and recurses; projection guesses $X \mapsto \lambda y. y$, which fails here since $a \neq f(a, a)$, and the surviving imitation line further resolves the holes. Several pre-unifiers result,

corresponding to the different ways the single argument a can be threaded through f :

```

substitutions: [X ↦ λ. f a a]; remaining flex-flex pairs: []
substitutions: [X ↦ λ. f a 1]; remaining flex-flex pairs: []
substitutions: [X ↦ λ. f 1 a]; remaining flex-flex pairs: []
substitutions: [X ↦ λ. f 1 1]; remaining flex-flex pairs: []

```

The four solutions are the four projections of the argument a onto the two occurrences of a in $f(a, a)$: the bound variable (shown as the de Bruijn index 1) may fill either, both, or neither f -argument. This branching, several pre-unifiers for one problem, is the non-uniqueness that distinguishes higher-order from first-order unification, as no most general unifier exists in the general case.

The disagreement test of Chapter 3 needs only the *existence* of a pre-unifier, which the lazy stream answers without enumerating the rest: for this problem, that test returns `true`.

Every entry point accepts `vis: true`, which materialises the search eagerly and returns a `ShotUn.Trace` alongside the solutions. The trace is pruned to the paths that reach a solution, so failed imitation/projection lines are dropped before rendering. `ShotUn.Trace.Mermaid.render/2` emits a diagram with each node's rule, remaining work-list, accumulated θ and deferred C typeset as $\text{\LaTeX}$, shown in Figure 4.1.

4.7 Depth, Completeness and Decidable Fragments

The budget makes the procedure terminating but incomplete: a pre-unifier that only appears past the current depth is missed. The incompleteness is a consequence of the problem's semi-decidability rather than a parameter to be tuned away. The tableau's response, developed in Chapter 3's search bounds and Chapter 6's iterative deepening, is to *raise* the budget when the search saturates and resume, rather than to fix a single depth. Because the parked γ - and primitive-substitution rules preserve their state across deepening, the extra unification depth is applied where it was previously exhausted, not from scratch.

Two properties make the bound well-behaved. Decomposition and deferral are free, so a problem consisting entirely of first-order structure and flex-flex pairs is solved at any positive depth; only genuine flex-rigid guessing consumes budget. The occurs-check on the bare-variable bind guarantees that the non-branching path always terminates, so raising the budget can only add solutions, never diverge on the ones already found.

Two special cases sidestep the search entirely and `ShotUn` routes to them when a problem qualifies (via `ShotUn.Fragment` and the `strategy: :auto` option). They are performance paths layered over the general procedure, not part of the calculus, and the tableau's soundness does not depend on them.

- **Miller pattern unification** [Mil91]. A problem is a *pattern* when every flex variable is applied only to distinct bound variables. The fragment is decidable and *unitary*, since a solvable pattern problem has a single most general unifier, so `pattern_unify/2` returns at most one solution with no depth bound. This is the common shape in practice, since many γ -instantiations produce exactly such applied variables.

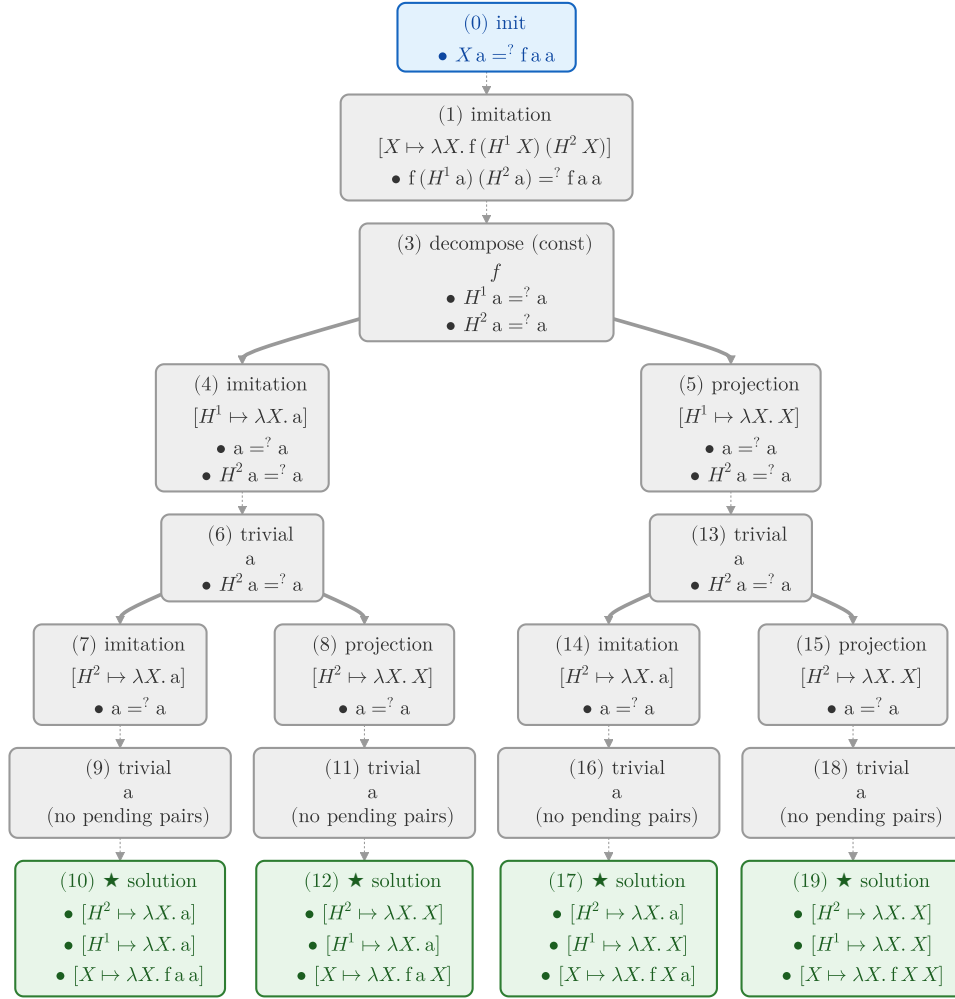


Figure 4.1: Enumeration trace for $X(a) \stackrel{?}{=} f(a, a)$. The root is the initial equation; the first fan-out is the general-binding branch (imitation of f against the two projections of a); each surviving path descends through decomposition and bare-variable binds to a solution leaf (marked $\star$) carrying its pre-unifier. Depth-exhausted and rigid-clash leaves are pruned out by `prune_to_solutions/1`.

- **Second-order matching** [Hue75], with decidability at every order settled by Stirling [Sti09]. When one side is ground and every type in the problem has order at most two, `match/2` enumerates the complete set of matchers by a terminating structural recursion, again without a depth bound.

Under strategy: `:auto`, `resolve_auto/1` tests the pattern precondition first, then the matching precondition, and falls through to depth-bounded pre-unification otherwise, always selecting the most specific decidable algorithm the problem falls into. For everything outside those fragments, and for the disagreement test that the tableau actually runs, pre-unification is the procedure.

Term Ordering

5.1 Overview

Demodulation was introduced in Chapter 3 as a branch-local simplification: an oriented equation $\ell \rightarrow r$ rewrites occurrences of ℓ to r , shrinking the branch’s formula set without touching soundness or completeness. That “oriented” carries an unstated obligation. Rewriting only terminates if the orientation is grounded in a well-founded order (otherwise $\ell \rightarrow r$ and $r \rightarrow \ell$ could alternate forever), and the rewrite is only sound if the order is stable under the substitutions a branch may still receive. This chapter supplies that order.

The order is NCPO-LNF, the $\beta\eta$ -long normal computability path order of Niederhauser and Middeldorp [NM25b], which adapts their NCPO [NM25a] from $\beta\eta$ -normal to $\beta\eta$ -long normal form and thereby to Nipkow’s higher-order rewrite systems. It is a higher-order generalisation of the recursive path orders [Der82] familiar from first-order superposition [BG94], acting directly on typed λ -terms in $\beta\eta$ -long normal form. Numbered definitions, lemmas and theorems cited below are those of [NM25b]. Two design decisions specific to this thesis frame the whole chapter:

- Where the reference implementation leaves the order’s parameters (precedences, statuses, accessibility) as *unknowns* for an SMT solver to discharge while proving a rewrite system terminating, we *fix* them as inputs and obtain a decidable boolean predicate on a single pair of terms. That is what a tableau prover needs: a yes/no orientation for one equation at a time, in place of a termination certificate for a whole system.
- The prover uses the order at *two* strengths. The paper’s order proper is sound but partial: it may refuse to orient a pair in either direction. That partiality is what *gating* a rewrite requires, and what *choosing a canonical form* cannot accept. The prover therefore layers a total heuristic extension over the sound core and keeps the two separate at every call site. The final section states that distinction and how it is enforced, and it is what makes Chapter 3’s demodulation sound.

5.2 Requirements for an Orientation Order

Recall the two properties a rewrite order carries. Well-foundedness rules out infinite descent, so rewriting to normal form terminates. Stability (closure under substitution) ensures that if $s > t$ then $s\theta > t\theta$ for every admissible θ , so an orientation decided now survives the substitutions a branch’s free variables may still receive under Chapter 6’s global reconciliation. In the first-order setting a recursive path order built from a precedence on function symbols delivers both [Der82, BN98]. The higher-order setting breaks the naive lifting in two places: terms carry λ -binders, so the order

must descend under abstraction and handle bound variables; and terms carry types, so the order must respect a companion order on types to stay stable.

NCPO-LNF addresses both. It assumes its inputs are in $\beta\eta$ -long normal form, with every subterm fully η -expanded to its type’s arity, which `shot_ds` already maintains, so the “LNF” precondition holds by construction and no normalisation step is needed before a comparison. Its recursion interleaves a term comparison with a companion type comparison, and it opens λ -abstractions against fresh variables as it descends.

`ShotTo` decides the single predicate $s >_{\tau}^1 t$ of Definition 8 of the paper: given two term IDs in the $\beta\eta$ -long-normal representation and a `%ShotTo.Parameters{}` struct, `ShotTo.gt?/3` returns a boolean. It is a decision procedure rather than an SMT-constraint generator, which is the contrast the paper’s Haskell reference implementation `hrstern` draws, and the one this chapter turns on. Because each λ -opening allocates a fresh `shot_ds` free variable, the implementation wraps every top-level comparison in a term-factory scratchpad so those variables die when the comparison returns rather than accumulating in the global pool.

5.3 The Type Order

The term order is stable only if it defers, at the leaves, to a well-founded order on types. NCPO-LNF uses the *admissible* type order of the paper’s Lemma 4 (itself Lemma 2.3 of the CPO paper [BJR15]).

Definition 5.1 (Admissible Type Order). Given a well-founded strict precedence $>_{\mathcal{S}}$ on base sorts, the admissible type order $>_{\mathcal{T}}$ is the smallest strict order on types containing $>_{\mathcal{S}}$ and the *right-argument* relation $(\tau, \bar{v}) \rightarrow_{\zeta} \bar{v} \rightarrow_{\zeta}$, and closed under right-congruence: $\bar{v} \rightarrow_{\zeta} \bar{v}' \rightarrow_{\zeta'}$ implies $(\tau, \bar{v}) \rightarrow_{\zeta} \bar{v} \rightarrow_{\zeta'} (\tau, \bar{v}') \rightarrow_{\zeta'}$ for every $\tau \in \mathcal{T}$.

Intuitively: a function type dominates its own result type, and among two function types with the same argument prefix the one with the greater result type is greater. On base sorts the order is just the user’s sort precedence.

`ShotTo.TypeOrder` implements this directly on the uncurried `%Type{goal: a, args: [t1, ..., tn]}` representation. `type_gt?/3` decides $>_{\mathcal{T}}$ and `type_geq?/3` its reflexive closure; the base-vs-base case consults `Parameters.sort_prec/2`, the function case peels the shared leading argument and recurses on the result type. Inside NCPO the comparison enters through `type_geq?/3`, where the `type_check?` flag on the internal `ncpo/6` predicate toggles whether $\tau(s) \succeq_{\mathcal{T}} \tau(t)$ must hold at a given node, corresponding to the paper’s choice between $>_{\tau}^{b,X}$ and $>_{\tau}^{b,X}$. A term comparison that would otherwise succeed is vetoed when the types disagree the wrong way.

With ι preceding o at the base level, so that $o >_{\mathcal{T}} \iota$ at the base sorts and $(\iota \rightarrow o) >_{\mathcal{T}} (\iota \rightarrow \iota)$ by right-congruence, Table 5.1 shows the three queries this instance decides.

The third entry is `false` because $\iota < o$: the type order is strict, and $\iota >_{\mathcal{T}} o$ does not hold.

5.4 The Ordering Parameters

A concrete instance of NCPO-LNF is fixed by five pieces of data. Together they are the entire tunable surface of the order, and, as Chapter 8 will exploit, the entire surface the ablation study varies when it studies the order’s effect.

Table 5.1: Type-order queries under the precedence $\iota < o$.

Query	Result
$o \succ_{\mathcal{T}} \iota$	true
$(\iota \rightarrow o) \succ_{\mathcal{T}} (\iota \rightarrow \iota)$	true
$\iota \succ_{\mathcal{T}} o$	false

Table 5.2: Orientability of the two differentiation rewrite rules under three status assignments.

Status	Rule 1	Rule 2
all mul	false	false
all lex	true	true
diff lex, rest mul	true	true

Definition 5.2 (Ordering Parameters). An NCPO-LNF instance is given by:

- a well-founded strict order $\succ_{\mathcal{S}}$ on base sorts;
- a precedence $\succ_{\mathcal{F}}$ on constants, with strict part $\succ_{\mathcal{F}}$ and equivalence $\approx_{\mathcal{F}}$;
- a *status* $\text{stat}(f) \in \{\text{lex}, \text{mul}\}$ per constant, selecting lexicographic or multiset comparison of argument lists under equal heads;
- a *basicness* predicate $\text{basic}(a)$ on base sorts;
- an *accessibility* predicate $\text{Acc}(f, i)$ on argument positions.

The first three are the familiar RPO parameters lifted to the typed higher-order setting. Basicness and accessibility are the higher-order additions: they control the *computability* machinery, meaning which subterms may be descended into while preserving stability, and their soundness depends on compatibility conditions (the paper’s Definitions 5 & 6) relating them to the type precedence.

Status is the parameter whose usable settings the LNF restriction changes, and the paper’s own examples show it. The symbolic differentiation rules of [NM25a], carried over as Example 6 of [NM25b], are oriented there with every symbol given multiset status. That assignment stops working once heads are saturated. With $\text{ar}(\text{diff}) = 2$ the partial application $\text{diff}(F)$ is stored as $\lambda r. \text{diff}(F, 1)$, so after $\langle \lambda = \rangle$ peels the outer binder the recursive goal compares two argument pairs whose second components are unrelated fresh variables, and no multiset comparison relates those. Lexicographic status walks the arguments left to right and decides at the first position, leaving a goal $\langle \mathcal{F} \mathcal{X} \rangle$ discharges. Only diff ’s status matters, which is what Table 5.2 shows for the two differentiation rules under three status assignments (a precedence with diff ranked above sin , cos , plus and times):

`ShotTo.Parameters` holds these as `sort_precedence`, `const_precedence`, `status`, `basic_sorts`, and `accessible`. Each accepts a map, a `MapSet`, or a predicate function, so a caller can be as coarse or as fine-grained as needed. Precedences are given as integer ranks: larger means greater, equal means $\approx_{\mathcal{F}}$. The lookup helpers `prec/2`, `sort_`

`prec/2`, `status/2`, `basic?/2`, `accessible?/3`, together with `gt?/3` and `equiv?/3` on constants, are what the core predicate calls.

Note that `ShotTo`'s own defaults are *permissive to the point of uselessness*: all sorts basic and equivalent, all positions accessible, all constants equivalent with `:lex` status. Under them NCPO collapses to the bare subterm order plus lexicographic comparison inside shared heads and almost nothing is oriented. The defaults are chosen this way so the API is usable without further configuration while every *meaningful* orientation forces the caller to supply at least a constant precedence. A caller that forgot to configure the order would find demodulation silently doing nothing, not silently doing something unsound.

The prover is such a caller, and it supplies exactly that one parameter. `ShotTx.Data.Parameters` carries `term_order: %ShotTo.Parameters{const_precedence: &:erlang.phash2/1}`: every constant name gets a stable hash-based rank, so the precedence is total and well-founded on symbols and NCPO-LNF can orient equations between arbitrary distinct constants, including parameters introduced by δ -expansion, which carry no name by which they could be ranked. Status, basicness and accessibility stay at their permissive defaults. A user-supplied map or function overrides the whole struct.

Moreover, on soundness of the parameters themselves: NCPO-LNF's stability depends on `Acc` and `basic` satisfying the compatibility conditions with the chosen type precedence. `ShotTo` does not enforce these, leaving them to the caller, but offers `Parameters.validate/2` as a best-effort check, and documents a *vacuously sound* fallback: leaving `accessible: :all` and `basic_sorts: :all` makes the compatibility conditions hold trivially, at the cost of a more conservative (weaker) order. The prover takes this conservative-but-safe route, which is the right trade for a soundness-critical component.

5.5 The Order

With types and parameters in place, the term order is a mutual recursion over the shapes of the two terms. Rather than the full rule set of Definition 8, we name the rules that fire and let the implementation stand as the precise statement.

The comparison $s >_t^1 t$ first checks $\tau(s) \succeq_{\mathcal{T}} \tau(t)$, then dispatches on the shape of s :

- **s has a constant head $f(\bar{t})$.** Three families of rules apply in order. $\langle \mathcal{F}\mathcal{X} \rangle$ closes immediately if t is one of the fresh variables introduced while descending under a λ on the right. $\langle \mathcal{F}\triangleright \rangle$ succeeds if some argument t_i of s already dominates t through the *accessible-subterm* relation, the higher-order rule that a term is greater than its subterms, routed through accessibility and basicness. Failing those, the comparison dispatches on t :
 - against another constant head $g(\bar{u})$: if $f \simeq_{\mathcal{F}} g$ with equal status, compare the argument lists by that status ($\langle \mathcal{F} =_{\text{lex}} \rangle / \langle \mathcal{F} =_{\text{mul}} \rangle$); if $f \succ_{\mathcal{F}} g$, require s to dominate every argument of t ($\langle \mathcal{F}\succ \rangle$);
 - against a variable-headed $y(\bar{u})$: the $\langle \mathcal{F}\mathcal{V} \rangle$ rule, which fires only at the top recursion level, requiring s to dominate the η -expanded head and each argument;

Table 5.3: The accessible-subterm rule.

Query	Result
$f(c) > c$	true
$c > f(c)$	false

Table 5.4: The precedence rule, with $g \succ_{\mathcal{F}} h \succ_{\mathcal{F}} c$.

Query	Result
$g(c) > h(c)$	true
$h(c) > g(c)$	false

- against an abstraction $\lambda\tau.v$: the $\langle\mathcal{F}\lambda\rangle$ rule opens t on a fresh variable added to the auxiliary set and recurses.
- **s is an abstraction $\lambda\tau.u$.** The $\langle\lambda\rangle$ rule opens s and checks whether its body already dominates t ; otherwise, if t is also an abstraction, $\langle\lambda=\rangle/\langle\lambda\neq\rangle$ align the two binders (reusing the same fresh variable when their types agree) and recurse.
- **s is variable-headed.** No rule makes a variable-headed term greater than anything; the order never orients *out of* a flex term, which is what keeps it stable under the substitutions those variables may receive.

ShotTo.Ncpo.ncpo_gt?/3 is the entry point; the internal ncpo/6 carries the paper's two extra parameters explicitly: the level $b \in \{0, 1\}$ (some rules, among them $\langle\mathcal{F}\mathcal{V}\rangle$, fire only at $b = 1$) and the auxiliary variable set X that accumulates the fresh variables introduced by λ -openings. The rule names above match the implementation's private functions (fx_rule?, f_subterm_rule?, ncpo_ff_case, ncpo_fv_rule, ncpo_flam_rule, ncpo_lam_head) and the computability helpers bawo/awo/sswo mirror the auxiliary relations of the reference. The last bullet is enforced structurally: dispatch_on_s returns false whenever s 's head is not a constant and s is not an abstraction.

Two worked comparisons. First the subterm rule (a term is greater than a constant nested in its arguments) which needs only a constant precedence, with f ranked above c : $f(c) > c$ by the accessible-subterm rule (c is an accessible subterm), while the converse fails, as Table 5.3 shows.

The precedence rule states that a term headed by a greater constant dominates one headed by a lesser, provided it dominates the lesser term's arguments. With g outranking h in the constant precedence, $g(c) > h(c)$ by the precedence rule, since $g(c) > c$ handles h 's argument, as Table 5.4 shows.

Both examples supply only a constant precedence and rely on the vacuously-sound accessible: :all / basic_sorts: :all defaults, which is the configuration the prover itself uses.

A third comparison is the one the prover actually faces. Chapter 3's Leibniz expansion replaces an equation by a quantification over predicates, so the two sides of that expansion are a pair the order can be asked about: the primitive equality constant at t against its Leibniz encoding.

Table 5.5: Comparing primitive equality at ι against its Leibniz encoding, under two precedences.

Precedence	compare/3
$= \succ_{\mathcal{F}} \forall, \equiv$	:greater
$\equiv \succ_{\mathcal{F}} \forall \succ_{\mathcal{F}} =$	:incomparable

$$\lambda \iota, \iota. 2 = \langle \iota \rangle 1 \qquad \lambda \iota, \iota. \forall \iota \rightarrow o. 1(3) \equiv 1(2)$$

Table 5.5 compares the two under two precedences.

With $=$ above the connectives the encoding introduces, the pair is oriented left to right, so the expansion may be used as a rewrite and rewriting with it terminates. Reversing the precedence yields `:incomparable` rather than the opposite orientation, since contracting the encoding back to $=$ would raise the head’s rank without decreasing anything else. Orientability of a definitional expansion is therefore a property of the precedence, and the precedence does not make the contraction available by being reversed.

The prover’s own precedence is `&:erlang.phash2/1`, which ranks $=$ at 25,810,966 and $\forall$ at 106,440,513, so under the default parameters this pair is `:incomparable` and the expansion is not available as a rewrite. Whether any particular definitional expansion is orientable is thus arbitrary under the default, though stable across runs, and a user who wants a specific expansion oriented has to supply a precedence that puts its head symbol on top. Nothing in Chapter 3 depends on this: Leibniz expansion is a tableau rule with its own cost, and the order gates demodulation alone.

5.6 No Transitivity: One Pair at a Time

An important caveat, because it shapes how the prover is allowed to use the order: **NCPO-LNF is not (yet known to be) transitive** [NM25a]. It is well-founded and stable, the two properties a *reduction* order needs, but transitivity is an open question. The practical consequence is sharp: the order may be used to decide the orientation of *one pair at a time*, but must not be used as a comparator to sort a list of terms or to build a total order over a set, because sorting silently assumes transitivity. `ShotTo`’s public surface reflects this: it offers `gt?/3`, `geq?/3`, and a four-valued `compare/3`, but no `sort` and no claim of a total order.

The limitation is a property of the order and cannot be avoided by the prover. What the prover *can* do is confine every use to pairwise orientation decisions, which is all demodulation and connective normalisation actually require. The next section is how that confinement is arranged.

5.7 The Total Prover-Side Wrapper

The paper’s order is sound but *partial*: for many pairs it answers neither $s \succ t$ nor $t \succ s$. `ShotTo.compare/3` reports this as a fourth verdict, `:incomparable`, alongside `:greater`, `:less`, and `:equal`. Partiality is appropriate when the question is whether ℓ may be rewritten to r , since the safe answer for an incomparable pair

is to block the rewrite, and unhelpful when the question is which of two commutative arguments is the canonical left, where any deterministic answer suffices. `ShotTx.Prover.TermOrder` addresses both cases with two separated layers.

The sound layer. `strict_gt?/3` is a direct passthrough to `ShotTo.gt?/3`, NCPO-LNF unmodified. By the paper’s Theorem 2 the pair (NCPO-LNF, CPO) is a $\beta\eta$ -normal higher-order reduction order: well-founded on $\beta\eta$ -long-normal terms, monotone, and stability-compatible under $\beta\eta$ -long-normal substitutions. This is the *only* decision suitable for gating a rewrite. It may return `false` both ways for the same pair, and every caller must be prepared to block the step in that case.

This is what Chapter 3’s demodulation relies on. `ShotTx.Prover.Demodulation` rewrites a subterm to an equation’s right-hand side only when `TermOrder.strict_gt?(lhs, rhs, order)` holds. Every rewrite step strictly decreases the term in a well-founded order, so rewriting to normal form terminates, and equations that fail the strict gate contribute no rewrite rule at all (`demodulation.ex`). The termination argument is therefore a local invariant enforced at each rewrite rather than an external claim about the prover. The order is one of the two independent filters on Chapter 3’s $\mathcal{E}(B)$; the other, groundness, is a refutational condition and has nothing to do with the order.

The heuristic layer. `compare/3` and `gt?/3` wrap `ShotTo.compare/3` and, only on `:incomparable`, break the tie deterministically, first by alphabetic order on the printed term, then by the raw integer term IDs. The result is *total* and decidable, but is **not** in general a reduction order: it is not guaranteed well-founded or monotone. Its sole use is to pick a canonical direction where any consistent choice suffices, and it must never gate a rewrite.

The prover uses this heuristic layer in exactly two places, both cosmetic. `orient_pair` in `branch.ex` stores an equation $\ell = r$ by trying `strict_gt?` in each direction first, and *only if both fail* falls back to the heuristic `gt?` to pick a storage direction. The connective-canonicalisation cases ($\vee$, $\wedge$, $\equiv$, $=$) use `gt?` purely to decide whether to swap the two sides into a fixed order; that pass is governed by the `orient` parameter. `:none` (the default) skips it entirely, `:shallow` normalises only each formula’s outermost connective, `:deep` recurses bottom-up through every nested commutative connective, including those inside atom arguments. In both places the choice affects which syntactic representative is stored, never whether a logically-significant rewrite fires. Conflating the two layers would be the classic superposition bug: orienting a rewrite by a non-well-founded relation, and losing termination. Keeping them apart should keep demodulation sound.

The two layers can be made to diverge. Under a precedence that leaves two unrelated constants incomparable, the sound layer refuses to orient the pair while the heuristic layer still returns a deterministic direction. With a and b equivalent in precedence (both rank 0) and unrelated by structure, NCPO-LNF cannot orient $\{a, b\}$ either way:

```
sound_a_gt_b: false
sound_b_gt_a: false
heuristic_compare: :less
```

Both `strict_gt?` queries are `false`. NCPO-LNF declines to orient the pair, so demodulation between a and b is correctly blocked. The heuristic `compare` still

returns `:less` (here by alphabetic tie-break, $a < b$), which is fine for choosing a canonical connective side and would be unsound as a rewrite gate.

5.8 Summary and Forward Pointers

NCPO-LNF gives the tableau a higher-order reduction order that is well-founded and stable, enough to make demodulation terminating and sound. The order is decided pairwise as a boolean predicate over fixed parameters rather than solved as an SMT termination problem. The prover consumes it at two strengths: the sound, partial `strict_gt?` gates every rewrite, and a total heuristic extension is confined to cosmetic orientation of commutative connectives. Its main open problem, non-transitivity, is contained by only ever asking pairwise questions.

Why the stability property is required rather than incidental is shown in Chapter 6: because branch-level rules never commit free-variable substitutions and the substitution arrives only later, from global reconciliation, an orientation decided on a branch must survive that late substitution. Chapter 8’s ablation study varies the parameters of this order (constant precedence, status, the accessibility and basicness fallbacks) among its parameters, measuring how much orientation strength the prover actually needs.

A Concurrent Actor-Based Architecture

6.1 Overview

The previous three chapters described a calculus and its term-level machinery: tableau rules that expand a branch (Chapter 3), a unification procedure that decides disagreement (Chapter 4), an order that gates rewriting (Chapter 5). None of them said *how the search is run*. This chapter does, and the answer is the thesis's central contribution: the search is run by a pool of concurrent actors on the BEAM [Arm03], coordinated through shared memory and a published stream of evidence, in place of the single backtracking loop a sequential implementation would use.

The design follows from a single logical constraint: the *rigidity* of free variables across branches, which forces a separation between what a branch may decide locally and what must be decided globally. Once that separation is made, the global decision is a distinct process, the local expansions may proceed in parallel, and the resulting structure is an actor system. The chapter is organised around that argument: first the constraint, then the architecture it implies, then the message flow that ties it together, and finally the two peer agents that exploit the same evidence stream for satisfiability and heuristic guidance.

6.2 The Rigid Variable Problem

The design follows from one fact about free-variable tableaux [Fit96, Häh01]. When the γ -rule instantiates a universally quantified formula, it introduces a *free variable* X standing for a witness to be determined later, and branch closure then amounts to finding a substitution that makes complementary literals unify. The variable is *rigid*: it denotes the same term everywhere it occurs. If the left branch of a split closes only under $X \mapsto a$ and the right branch closes only under $X \mapsto b$ with $a \neq b$, the tableau is *not* closed, because no single X satisfies both. A proof requires *one* global substitution that closes *every* open branch simultaneously. This is what makes naive parallelism unsound.

Proposition 6.1 (Unsoundness of locality for rigid variables). *A branch that commits to a local substitution $X \mapsto t$, using it to close itself, can render the whole search either unsound or incomplete: unsound, if a sibling branch later closes under a substitution inconsistent with $X \mapsto t$ yet the search reports success; incomplete, if the local commitment pre-emptively constrains a variable that a sibling needed to bind differently, so that a global solution which exists is never found.*

Proof. Take the split of the preceding paragraph, in which the left branch closes only under $X \mapsto a$ and the right only under $X \mapsto b$ with $a \neq b$. If each branch applies its own substitution and reports itself closed, the search reports the tableau closed although no single instance of X closes both branches, so a satisfiable set is refuted.

If instead the left branch's commitment is imposed on its siblings, the right branch is searched under $X \mapsto a$ alone, and a global substitution closing every branch by binding X elsewhere lies outside the space searched, so a refutation that exists is not found. $\square$

The consequence is a strict discipline, stated identically across the prover's rewriting and branch modules:

No branch-level rule may commit to a value for any free variable. Every θ -commitment is recorded as *evidence*, not applied. The reconciliation of all such evidence into a single global pre-unifier is performed elsewhere, once, over all open branches at once. This is why Section 3.6's demodulation admits only rewrites with the *empty* matcher (the equation's LHS must be structurally identical to the target, no θ), and why Section 5.2's order must be *stable* under substitution: an orientation decided on a branch has to survive the global substitution that arrives later.

Read architecturally, the discipline partitions the work into two kinds. *Local, parallelisable* work: expanding a branch by the tableau rules, collecting the literal pairs that *could* close it. *Global, serialised* work: taking the closure options from all branches and finding one substitution that satisfies them together. The first kind has no cross-branch dependency, since a branch expands using only its own formulas, so any number of branches may be expanded at the same time; how many processes are used to do so is an independent decision, taken in Section 6.4. The second kind is a single constraint-satisfaction problem over shared state. The architecture implements that partition, and the rest of the chapter describes its realisation on OTP.

6.3 The Supervision Tree

A proof is a *session*. Each call to the prover starts a fresh supervision tree, runs the search, returns the verdict, and tears the tree down. The tree's shape encodes the local/global partition directly, and it is declared in one place (`SessionSupervisor.init/1`) whose child list is given verbatim in Listing 6.1, since the order of the children and the restart strategy both matter.

The `EtsKeeper` is first because it owns the session's shared, public ETS tables (`:work_queue`, `:idle_queue`, `:tombs`, `:traces`, `:provenance`, `:stats`, `:suggestions`) and every other process fetches its table references from it. Next come the `TaskSupervisor` that isolates the asynchronous CSP dispatches, the `Manager` that orchestrates the search, and the three peer agents: the `ContradictionAgent` ($\mathcal{CA}$), the `SuggestionAgent` ($\mathcal{SA}$) and the `ModelAgent` ($\mathcal{MA}$), which are named by those symbols throughout. The worker pool's `DynamicSupervisor` is last. Figure 6.1 shows the same tree.

Both the order of children and the restart strategy carry meaning. `SessionSupervisor` uses `:rest_for_one`: children are started in the listed order, and if one crashes, it and everything started *after* it restart, while everything started *before* it stays untouched. The `EtsKeeper` comes first so that it survives a crash of any downstream process, since the tables holding statistics, traces, and the work queue outlive a `Manager` or agent failure and remain available for post-mortem


```

1 children = [
2   {ShotTx.Prover.EtsKeeper, {session_id, params}},
3   {Task.Supervisor, name: via(session_id, :task_supervisor)},
4   {ShotTx.Prover.Manager, {session_id, formulas, defs, params}},
5   {ShotTx.Prover.ContradictionAgent, {session_id, params}},
6   {ShotTx.Prover.SuggestionAgent, {session_id, params}},
7   {ShotTx.Prover.ModelAgent, {session_id, params}},
8   # DynamicSupervisor for the worker pool (started last)
9   worker_pool_child(session_id)
10 ]
11
12 Supervisor.init(children, strategy: :rest_for_one)

```

Listing 6.1: ShotTx.Prover.SessionSupervisor.init/1 (children, in start order)

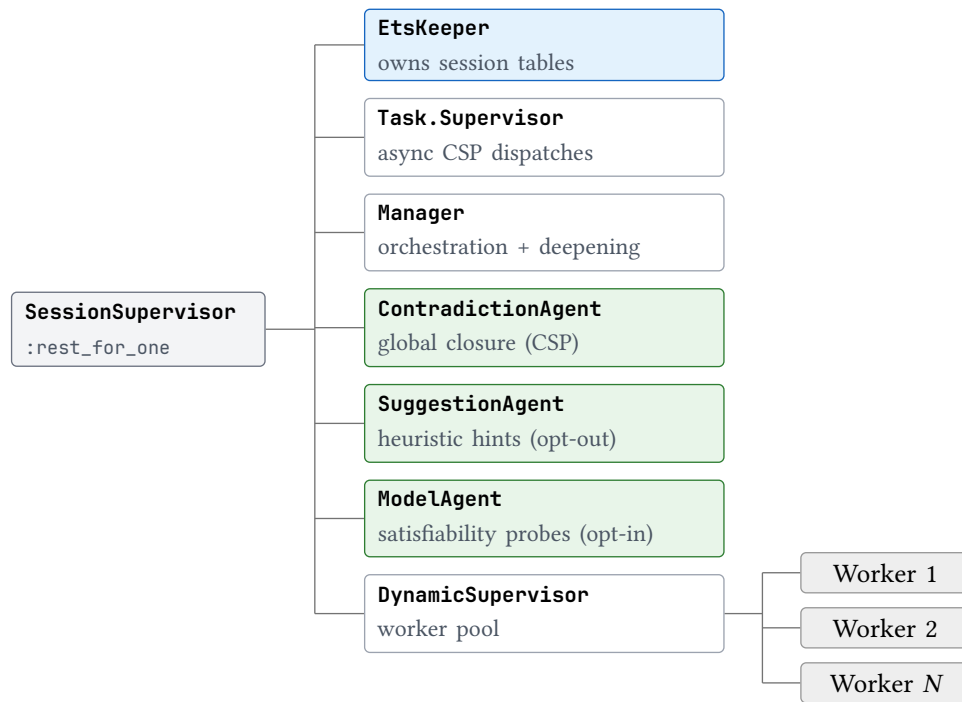


Figure 6.1: The session supervision tree. The EtsKeeper is started first, so it survives a crash of any downstream process; the worker pool’s DynamicSupervisor is started last.

inspection. The worker pool comes last, under its own DynamicSupervisor, because workers are the most numerous and the most disposable.

ShotTx.Prover.SessionSupervisor.init/1 lists these children in this order, with strategy: :rest_for_one. The whole tree is addressed through a Registry keyed on {session_id, name}, so multiple proof sessions run concurrently in one VM without colliding. The prover is itself safe to call from parallel callers. The public ShotTx.Prover.prove/* starts one tree under the top-level ShotTx.SessionSpawner, blocks synchronously on the Manager until the search terminates, and lets the tree

shut down.

6.4 Pure Branch Logic, Stateful Shell

Workers are where the tableau rules actually fire, and their design is the second decision of consequence after the rigidity discipline. A worker is a long-lived process that repeatedly *steals* a branch from the work queue, expands it one step, and deals with the result. The expansion itself, however, is written as a *pure function* that touches no process state and performs no side-effects; it returns a description of what should happen, and the worker carries it out (Outbox pattern [HW03]).

Branch.step/4 takes a branch and the current search bounds and returns one of a fixed set of *results*, each a plain data value. The calculus is Chapter 3's rule set, and the result kinds are only the shapes into which the shell must dispatch:

- `{:continue, branch', effect}`: the branch advanced; effect is a (possibly empty) intended side-effect for the shell to run;
- `{:split, a, b}`: a β -style branch into two siblings;
- `{:instantiate, branches}`: a constant/parameter instantiation fan-out into several branches;
- `{:closed, branch'}`: the branch closed by a ground contradiction;
- `{:idle, branch'}`: the queue is empty but γ -rules are parked; wait for deepening;
- `{:saturated, {defs, literals}}`: the queue is empty and `model_certain?/2` holds: the branch is open, and its literals are a candidate countermodel;
- `{:exhausted, branch'}`: the queue is empty and `model_certain?/2` fails: no rule remains, but satisfiability is undecided.

This purity makes the concurrency tractable. Because step/4 is a pure function of a branch, a branch can be moved between processes freely, re-run deterministically, traced without instrumentation, and reasoned about in isolation from the scheduler. All the genuinely hard concurrency lives in the thin shell that interprets the seven results; the logic of the calculus is contained in a function that is independent of any process.

ShotTx.Prover.Branch is documented as the pure functional core. ShotTx.Prover.Worker is the shell: `checkout_work/1` steals a `{priority_key, branch}` from the `:work_queue` ETS table, `step/4` runs, and `handle_step_result/2` dispatches. Splits and instantiations push their new branches back onto the queue with `push_work/3`; every lifecycle result also publishes the branch's trace to the `:traces` table and *broadcasts an evidence message* (below). The two effect kinds a `:continue` may carry (`{:notify_ca, clashes}` and `{:record_provenance, records}`) are exactly the two writes a branch is allowed to request but not perform itself. To stay responsive, a worker yields after a bounded number of steps rather than running a branch to completion, so no single branch can starve the pool.

The worker never binds a free variable. When a branch discovers that a literal pair *would* close it under some unifier, the worker broadcasts `{:local_clashes, branch_id, clashes}` without unifying or committing, and moves on. The commitment is the concern of $\mathcal{CA}$.

6.5 The Evidence Stream

The channel between workers and $\mathcal{CA}$ is a published stream rather than a direct call. Every worker, on every branch lifecycle event, broadcasts one message to a per-session PubSub topic named `branch_evidence_<session>`. Two agents subscribe to that topic and receive every message independently; neither the worker nor the topic knows who is listening.

The published events are exactly the branch lifecycle: `{:branch_split, parent_id, child_ids}`, `{:branch_closed, branch_id}`, `{:branch_saturated, branch_id, {defs, literals}}`, `{:branch_exhausted, branch_id}`, and `{:local_clashes, branch_id, clashes}`.

Publishing rather than calling decouples the two sides, at a cost the design accepts explicitly. Because the worker-agent edge is a broadcast, not a synchronous call, the BEAM's per-pair message-ordering guarantee no longer serialises events across *different* workers. A `:branch_closed` for a child can arrive at a subscriber *before* the `:branch_split` that announced the child existed. Both $\mathcal{CA}$ and $\mathcal{SA}$ are written to tolerate exactly this: the split handler excludes any child already recorded as closed, and the closure handler tolerates branches it has never seen as active. The strict ordering guarantee is given up in exchange for the decoupling a published stream provides, and the cost is borne by defensive handlers rather than by a synchronisation point.

The broadcast goes through a Registry dispatch on `ShotTx.Prover.PubSub` to the `"branch_evidence_#{session_id}"` topic (`Worker.broadcast_evidence/2`); $\mathcal{CA}$ and $\mathcal{SA}$ each `Registry.register` on that topic in their `init/1`. The moduledoc names the out-of-order case: a child's closure overtaking its parent's split, and both handlers guard against it. Manager commands (`:settle`, `:verify_all_closed`, `:verify_csa`, `:verify_exhausted`) still arrive as direct `GenServer` calls and casts, because those are *coordination*, not evidence: the distinction between broadcast evidence with many subscribers and a direct command with one recipient is maintained throughout the design.

6.6 Global Closure as Constraint Satisfaction

$\mathcal{CA}$ accumulates, from the evidence stream, one thing per open branch: the set of literal pairs that *could* close that branch, each pair carrying the unification problem whose solution would close it. Global closure is then a constraint-satisfaction problem: choose, for each open branch, *one* of its closing options, such that the chosen options' unification problems have a *common* solution.

Here the architecture reuses the procedure of Chapter 4. The CSP is solved without a bespoke solver, by handing the *union* of the chosen options' equations to the very pre-unification procedure of Chapter 4 and asking for a single solution. If one exists, it is a global pre-unifier closing every branch; if none does across all choices, the tableau is not yet closed.

Definition 6.2 (Global closure). Let each open branch b_i contribute a set O_i of closing options, each option a set of unification equations. A *global closure* is a choice $o_i \in O_i$ for each i together with a single pre-unifier Θ of $\bigcup_i o_i$. The tableau is closed iff such a choice and pre-unifier exist.

$\mathcal{CA}$'s internal `find_global_closure/2` sorts the per-branch option lists by size, forms their cartesian product lazily, and for each combined choice runs `ShotUn.unify(Enum.concat(choice), depth) |> Enum.take(1)`, which is Chapter 4's lazy stream, asked for its first element. The first choice whose union unifies is returned. Two measures bound the work: the cartesian product is a lazy `Stream`, so it stops at the first success without materialising every combination, and the unification stream is truncated at one solution, so the CSP decides existence rather than enumerating solutions. The search depth is the session's `unification_depth`, the same bound Chapter 4 described, so raising it under iterative deepening deepens both the branch-level and the global unification in lock-step.

A closed branch is *removed* from the active set the moment its closure is recorded: a closed branch no longer constrains the CSP, and leaving it in would force the solver to pick a dummy option for it, inflating the cartesian product. When the active set empties, every leaf is closed and the tableau is closed under the empty substitution, reported internally as `:unsat` (the negated conjecture is unsatisfiable, i.e. the conjecture is a theorem).

The CSP is dispatched on a supervised `Task` (this is what the `Task.Supervisor` in the tree is for) so that a long solve does not block $\mathcal{CA}$ from ingesting new evidence, and it runs eagerly after every clash or split as long as no other solve is in flight. The prover attempts closure at the earliest point at which the evidence permits it, rather than only when the search stalls.

6.7 The Message Flow

The components combine into a single message sequence. Figure 6.2 traces one branch from checkout to a global closure that ends the session. It is the prover's principal execution path: the worker expands and broadcasts, $\mathcal{CA}$ accumulates and solves, the Manager adjudicates and returns.

Three features of this flow determine the architecture's behaviour. First, the worker and $\mathcal{CA}$ never call each other. The worker's only outbound communication about closure is a broadcast; $\mathcal{CA}$'s only inbound is a subscription. Either could be replaced, duplicated, or (in $\mathcal{CA}$'s case) joined by a second subscriber without any modification to the worker. That is what $\mathcal{SA}$ and $\mathcal{MA}$ do. Second, the CSP fires *eagerly and asynchronously*. Every piece of new evidence can trigger a solve, and the solve runs on a `Task` so $\mathcal{CA}$ remains responsive. The prover therefore attempts termination continuously and is never serialised on the closure check. Third, the Manager's role in the steady state is small (seed, spawn, adjudicate the final result) and grows only when the search stalls.

6.8 Iterative Deepening

The search bounds of Chapter 3 (the γ -instantiation limit and the primitive-substitution depth) are not fixed for a session. They start low and are *raised* when

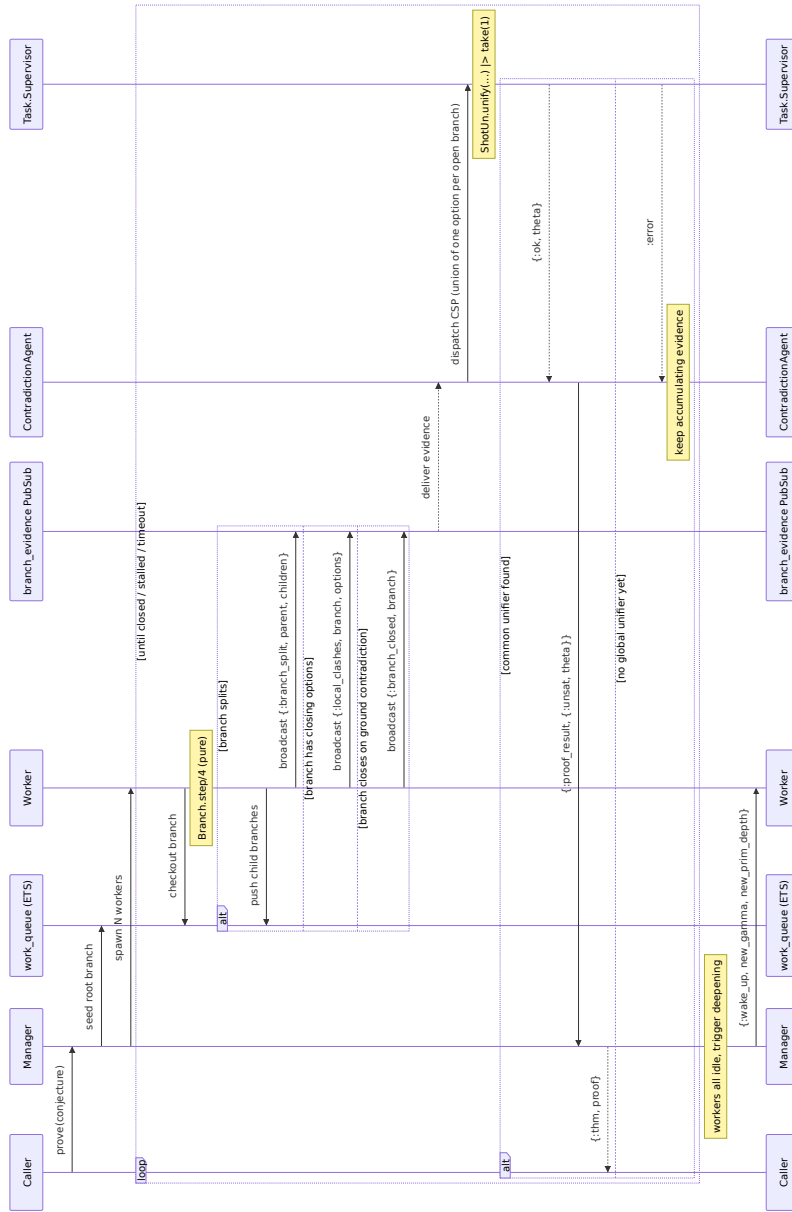


Figure 6.2: The message sequence from a caller's `prove(conjecture)` through worker expansion, evidence broadcast, and `ContradictionAgent` closure, to the Manager's reply.

the search saturates, so shallow proofs are attempted first and the cost of greater depth is incurred only when no shallow proof exists. In this architecture, deepening is a coordination event that leaves the existing branches in place.

When a worker finds no applicable rule within budget, it parks the branch on the `:idle_queue` and reports idle to the Manager. When *all* workers are idle, the Manager adjudicates before it deepens, and what it does depends on why the pool stalled. If some branch saturated, it asks the CA to check the candidate countermodel. If branches are merely parked, it asks the CA to *settle*: one last CSP over the parked state, in case a global closure exists that no single branch could see. Only if that fails does it increment the γ and prim-subst limits and broadcast `{:wake_up, new_gamma, new_prim_depth}` to the workers, which pull their parked branches back off the idle queue and resume from where they stopped. If nothing is parked but some branch is exhausted, it asks for one final closure check before answering `:unknown`; and if nothing is parked or exhausted, every branch is closed and it asks the CA to verify that. Nothing is recomputed in any of these paths; deepening is applied at the frontier where the previous bound was exhausted.

`Manager.check_and_trigger_deepening/1` fires when the idle-worker set becomes full and dispatches on that four-way condition, in the order saturated, parked, exhausted, all-closed; parked branches are checked before exhausted ones because an exhausted branch is a dead end for itself, not for the tableau. `deepen_or_report_unknown/1` either reports `:unknown` (if iterative deepening is disabled, which is an ablation parameter) or raises the limits and wakes the pool. The `settle` call is bounded by whatever remains of the proof's own deadline rather than run to completion: while the Manager blocks on it, it cannot process its own `:timeout`, and the CSP it is waiting for is an exponential cartesian product. Giving up on a `settle` is safe, a late answer is treated as `:open`, which merely deepens or falls through to the timeout already queued behind it. Because parked branches retain their full state on the idle queue, and because the `:settle` path reuses the same eager CSP machinery, the stall-settle-deepen cycle adds no recomputation: it is the same processes and the same tables, only deeper.

6.9 The Peer Agents

The evidence stream was built for $\mathcal{CA}$, but nothing about it is specific to $\mathcal{CA}$. Because closure evidence is *published*, other processes can subscribe to the same stream and do other things with it, without the workers needing to know. The prover ships two such peers, $\mathcal{SA}$ and $\mathcal{MA}$, and they are the clearest demonstration of what the published-evidence design permits. Chapter 7 treats both in full, alongside the proof reconstruction that reads the same tables; this section establishes only what the architecture needs.

The suggestion agent $\mathcal{SA}$ runs a strictly *weaker* unification than $\mathcal{CA}$: pair-level only, never the multi-branch CSP. Where $\mathcal{CA}$ asks for a pre-unifier Θ closing all branches simultaneously, $\mathcal{SA}$ asks heuristically for a variable instantiation that closes some other branch. It turns each resulting $X \mapsto t$ into an instantiation *hint* using the `:provenance` table to recover which quantifier introduced X and on which branch, and publishes the hint into the `:suggestions` table. Workers, on checking out a branch, splice applicable hints in as cheap synthetic instantiation steps, priced at

2 alongside the α -rules. The whole mechanism sits behind `suggestions_enabled`, which is on per default. Setting it to false is the ablation: $\mathcal{SA}$ then does not even subscribe, and the prover's behaviour is that of $\mathcal{CA}$ plus workers. A second parameter, `suggestion_cascade_ceiling` (3), bounds the feedback loop in which an applied suggestion produces fresh clashes, hence fresh unifiers and fresh suggestions: each hint may be spliced at most that many times across all descendants of the branch on which it originated.

$\mathcal{SA}$ is more than an optimisation, for a reason that follows from the rigidity this chapter opened with. A branch never applies a substitution to itself; the substitution lives in $\mathcal{CA}$'s solution and arrives, if at all, only at the end. The branch therefore never sees the *structure* that applying Θ 's substitution θ would have introduced: the new redexes, the new heads, the new candidate closing pairs that only exist once X has become t . The calculus can still reach that structure, but only through γ and primitive substitution, which enumerate blindly and are the two most expensive rules it has. $\mathcal{SA}$ reaches the same structure at lower cost: it takes a substitution that some *other* branch's evidence has already shown to be plausible and offers it here as an ordinary instantiation step, so the structure appears without an enumeration step.

The model agent $\mathcal{MA}$ is $\mathcal{CA}$'s dual. $\mathcal{CA}$'s goal is *global closure* (the conjecture is a theorem); $\mathcal{MA}$'s goal is *satisfiability* (the conjecture has a countermodel). It periodically enumerates the open, saturated branches and dispatches each to an external model finder through a pluggable backend; a `:sat` verdict on any branch terminates the session with a confirmed countermodel, reported to the Manager through the same idiom $\mathcal{CA}$ uses. Its backend is a behaviour connecting to a Isabelle/Nitpick server or staying passive, selected by `model_agent_backend` (`:none` by default). $\mathcal{MA}$ is evidence that the architecture generalises: the same published stream feeds a prover of theorems and a finder of countermodels, running side by side.

Both peers `Registry.register` on `branch_evidence_<session>` exactly as $\mathcal{CA}$ does, and both are children of the `SessionSupervisor` peer to $\mathcal{CA}$. $\mathcal{SA}$'s hints flow `worker→(evidence)→ SA →:suggestions table→worker`, which is a full loop through shared memory that never touches $\mathcal{CA}$. $\mathcal{MA}$ isolates each backend probe on a supervised, monitored Task, so a crashing model finder frees its branch for re-probing rather than blocking $\mathcal{MA}$, and it skips branches whose frontier is smaller than `model_agent_min_frontier` (3) or larger than `model_agent_max_frontier` (100), with at most `model_agent_max_in_flight` (2) probes outstanding. The default configuration runs workers, $\mathcal{CA}$ and $\mathcal{SA}$, with $\mathcal{MA}$ inert at `model_agent_backend: :none`. Each of the three (`contradiction_agent`, `suggestions_enabled`, `model_agent_backend`) is an independently ablatable layer. Chapter 8 ablates the first two; $\mathcal{MA}$ stayed at `:none` throughout that study, so its contribution is not measured there.

6.10 A Proof, End to End

Everything above runs behind a single synchronous call. The public `prove/*` entry point starts the session tree, blocks on the Manager, and returns a verdict once the search terminates. We prove a small higher-order theorem and inspect the result: $(\forall X. p(X)) \supset p(a)$, a one- γ , one-branch theorem.

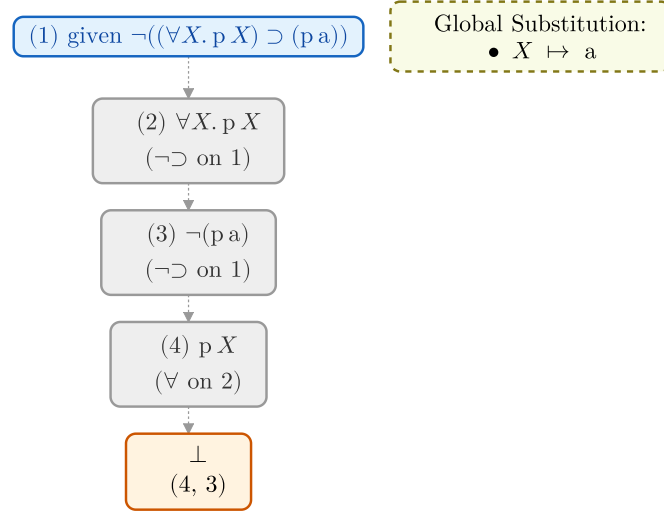


Figure 6.3: The reconstructed proof of $(\forall X. p(X)) \supset p(a)$.

Verdict: THM

The negation is unsatisfiable, so the conjecture is a theorem. `prove/*` returns more than a verdict, however. On success it returns `{:thm, %ShotTx.Proof{}}`, a reconstructed derivation (Chapter 7 builds these from the `:traces` table). We can render that actual proof object as a tree, so what the concurrent search found is shown as an ordinary textbook derivation, as in Figure 6.3.

Behind this one tree: the Manager seeded the negated conjecture, a worker applied $\neg\supset$ to split off $\forall X. p X$ and $\neg(p a)$, the γ -rule introduced a fresh X for the universal, the worker broadcast the clash $\{p X, \neg p a\}$, $\mathcal{CA}$'s CSP handed $\{p X \stackrel{?}{=} p a\}$ to `ShotUn.unify` and got $[X \mapsto a]$, and that global substitution (closing the single open branch) was reported back and reconstructed into the derivation shown. The unifier in the proof's substitution is the one Chapter 4 produces for this disagreement: local expansion, published clash, global unification.

For a theorem with a genuine branch split, the same machinery does strictly more. Several branches broadcast clashes, and the CSP must find one pre-unifier Θ satisfying all of them at once. The shape of the interaction is unchanged. Whether the tableau has one open branch or many, closure is always collecting the options and unifying their union.

6.11 Discussion

The architecture follows from the rigidity constraint by a chain of steps, each a consequence of the one before it. Rigidity requires closure to be decided globally. A global decision over shared evidence is taken by a single serial process. The branch expansions feeding that decision carry no mutual dependency and admit parallel execution. Parallel producers and a shared consumer are decoupled by a published stream, and a stream that exists for one consumer admits further ones at negligible cost, which is what makes a countermodel finder and a heuristic guide available.

The BEAM supplies cheap processes, supervised failure isolation and shared ETS, so each step of the chain is inexpensive to realise.

The result is a prover whose *logic* lives in pure functions (the calculus of Chapters 3 to 5) and whose *coordination* lives in a small number of OTP processes, joined at one point: where a branch's local clash becomes global evidence. Chapter 7 follows the parts this chapter has only placed: the two peer agents $\mathcal{SA}$ and $\mathcal{MA}$, and the proof reconstruction that reads the `:traces` table. Chapter 8 measures the contribution of each component, ablating the agents, the deepening, and the eager CSP one at a time.

Peer Agents and Proof Reconstruction

7.1 Overview

The observation Chapter 6 closed on was not developed there: once closure evidence is *published* rather than called for, a second consumer costs nothing. The workers broadcast, $\mathcal{CA}$ subscribes, and any further process registering on the same topic sees the same stream without a line of the worker changing. This chapter develops the three components on the other side of that stream.

Two of them are the peer agents Chapter 6 placed but did not open. $\mathcal{SA}$ turns evidence about one branch into a cheap instantiation hint for another. $\mathcal{MA}$ turns a saturated branch into a satisfiability probe against an external model finder. Neither is required for the calculus to be complete, and both are ablatable, which is what makes them measurable in Chapter 8.

The third component reads a different shared table. Every branch writes its own derivation history to `:traces` as it proceeds, and a session ending in a verdict has, in that table, what is needed to rebuild the derivation as a textbook object. Proof reconstruction makes the result of the concurrent search checkable: whatever route the population of workers took, the artefact returned is an ordinary tableau proof.

The lineage of this design is set out in Section 1.4: Ω -Ants and its resource-adaptive form OANTS [BS98, Sor01, BSJK08], the cooperative use of an external reasoner in LEO-II [BSJK05, BPTF08], and the agent framework of Leo-III on the LeoPARD substrate [SWB16, BSW17, WSB15]. Two differences bear on the agents of this chapter. In Ω -Ants the blackboard is the coordination point and the agents exist to fill it, whereas here the shared tables are passive and coordination happens through a broadcast, so an agent is added by subscribing rather than by registration with a scheduler. Ω -Ants agents also suggest the next command to a proof planner, whereas $\mathcal{SA}$ and $\mathcal{MA}$ never direct the search: one publishes hints that workers may ignore, the other answers a question the search did not ask.

7.2 The Suggestion Agent

A branch never applies a substitution to itself. Chapter 3's rules are binding-free by construction and Chapter 6's workers may not commit to a value for any free variable, so the pre-unifier Θ closing the tableau lives in $\mathcal{CA}$'s solution and arrives, if it arrives, at the end. The consequence is easily overlooked: the branch never observes the *structure* that applying Θ 's substitution θ would have introduced. Once X becomes t , new redexes appear, heads that were flexible become rigid, and formulae that were opaque become decomposable. Each of those is a rule application the branch could have made, and a closing pair it could have found. This is the cost at which rigidity secures soundness under parallel search.

The calculus can still reach that structure. It reaches it through γ , which enu-

merates instances, and through primitive substitution, which enumerates general bindings. Both are the most expensive rules in Chapter 3’s cost table, and both enumerate blindly. $\mathcal{SA}$ reaches the same structure at lower cost, by taking a substitution that another branch’s evidence has already shown plausible and offering it here as an ordinary instantiation step.

Definition 7.1 (Instantiation Hint). Let φ, ψ be a disagreeing literal pair reported by some branch, and let σ be a pre-unifier of that pair. For each binding $X \mapsto t$ in σ , let Φ be the matrix of the rule that introduced X and B_0 the branch on which it was introduced. The pair (Φ, t) is an *instantiation hint originating on B_0* . A branch B *admits* the hint if B_0 is a prefix of B , and applying it adds $\Phi(t)$ to B as an ordinary instance.

The definition rests on three conditions, and each is a point at which no hint is produced.

The first is the matrix. A hint is meaningful only if the quantifier that introduced X can be identified again, which is what the `:provenance` table records: for every fresh variable, the matrix, the source formula, the originating branch, and whether the variable came from γ or from a prim-subst hole. Without that record a unifier is a statement about terms; with it, a unifier becomes a statement about a rule that some branch still has available.

The second is the originating branch, which is what makes reuse of a hint sound. Hints are keyed by branch prefix, and a worker splices only those originating on an ancestor of the branch it holds. A hint originating on a sibling is not admitted, since the matrix it names need not exist there.

The third is the type. A hint is applied as an application of the matrix to the term, so the term must inhabit the matrix’s argument type. A well-typed σ does not guarantee this, because the holes of a general binding carry the binding’s types rather than the quantified variable’s. Mismatches are dropped at publication.

The pipeline is five steps over one table. $\mathcal{SA}$ ingests `{:local_clashes, branch_id, pairs}` from the topic $\mathcal{CA}$ also reads. It unifies each pair on its own and takes at most one solution, since a multi-pair regime would be the CSP and the CSP belongs to $\mathcal{CA}$; $\mathcal{SA}$ is therefore weaker than $\mathcal{CA}$ by construction. Each binding of the resulting σ is looked up in `:provenance`, type-checked against the matrix, and written to the `:suggestions` table under the key `{birth_branch, recipe, term}` with `:ets.insert_new/2`, so a hint published twice is stored once. A successful insert dispatches `:work_available`, which wakes idle workers instead of leaving them to the next scan.

Splicing happens at checkout and at yield rather than at publication. A worker holding a branch traverses its own ancestor prefixes, matches them against the table, and injects `{:suggested_instantiate, recipe, term}` onto the branch’s queue. The synthetic rule is priced at 2, cheaper than γ and far cheaper than prim-subst, but dearer than the atomic rule at 1, so a hint never preempts closure detection on a branch about to close.

Dropping a type-mismatched hint costs nothing. A hint is a search shortcut, and this one names no formula the branch could hold. Publishing it would raise inside rule application, on a different branch from the one that produced the evidence, and cost that branch its verdict.

Definition 7.2 (Cascade). A spliced hint may produce fresh disagreements, hence fresh pre-unifiers, hence fresh hints. The *cascade depth* of a hint is the number of times it has been spliced across all descendants of its birth branch, and splicing is refused once that depth exceeds a fixed ceiling.

The ceiling is `suggestion_cascade_ceiling`, default 3. The counter occupies a fixed position in the suggestion row and is bumped with `:ets.update_counter/3` at the splice site, in a single atomic operation: two workers racing for the last slot cannot both take it, and no lock is required. This closes the feedback loop between $\mathcal{SA}$ and the workers, which runs from worker to evidence to $\mathcal{SA}$ to the `:suggestions` table and back to a worker, entirely through shared memory and without touching $\mathcal{CA}$.

$\mathcal{SA}$ never declares closure. Pair-level search cannot witness a multi-branch closure, so `:unsat` remains $\mathcal{CA}$'s alone, and the worst a wrong hint can do is waste a step. That asymmetry is why the agent is safe to leave on by default: it can only add formulae the γ -rule was entitled to add.

The ablation is a single flag. Running one problem both ways shows the verdict unchanged and the route different, for the conjecture $(\forall X. p(X)) \supset ((p(a)) \wedge (p(b)))$:

- `suggestions_enabled: true` gives THM
- `suggestions_enabled: false` gives THM

Both rows report THM. $\mathcal{SA}$ is a heuristic over a complete calculus, so switching it off may cost steps but never the verdict; a discrepancy here would indicate a bug in the splice rather than a property of the problem. Chapter 8 runs the comparison across the TPTP corpus, where the quantity recorded is the number of problems solved; the per-rule counters that would show the γ -instantiations avoided are not written to the sweep's rows.

7.3 The Model Agent

$\mathcal{MA}$ inverts the question. $\mathcal{CA}$ collects evidence towards a global pre-unifier Θ closing every branch, while $\mathcal{MA}$ takes a single branch that has stopped producing rules and asks an external tool whether it describes a model. A `:sat` verdict on any branch ends the session, because one satisfiable branch witnesses satisfiability of the whole input. Consulting an external reasoner about a subproblem and integrating the verdict that comes back is the arrangement LEO-II uses for its first-order backend [BSJK05, BPTF08]; what differs here is that the external system is asked for a model rather than for a refutation.

This is a separate agent rather than a check inside the branch because of the boundary drawn in Chapter 3. `model_certain?` decides an under-approximation of openness using only what the calculus can see, namely identity, bounded unifiability, and the oriented equations of $\mathcal{E}(B)$, and it reports *exhausted* rather than open whenever extensionality would be needed to settle a pair. $\mathcal{MA}$ decides some of those exhausted branches by asking a tool that decides more than the calculus does.

Definition 7.3 (Frontier Projection). Write $\text{Fr}(B)$ for the frontier of a branch and $\text{Def}(B)$ for its definition map. The *frontier projection* of B is the problem whose axioms are $\text{Fr}(B)$, whose definitions are $\text{Def}(B)$, and which has no conjecture. A model of the projection is a model of B .

The projection ships the frontier rather than $\text{Lit}(B)$, which is what the frontier exists for. The frontier tracks what the branch commits to: a rule consuming its source replaces it there, while γ and prim-subst leave their sources in place, because a fresh instance is a hint rather than a new commitment. Paramodulants and γ -fresh atoms therefore reach $\text{Lit}(B)$ without entering the frontier, which suits a model finder, since they are consequences and shipping them would enlarge the vocabulary the tool must interpret. Constant declarations are derived from the axioms, with constants occurring only in definitions added afterwards so their types are not lost, and the axiom order is sorted so the emitted problem is deterministic.

The agent is a timer loop. Each element of it keeps an expensive external call from becoming a bottleneck.

- It scans the live branches on a tick, reading them out of the shared work and idle queues and discarding any whose ancestor has been tombstoned.
 - A branch is *eligible* when its frontier is neither too small nor too large, when its frontier version has advanced since the last probe of that branch, and when no probe for it is in flight. Versioning stops the agent from re-asking the same question, since a branch that has not changed its commitments will not answer differently.
 - Eligible branches are dispatched up to a cap on concurrent probes, each on a supervised, monitored task.
 - Verdicts arrive asynchronously. `:sat` ends the session, a backend error clears the recorded version so the next tick retries, and anything else is dropped.
-

Both frontier bounds concern the model finder rather than the prover. A frontier below `model_agent_min_frontier` (3) carries no information, since a branch asserting a single atom is trivially satisfiable, and one above `model_agent_max_frontier` (100) exceeds what finite model finders handle, since they scale poorly in vocabulary. `model_agent_max_in_flight` (2) reflects that an Isabelle server serialises model finding, so further concurrency only fills a queue.

Failure isolation is why the probes are `async_nolink` and monitored. A crashing backend must not take the agent with it, and the `:DOWN` handler removes the branch from the in-flight map so it becomes eligible again rather than remaining stuck. This applies the supervision argument of Chapter 6 at the finest granularity the system has, where the unit of failure is one probe on one branch.

The backend is a behaviour with two callbacks, `split` so a probe never blocks the agent's mailbox: a dispatch returning a handle immediately, and a wait on that handle bounded by a time budget. Three implementations exist. `:none` is the default, under which the agent starts, logs once, and never schedules a tick, so the layer costs one idle process. A stub returning `:unknown` unconditionally is the safe choice for tests. The Isabelle/Nitpick backend [BN10] sends the projected problem to a warm server and runs `nitpick[satisfy]`, which consumes the shape the projection produces, namely axioms without a conjecture. Because the behaviour is a plain callback module, a different finder is a new module and nothing further.

When $\mathcal{MA}$ is about to declare `:sat` it checks whether the session has already been decided, and logs an error and emits telemetry if so. A simultaneous `:sat` from $\mathcal{MA}$ and `:unsat` from $\mathcal{CA}$ on a consistent problem is impossible, so the check does not guard against a race. It is an alarm for a soundness bug in the frontier or in atom decomposition, placed where such a bug would first become visible.

With the backend at its default, satisfiability is still reported for branches the calculus can settle by itself, and the result carries a model rather than a derivation. For the conjecture $p(a) \supset p(b)$:

CSA
 $\neg(pb), pa$

The verdict is CSA. The branch separates pa from $\neg(pb)$ at an argument position holding two distinct rigid base-type constants, which is the configuration a free term model satisfies, so Section 3.1 calls the branch open without an external probe. $\mathcal{MA}$ exists for the branches that fail that test and are reported exhausted.

7.4 Proof Reconstruction

The prover returns more than a verdict. On `:unsat` it returns a derivation tree, which is rebuilt after the search from the trace records the branches wrote as they proceeded. Recovering a readable derivation from a compact record of a search is the problem expansion trees [Mil87] address for higher-order proofs.

Definition 7.4 (Trace). The *trace* of a branch is the sequence of entries $(s, r, \bar{p})$, oldest first, where r is a rule applied on that branch, s the formula it was applied to, and $\bar{p}$ the formulae it produced. A branch that closed conditionally carries a final entry naming the pair whose unification closes it.

Traces live in the `:traces` table, keyed by branch id, and are written by the worker at every lifecycle event rather than at the end. A branch that is split or instantiated publishes each child's trace and deletes the parent's, so the table holds the current leaves and never accumulates a redundant interior. The history each branch carries is recorded inside rule application, after the rule has run, so the produced list is exact rather than predicted, and a closed branch's trace includes the step that closed it. Reconstruction reads the table once, at the end, from $\mathcal{CA}$'s own state.

Rebuilding a tree from a set of leaf traces takes two steps. The first turns a trace into a linear sequence of events, discarding bookkeeping that has no place in a derivation. The second merges those sequences into a tree by shared prefix.

Definition 7.5 (Linearisation). The *linearisation* of a branch's trace is the sequence of events obtained by mapping each entry to the formula it contributes to this branch, dropping entries that contributed none. For a branching rule the contribution is the alternative this branch took, selected by consuming one segment of the branch identifier.

Branch identifiers encode the path through every split as suffixes, `_A` and `_B` for the two sides of a β -rule and `_I{idx}` for the `idx`-th child of an (inst), so linearisation reads the identifier as a stack and pops one segment at each branching entry. A trace

can therefore be interpreted without the tableau, because the identifier records the position.

Several entries linearise to nothing, and each corresponds to a step the calculus did not take. A γ or prim-subst firing that produced no instances is a rule parked at its limit. A tautology or contradiction verdict produced no formula. An (inst) with an empty enumeration is the degenerate case of Section 3.8, where the rule reduces to plain admission. The atomic rule appears only where it unfolded a definition, which is why the definition unfolding of Chapter 3 surfaces in the finished proof as its own unfold step.

Definition 7.6 (Derivation). Given the linearisations of all closed branches, the *derivation* is built by grouping the sequences on their first event: sequences agreeing there share a single step, and the construction recurses on their tails. A step whose group splits into several distinct first events acquires one child per group.

Sharing by prefix is what merges the parallel runs into one derivation. Two branches diverging at a β -rule share every step above it once and fork below it, and a run without branching stays a single chain. The initial formulae, the assumptions together with the negated conclusion, become given steps chained above the whole derivation, and each step is numbered as it is created, so the sources of a step are line numbers in the ordinary sense. Closure leaves cite the pair that closed them, and a countermodel leaf carries the branch's atoms and definitions instead of a formula. The seeded $\top$ and $\neg\perp$ of Section 3.8 are stripped from that leaf and from every reported model, so the two entries every branch starts with are never mistaken for facts the search established.

The conjecture Chapter 6 proved end to end can be rendered as text rather than as a diagram, which shows the numbering and the sources directly, for $\forall X. p(X) \supset p(a)$:

1. $\neg((\forall(\lambda. p1)) \supset (pa))$	[given]
2. $\forall(\lambda. p1)$	$[\neg\supset \text{ on } 1]$
3. $\neg(pa)$	$[\neg\supset \text{ on } 1]$
4. pX	$[\forall \text{ on } 2]$
5. $\perp$	$[4, 3]$
Global Substitution: $\{a/X\}$	

The proof object carries both components of the pre-unifier $\Theta = (\theta, C)$ found by $\mathcal{CA}$: the substitution θ and the residual flex-flex set C . The tableau is closed under θ , and C belongs to the answer rather than being omitted from it, since by the definition of Chapter 4 every unifier of C extends θ to a unifier of the closing equations. Chapter 6's rendering of the same object as a Mermaid tree presents this data under a different formatter.

7.5 Discussion

The three components of this chapter differ in function and agree in how they attach to the search. $\mathcal{SA}$ reads the evidence stream and writes a table that workers consult. $\mathcal{MA}$ reads the branch queues and writes to the Manager. Reconstruction reads a table and writes nothing. None of them is called by the search, none can block it, and each can be disabled by a flag or simply left unread.

This bears out the property claimed in Chapter 6 rather than establishing a new one: where closure is published rather than requested, an additional consumer costs close to nothing. What remains open is whether these particular consumers reduce search cost enough to justify their overhead, which Chapter 8 addresses for $\mathcal{SA}$ and $\mathcal{CA}$ by switching each off against a fixed baseline. $\mathcal{MA}$ ran at `model_agent_backend: :none` throughout that study, so neither its cost nor its contribution is measured.

Evaluation

8.1 Overview

The architecture of Chapter 6 and the rules of Chapters 3 to 5 were each introduced with an argument for why they belong in the prover. This chapter measures what each of them contributes, by switching one off at a time against a fixed baseline and running the whole TPTP higher-order corpus at a uniform budget, and then characterises the prover’s coverage on a smaller structured set against five reference systems. Nothing measured here was tuned: every configuration is the default of Chapter 6 with one value changed, each ran once, and no portfolio or per-problem strategy selection took place, so the counts rank the components against one another and are a lower bound on what the calculus reaches.

Three results carry the chapter. The contradiction agent $\mathcal{CA}$ accounts for roughly half of all refutations, since disabling it costs 364 of 682 and nothing else in the matrix is within a factor of four of that. Iterative deepening, demodulation, atom decomposition, finite α -quantification and the initial γ -limit each contribute measurably, and eager definition unfolding improves on the default by 48 problems, while primitive substitution, the suggestion agent $\mathcal{SA}$, instance-based γ -expansion, the primitive substitution batch size and the size of the worker pool all lie inside the run-to-run band of about ten problems established in Section 8.3. The pool is the answer to the question Chapter 1 left open: eight workers perform about three times the inference of one within the same budget and solve two problems fewer, because the additional inference widens the frontier of open branches without raising the rate at which global closure is attempted, and global closure is what produces the refutations.

Every figure here is scored on refutations. Countermodel answers are reported separately and are not counted as solves, for the reason given in Section 8.2: the prover infers the countermodel verdict from a syntactic test and never checks it, so a `CounterSatisfiable` answer is not attested whether or not it agrees with the TPTP status.

8.2 Method and Scoring

The corpus is every TH0 and TH1 problem of the TPTP library [Sut17], 5109 problems in the version used here. Sixteen configurations were run over the full corpus: the baseline, one configuration per component switched off relative to it, and two numeric sweeps at three values each. Each configuration ran once, and each problem received a uniform budget of `timeout` (2000 ms), so a difference in solved count between two rows is attributable to the component under ablation rather than to differing time. The derived tables this chapter reads are in `files/`; the 81,744 per-problem records they summarise are held in the implementation repository.

The sweep ran on commit 2a05f05 between 2026-08-15 and 2026-08-16, on an

Table 8.1: Baseline outcomes over 5109 problems.

Outcome	Problems	Share (%)
timeout	3423	67.0
thm	685	13.4
parse_timeout	628	12.3
parser_error	221	4.3
no_conjecture	77	1.5
csa	62	1.2
unk	8	0.2
prover_error	5	0.1

AMD Ryzen 7 4700U with 15 GiB of memory under Arch Linux, where `worker_pool_size (:auto)` resolves to eight. The follow-up probe of Section 8.6 ran on an AMD Ryzen 7 3700X in a container under a Linux subsystem on Windows 11 (Ubuntu), where the runtime reports sixteen online schedulers. The two pool sizes are therefore eight against one in the matrix and sixteen against one in the probe, which bears on any comparison of the two inference ratios.

An answer counts as a solve when it agrees with the SZS status [Sut08] declared in the problem file, and three conventions narrow that. Refutations only: the countermodel verdict is decided by a pairwise syntactic test over a saturated branch and is never checked, so a countermodel agreeing with the TPTP status rests on the same unvalidated predicate as one contradicting it. Countermodels are therefore reported as a separate column and excluded from the solved counts, and restricting to refutations leaves the ordering of the sixteen configurations unchanged.

Agreement with the problem file: three problems whose header carries no usable status were answered by the prover and are not counted. Scoring is also narrower than the SZS ontology, since only Theorem and CounterSatisfiable are scored and every other status is recorded as undecidable; re-scoring the sweep under the full ontology produced no further disagreements.

8.3 Baseline

The baseline is the default `%ShotTx.Data.Parameters{}` of Chapter 6 with `timeout` set to 2000 ms: both peer agents present in their default state, every calculus rule enabled, iterative deepening on, and `worker_pool_size (:auto)`. One departure from the defaults bounds what the chapter can claim: `model_agent_backend (:none)` in every one of the sixteen configurations, so $\mathcal{MA}$ ran as an idle process throughout and its contribution is not measured here.

`parse_timeout` and `parser_error` in Table 8.1 are properties of `ShotDs.Tptp` rather than of the prover: the parser never sees `%Parameters{}`, so a problem it cannot read is unreadable under every configuration. `no_conjecture` marks problems that are not attemptable as posed.

Removing the 628 parse timeouts, the 221 parse failures and the 77 problems without a conjecture leaves 4183 genuine attempts, of which the baseline solved 747 (thm & csa). One problem in six never reached the prover, so the corpus-level rate is

Table 8.2: Sixteen configurations, refutations and deltas against the baseline. Refs. = refutations, Uniq. = uniquely solved, CMs = countermodels agreeing with the TPTP status, Contra. = countermodels contradicting it.

Configuration	Parameter	Refs.	Δ	Gained	Lost	Uniq.	CMs	Contra.
eager_unfold_defs	unfold_defs: :eager	730	48	72	24	44	61	2
no_beta_variant	beta_variant: false	703	21	41	20	16	62	1
prim_batch_16	prim_subst_batch_size: 16	685	3	21	18	2	61	1
prim_batch_4	prim_subst_batch_size: 4	683	1	24	23	0	61	2
baseline	(none)	682	0	0	0	3	61	1
no_prim_subst	primitive_substitution: false	681	-1	23	24	2	61	1
no_suggestions	suggestions_enabled: false	681	-1	34	35	11	61	2
serial	worker_pool_size: 1	680	-2	36	38	10	61	2
no_instance_based_gamma	instance_based_gamma: false	679	-3	16	19	1	61	2
no_atom_decomposition	atom_decomposition: false	670	-12	26	38	3	61	3
no_finite_o_quantification	finite_o_quantification: false	662	-20	17	37	0	60	1
gamma_limit_3	initial_gamma_limit: 3	661	-21	17	38	1	61	1
gamma_limit_5	initial_gamma_limit: 5	657	-25	18	43	3	61	1
no_demodulation	demodulation: false	641	-41	20	61	6	61	0
no_iterative_deepening	iterative_deepening: false	595	-87	15	102	0	54	2
no_contradiction_agent	contradiction_agent: false	318	-364	5	369	0	61	0

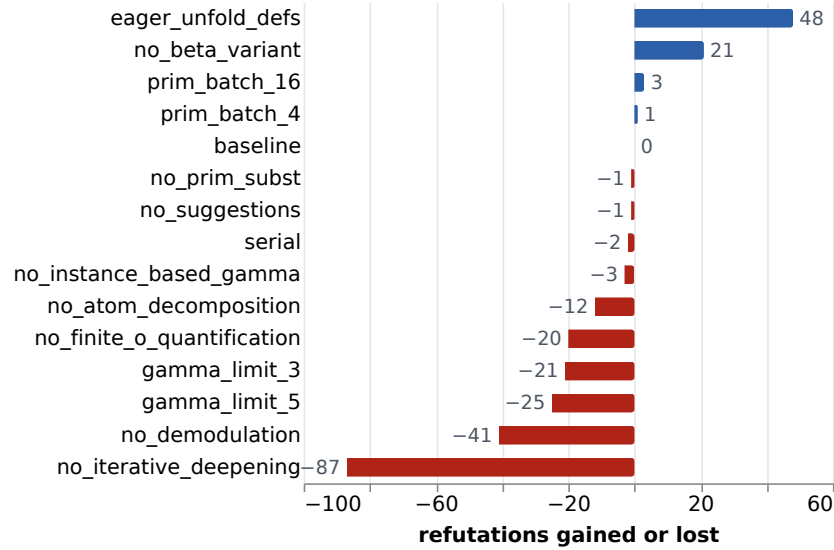


Figure 8.1: Refutations gained and lost against the baseline. no_contradiction_agent is off this scale at -364.

bounded by the parser rather than by the calculus, and repairing the parser is the largest single coverage gain available.

The distribution of solve times over the baseline's refutations is flat well before the budget expires. What the prover solves, it solves quickly, and the problems it does not solve are not close to being solved, which is also why the ablation deltas are stable against the choice of 2000 ms. A few solves lie past the budget, which is the cooperative timeout of Section 8.9.

The three sections that follow read against Table 8.2, one table of deltas from this baseline.

Bars to the right of zero in Figure 8.1 are configurations that solve more than the baseline. Sign is carried by the direction of the bar as well as by the colour, so the

figure needs no legend. Gained and Lost in Table 8.2 are set differences rather than a decomposition of the delta: a configuration can be flat overall while exchanging tens of problems in both directions, which several of them do.

The size of that exchange was measured rather than inferred. The two worker-pool configurations were run three times each over the 1677 problems on which the agent dispatches at least one closure search, at the same budget. Of the 3354 pairs of a configuration and a problem, 84 answered differently across three identical repeats, which is 2.5 %. Estimating each problem’s per-run solve probability from its three draws puts the standard deviation of one configuration’s solved count at 4.0 problems under the pool and 3.4 serial, and the standard deviation of the difference between two independent single runs at 5.3. A net delta inside roughly ten problems is therefore not evidence of a contribution, and the sections below support an ordering into tiers rather than a point estimate of any single delta. The exchanged problems are also not hard ones traded against easy ones: of the 38 problems the baseline solves and the serial configuration does not, the median takes 84 ms and none takes longer than 1497 ms, so the exchange reflects the order in which the search reaches branches rather than the time available to it.

8.4 Ablating the Agents, the Calculus and the Search Control

Two of the three layers of Chapter 7 were switched here. Disabling `contradiction_agent` costs 364 of 682 refutations, and local ground closure alone, which is what remains when `CA` never dispatches a global search, solves 318 problems. The result is the chapter’s largest and the one the architecture predicts, since rigidity makes closure a global constraint and a prover deciding closure only locally answers a weaker question. It also fixes the reading of the other rows, because a configuration solving 318 problems is not far enough into any search to be informative about the rules.

Disabling `suggestions_enabled` costs one problem, which is inside the floor. `SA` is a heuristic over a complete calculus, so this is a statement about the value of the hints rather than about the verdicts: the configuration exchanges 34 problems against 35 and is the third largest source of uniquely solved problems in the matrix, at 11. The hints change which proofs are found without changing how many. Where they go is not in the data, since `TptpRunner` records steps and `rules_total` and not the per-rule breakdown that would separate a hint shortening a proof from one displacing a rule making progress.

`MA` was not ablated. `model_agent_backend (:none)` held in all sixteen configurations, so the layer was inert throughout and this chapter reports nothing about it.

The rules of Chapter 3 divide into three groups by contribution. Two are substantial: removing `demodulation` costs 41 refutations, the largest calculus effect in the matrix, losing 61 problems while gaining 20, and removing `atom_decomposition` costs 12, losing 38 against 26 gained. The second is also the configuration with the most countermodel answers contradicting TPTP, at three, which fits the mechanism: without decomposition a branch reaches saturation earlier and the unchecked countermodel test is consulted sooner. Two more concern *o*-typed subterms: remov-

ing `finite_o_quantification` costs 20, and removing `instance_based_gamma` costs three, which is inside the floor.

One change improves on the default. Setting `unfold_defs` to `:eager` gains 48 refutations, gaining 72 and losing 24, and supplies 44 uniquely solved problems, by a wide margin the largest such count in the matrix. Ten of the 72 gains were reported past the 2000 ms budget, some as late as 6 s, because the prover’s timeout is cooperative, so the gain is real and smaller than 48; and an eager unfolding enlarges the formulae a branch carries, so the improvement is specific to a corpus in which definitions are shallow. That a single non-default value outweighs every component’s contribution is the clearest indication that the baseline is untuned.

One rule is a net cost at this budget. Switching off `beta_variant` gains 21 refutations while losing 20 and gaining 41, so the rule contributes 20 refutations and displaces 41.

Two numeric parameters were swept rather than switched, and both say the default is the value to keep. The initial γ -limit was run at 1, 3 and 5: the default of 1 is best by a wide margin, since 3 costs 21 refutations and 5 costs 25, and a higher initial limit fills the first deepening round with instantiations that do not contribute to a closure. The primitive substitution batch size was run at 4, 8 and 16, and the three values span three problems in total, which is inside the floor. Neither parameter is worth exposing as a tuning surface, and neither was tuned here.

The search control was ablated at one switch. Removing `iterative_deepening` costs 87 refutations and loses 102 problems against 15 gained, which is the second largest effect in the matrix and the largest that is not an agent. The deepening schedule of Chapter 3 is doing what it was introduced to do: the alternative, a single search at the full bound, exhausts the budget in a region of the space that contains no closure. It is also the one configuration in which the countermodel column moves, from 61 agreeing answers to 54, since a branch that saturates at a lower bound never reaches the state that would have closed it.

The remaining search control parameter is the size of the worker pool, which the next section treats on its own.

8.5 Scaling with the Worker Pool

The pool is compared at `worker_pool_size(:auto)`, which resolves to one worker per online scheduler and so to eight on the machine the matrix ran on, against `worker_pool_size: 1`. The two configurations differ in that parameter alone. To keep the comparison free of the parse cache and of the difference in total sweep time, the counters below are restricted to the 3383 problems on which both configurations exhausted the budget, so the wall clock available to each is equal by construction.

In Table 8.3, steps and branches closed locally are worker-side counters; global closure searches is the number of times $\mathcal{CA}$ dispatched a search through `find_global_closure/3`. The last row is over the 705 problems both configurations solved, where the ratio is a latency ratio rather than a throughput one.

The pool multiplies inference by 3.27 and local branch closure by 3.13 within the same wall clock, and increases the number of global closure searches by 1.17, which as a rate is 0.46 global closures per thousand steps against the serial configuration’s 1.28. On the problems both configurations solve the pool is faster by a factor of 1.28

Table 8.3: `worker_pool_size :auto` against 1.

Quantity (statistic)	Pool	Serial	Ratio
problems compared (count)	3383	3383	—
steps (total)	5 414 451	1 654 858	3.27
branches closed locally (total)	121 798	38 853	3.13
peak open branches (median)	66	21	3.14
global closure searches (total)	2469	2115	1.17
global closure searches per 1000 steps (rate)	0.46	1.28	0.36
problems solved by both (count)	705	705	—
wall clock on those, ms (total)	61 003	77 785	1.28

in aggregate, and it solves two problems fewer. The parallel speedup therefore lands on branch expansion, and the component producing roughly half of all refutations does not receive it. Three properties of $\mathcal{CA}$ account for that, each visible in the agent rather than inferred from the numbers.

At most one global closure search runs at a time: `check_global_closure/1` returns immediately when `pending_search` holds a task, so a clash or split arriving during a search increments `evidence_version` and is otherwise discarded. A search is dispatched only when `insufficient_options?/1` is false, which requires every open branch to carry at least one clash candidate, so a single branch without one suppresses the check for all of them. Once dispatched, `find_global_closure/3` enumerates the cartesian product of the per-branch option lists, so a dispatched search costs exponentially more as the frontier widens, and abandons at the proof deadline.

The frontier connects those properties to the measurement: additional workers raise the rate at which branch evidence arrives and hold three times as many branches open, and both make a dispatch less likely and a dispatched search more expensive. The taxonomy of [Bon00] classifies the outcome as a parallel search that is not a faster version of the sequential one, and it is the cost of the partition Chapter 6 made between local expansion and one serialised global decision. What the sweep does not settle is whether the agent is saturated by the cost of each search or starved of opportunities to begin one.

8.6 Closure Search under the Two Pool Sizes

The sweep cannot separate the two accounts of that, because `TptpRunner` exports neither the time spent inside a search nor the number of checks suppressed, although `Stats` records both. A follow-up run recorded them. The two worker-pool configurations were run three times each over the 1677 problems on which either dispatched at least one search during the sweep, at the same 2000 ms budget, with one counter added to the clause of `check_global_closure/1` that returns when a search is already in flight, since that path previously recorded nothing. This run is on the second machine of Section 8.2, where the pool is sixteen workers rather than eight, so its ratios are read against a wider pool than the matrix's.

In Table 8.4 the pool leaves the three quantities that determine the verdict close to unchanged: the number of searches, the time spent searching, and the number of

Table 8.4: ContradictionAgent counters, 5031 runs per configuration.

Measure (statistic)	Pool	Serial	Ratio
tableau steps (total)	14 353 411	2 460 501	5.83
peak open branches (median run)	196	30	6.53
runs dispatching ≥ 1 search (count)	4826	4442	1.09
searches dispatched (total)	9347	8317	1.12
searches per 1000 steps (rate)	0.65	3.38	0.19
checks skipped, no clash options (total)	208 292	175 057	1.19
checks suppressed, search in flight (total)	5 559 732	677 507	8.21
suppressed per search dispatched (rate)	595	81	7.30
recorded time inside search, ms (total)	3 997 654	4 113 722	0.97
share of wall clock in search, % (rate)	46.5	51.6	0.90
longest single search, ms (max)	1982	1977	1.00
branches per search (mean)	21.41	10.37	2.06
candidates per search (mean)	996.84	161.05	6.19
searches finding a closure, % (rate)	12.4	13.6	0.91
searches killed unfinished, % (rate)	17.8	10.8	1.65
problems solved, majority of 3 (count)	389	381	1.02

problems closed. The quantities it does multiply describe the space each individual search has to cover.

The two accounts describe the same mechanism. A closure search occupies the only slot for a large part of the budget, 46.5 % of wall clock under the pool and 51.6 % serial, and every check arriving while it runs is discarded, so searches are at once expensive and infrequent and additional workers cannot make them more frequent. Under the pool 595 checks are discarded for every search that runs, against 81 serial, which at 8.21 is the quantity the pool moves furthest. Saturation is therefore present with a single worker and is not produced by the wider frontier. What the pool changes is what a search has to cover, 6.19 times as many candidates over 2.06 times as many open branches, and 17.8 % of its dispatches are killed by the deadline before finishing against 10.8 % serial.

The 46.5 % understates the cost, since `csp_duration_us` is recorded on the completion path of the search task, so a search killed at the deadline contributes nothing and killing is more common under the pool. The share is the fraction of the proof during which a closure search was running rather than the fraction of the proof's effort diverted into one, because the search runs in its own supervised task.

Three changes to $\mathcal{CA}$ follow from these counters, and Chapter 9 takes them up as future work. A per-search deadline set as a fraction of the remaining budget, rather than the session deadline currently passed as `deadline_us`, would replace one search that cannot finish with several that can, since only 12.4 % of dispatches find a closure. Retaining the most recent suppressed trigger and running it when the slot frees would recover one of the several hundred opportunities now discarded per search. Allowing a small number of searches in flight over disjoint subsets of the open branches would put the pool's remaining schedulers on the step that decides the verdict. On this evidence the wider pool is not otherwise worth its inference.

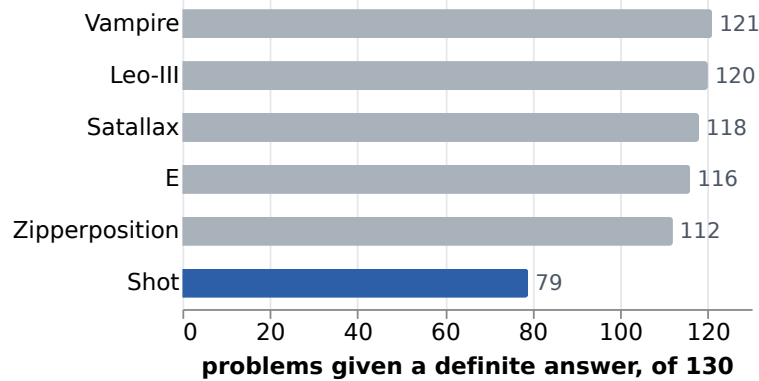


Figure 8.2: Coverage on the structured problem set. Reference provers at a 10 s limit; SHOT at 5 s.

8.7 The Structured Problem Set

The matrix measures the prover’s components against one another. It says nothing about how the prover stands against systems built on other substrates, and it does not characterise the problem classes the search decides, since the TPTP corpus is too heterogeneous for either question and 67 % of it reaches only a timeout. A second and smaller study addresses both. `ShotTx.Benchmark.HoLSuite` normalises the structured higher-order problems of Appendix A into 130 self-contained THF problems, run at the prover’s defaults with timeout (5000 ms), three times over the whole set, each problem in a fresh runtime since state carried between proof sessions would otherwise make a verdict depend on what ran before it. Reference verdicts for the same problem text came from Vampire 5.0.1, E 3.5.1, Zipperposition 2.1 [BBT⁺21], Leo-III 1.8.0 [SB21] and Satallax 3.5 [Bro12] through a public submission service at a 10 s limit on that service’s hardware.

As Figure 8.2 shows, the prover proved 76 of the 130 on all three runs and 79 on the best of them, returned three unknown answers, produced no countermodel, and produced no answer contradicting the reference panel. Of the 130, 122 carry a Theorem consensus across the five reference systems, so the prover decides roughly two thirds of what the panel decides; the remaining eight are the designated non-theorems, which no reference system refutes either.

Over the 79 problems it proves, its median self-reported solve time is 0.035 s, against 0.012 s for E, 0.013 s for Zipperposition, 0.024 s for Vampire and 1.374 s for Leo-III. Compared with the best reference time on each problem it is 2.7 times slower at the median, 43 times at the ninth decile and 388 times at worst, and faster on 14 of the 79, which share a shape: small propositional or reflexivity-style goals that close in a handful of rule applications, where the tableau reaches a contradiction directly and a saturation-based system pays for a clausification it does not need. This is the measurement Chapter 1 conceded would be needed, taken on an untuned prototype, and read together with Section 8.5 the parallelism does not compensate for the constant factor, since the pool multiplies inference rather than closure.

The comparison is loose in two ways that bound how it should be read. The systems ran on different hardware, and the prover’s figure excludes runtime startup while the service’s wall clock includes process launch. Differences below roughly a factor

Table 8.5: The 130 problems by the class in which they are valid. Stable = proved on all runs, Flip = proved on some, Never = never proved.

Model class	Problems	Stable	Flip	Never
$\mathfrak{M}_\beta$	47	37	1	9
$\mathfrak{M}_{\beta\eta}$	1	1	0	0
$\mathfrak{M}_{\beta\mathfrak{b}}$	21	19	1	1
$\mathfrak{M}_{\beta\eta\mathfrak{b}}$	1	0	0	1
$\mathfrak{M}_{\beta\xi}$	6	0	0	6
$\mathfrak{M}_{\beta\xi\mathfrak{b}}$	14	2	1	11
$\mathfrak{M}_{\beta\mathfrak{f}}$	6	0	0	6
$\mathfrak{M}_{\beta\mathfrak{f}\mathfrak{b}}$	9	3	3	3
property $\mathfrak{q}$	3	1	0	2
signature	13	12	0	1
non-theorem	8	0	0	8
warm-up	1	1	0	0

of two carry no signal; what the numbers support is an order of magnitude and the question of which problems each side decides at all.

Six problems were proved on some runs and not others, and the distribution is bimodal rather than borderline: one closes in 33 ms on two runs and exhausts 5000 ms on the third. The shared queue imposes no preference over which open branch a free worker takes, so which worker reaches the productive primitive substitution instance decides the verdict, which is the instability quantified at 2.5 % in Section 8.3.

8.8 Coverage by Model Class

Every example in the set carries the largest model class in which it is valid, over the semantics of [BBK04], and validity in a larger class implies validity in every smaller one. Appendix A sets out the two dimensions and works the examples individually. Grouping the three runs by that annotation separates the prover’s coverage along one line.

The properties the prover decides are β - and η -conversion together with Boolean extensionality; the ones it does not are the two functional principles, the congruence rule for abstractions and full functional extensionality. Of the 70 problems that are valid without either functional principle, 57 are proved on all three runs. Of the 35 that require one of them, five are. The class names in Table 8.5 are those of Appendix A, which sets out the eight classes and the property each one adds.

The two pure cases are the sharpest. Example 13, extension with identity, requires the congruence rule for abstractions and nothing else; Example 14, extension of identity, requires full functional extensionality. Neither is proved in any of its six variants on any of the three runs. At the other end η costs nothing, since terms are held in η -long β -normal form throughout, and Boolean extensionality costs little, with 19 of the 21 problems that need it proved.

The calculus accounts for the line, and Appendix A states the commitment the table measures: the renaming and instantiation rules give Boolean extensionality,

the η -long β -normal representation of Chapter 2 gives η without a rule firing, and the equality-expansion rules give the functional principles. That last commitment is the narrow one. An equality at a functional type expands into extensional equality, so both functional principles are available to a formula that already is an equality, and nothing in Chapter 3 turns a rigid-headed atom pair into a disequality goal, which is the rule identified as absent in Section 3.11. A branch needing one of them to separate two atoms saturates rather than closing, which is what the three unknown answers do, while the variants of the same theorems that state the equality in the conjecture are proved in under 60 ms.

The exceptions locate the line rather than blurring it. Of the nine problems that need functional and Boolean extensionality together, the three proved include Example 21, which states that there are exactly four functions of type $o \rightarrow o$ and which `finite_o_quantification (true)` decides by finite expansion over a pure o -type without any extensionality rule firing. A rule covering one class of extensionality obligation decides the examples inside its scope and leaves the rest untouched, which is the shape a rule for the congruence principle would have to take. The individual problems are not worked here: Appendix A names each example, the class it belongs to and the commitment it tests, and runs it.

8.9 Threats to Validity

Nothing was tuned, and one run was taken per configuration. Each delta therefore carries the band of about ten problems established in Section 8.3, and the matrix is a one-at-a-time ablation, so interactions between components are unmeasured. The union of solves across all sixteen configurations is 905 problems against the baseline's 747, which bounds what a portfolio over them could reach and indicates how far from tuned the baseline is. No figure here is this prover's ceiling.

The countermodel verdict is inferred, not checked. `Branch.model_certain?/2` tests pairwise syntactic non-complementarity over a saturated branch, without enforcing congruence, using the two-element cardinality of o or consulting the unifier. Across the whole sweep 22 answers contradict the TPTP status, spanning five problems whose conjectures hold only because o has exactly two elements, and every one is a countermodel claimed on a theorem; no refutation in 81,744 rows contradicts the status. Countermodels are excluded from the scoring above for that reason, which also leaves the 61 agreeing countermodel answers unattested. One of the two defects behind them has since been corrected; the general one remains, since the verdict is still a syntactic test rather than a satisfiability check.

The prover does not always answer identically on repeated runs. Workers race, evidence reaches the agent in a nondeterministic order, and the budget cuts wherever it falls. The instability also persists at `worker_pool_size: 1`, which is presumably because the total term order of Section 5.7 uses `:erlang.phash2/1` on the nondeterministic Erlang references given as unique names to parameters.

The probe ran on a later tree and on other hardware. The counters of Section 8.6 and the estimate of run-to-run variation come from commit 4a9c4f3 with one instrumentation line added, while the matrix comes from 2a05f05. The two corrections that landed between them, to `Branch.distinguishable?/2` and to `Proof.interior_event/2`, touch neither the calculus nor the agent's scheduling, so the mechanism described in that section is unaffected, but the two sets of absolute counters should

not be combined, and the probe ran at sixteen schedulers against the matrix's eight.

The budget is cooperative, and one problem in six never reached the prover. A worker notices the deadline between steps, so a step entering a long pre-unification enumeration overruns it: forty-five solves across the sweep were recorded past 2000 ms, as late as 6 s, with `eager_unfold_defs` accounting for eleven. Those solves are counted here and would not be counted under a hard budget. The parse failures of Section 8.3 bound every corpus-level statement, since the solved counts are counts over what was attempted. Per-configuration wall clock is not comparable either, because the baseline ran first and populated the shared parse cache, so per-problem times are quoted and per-configuration totals are not. A further 57 results, about four per configuration, were recorded as prover errors because `ShotTx.Proof.interior_event/2` had no clause for `{:instantiate, ...}` trace entries, and are counted as unsolved.

The two studies were run under different conditions. The matrix ran at a 2000 ms budget on the full corpus with one run per configuration; the structured set ran at 5000 ms over three runs with a fresh runtime per problem. Solved counts from the two are not comparable, and the reference verdicts of Section 8.7 were obtained on other hardware through a public service.

Conclusion

9.1 Summary

This thesis has built SHOT, a prover for classical higher-order logic consisting of a free-variable tableau calculus over Church’s simple type theory [Chu40] with primitive equality, a concurrent architecture for it on the BEAM, three optional agents attached to a stream of closure evidence, and reconstruction of an ordinary tableau proof from the traces a concurrent search leaves behind. The implementation is four Hex packages, `shot_ds` for terms, types and the semantics of the type theory, `shot_un` for pre-unification, `shot_to` for the term order and `shot_tx` for the calculus, the architecture and proof objects, the first three usable without the prover. The thesis itself is the set of Livebook notebooks in which that behaviour is executed rather than paraphrased.

One property of free-variable tableaux organises the whole of it. A free variable is rigid, so a value that closes one branch is a commitment on every branch, and closure is therefore the single decision a branch cannot take on its own. Everything else a branch does is local. The design pursued here separates those two things as far as they can be separated: the calculus of Chapter 3 makes every branch-level rule binding-free, so that branch expansion never commits to a value and never has to withdraw one, and the architecture of Chapter 6 parallelises the local work while leaving one serialised decision over the accumulated evidence.

9.2 Design Decisions and Assumptions

The system is the result of a small number of decisions, each of which buys a property and assumes something in exchange. Collecting them is the more useful summary of what has been built.

A tableau calculus, and no clausification. The input formula reaches the calculus in the shape it was written in, decomposed by rules that mirror its connectives, so the derivation is legible and the failure of a search says which branch remained open. This is the property Chapter 1 argued for against the superposition systems that outperform tableau provers on the same benchmarks, and it assumes that structure preservation and a legible proof object are worth the coverage that clausification and saturation would buy. Rigidity is what the assumption costs, since it is the one global dependency a tableau has and a clausal system does not.

Binding-free rules, so that no branch commits. No rule of Chapter 3 applies a substitution. A rule that finds two literals which could be made complementary records the pair as closure evidence and continues, and the pre-unifiers computed to test the pair are discarded. Branch expansion is then independent work with no backtracking over instantiations, which is what makes it parallelisable at all. The discipline is assumed throughout the term-level chapters and constrains them:

rewriting admits only rules whose left-hand side matches with an empty matcher, the term order of Chapter 5 is required to be stable under a substitution that has not been computed, and a branch reporting itself open reports an under-approximation, since a pair it could not separate syntactically may still be separable.

Global closure as one constraint satisfaction problem. What the branches publish is a set of candidate closing pairs per branch, and closing the tableau means choosing one pair from each open branch such that the chosen pairs admit a common pre-unifier. Solving that once, in a dedicated process, gives a higher-order counterpart to incremental closure [Gie01] and removes instantiation backtracking from the search entirely. The assumption is that this single global step is small relative to the local work. It is the assumption the measurements contradict, and the one place where the design as built does not hold up.

A bounded worker pool over shared structures, rather than a process per branch. Branches are values held in a shared ordered table, and a fixed pool of long-lived workers takes one, applies a bounded number of rules to it and returns it. The alternative, adopted at first order by Goéland [CRD⁺22], is one process per branch, and it was rejected because branches are created faster than they are closed, each retains its full formula set, and each would publish evidence to the same single consumer. The pool assumes instead that a branch is cheap to hand between processes, which is why the data structures are what they are: terms are hash-consed into a global pool and referred to by identifier, intermediate terms are built in process-local scratchpads and committed only when a result survives, and the session's state is held in public tables addressed by branch prefix so that closure, tombstoning and hint lookup are single operations.

Evidence is published, not requested. A branch's only outbound message about closure is a broadcast of its candidate pairs, and the consumers subscribe. The contradiction agent $\mathcal{CA}$ that solves the global constraint is one subscriber, and once one exists a second costs close to nothing, which is what makes the suggestion agent $\mathcal{SA}$ and the model agent $\mathcal{MA}$ of Chapter 7 available at all. Two properties are assumed of every consumer: it cannot block the search, and it cannot be required for soundness. Both hold, so each agent is switchable and the question of whether an agent helps is a heuristic one rather than a question about the calculus. The direction of control is the difference from the concurrent higher-order systems named in Chapter 1, where agents advise a scheduler.

Boolean extensionality by decomposition, the functional principles by equality expansion. An o -typed subterm in an argument position is opaque to rules that act on heads, so renaming moves it onto the branch as an equation and instantiation discharges the parameter left behind by case analysis over a finite domain, which is the syntactic content of $\mathcal{D}_o$ having two elements. The functional principles are supplied by the rules that expand an equality at a functional type. The assumption there is narrower than it looks: those rules apply to a formula that already is an equality, and nothing turns a pair of rigid-headed atoms into a disequality goal, so the calculus reaches the functional principles only where the problem states them as equalities.

A complete propositional decision procedure inside the search loop. Every formula is passed through a reduced ordered binary decision diagram [Bry86] before a rule is chosen for it, which is not what provers in this family normally do. The justification is a cost assumption specific to higher-order search: γ and primitive

substitution exceed the propositional layer by orders of magnitude, so a procedure that keeps a formula away from them repays itself, and a tautology or a propositional contradiction is decided at cost zero. What the decision assumes in exchange is that a normalised propositional form is an acceptable substitute for the one the problem was written with, which sits awkwardly beside the first decision on this list and is taken up as future work.

Bounds with iterative deepening in place of a completeness proof. Pre-unification is semi-decidable and γ and primitive substitution do not terminate, so each is bounded and the bounds are raised together when the search saturates, with parked rules resumed where they left off rather than recomputed. Soundness is argued rule by rule; the fairness of the schedule is what stands in for a completeness argument, and Chapter 1 states that no Henkin completeness proof is given for the combination of demodulation, extensionality and the bounds.

Refutation is the answer; a countermodel is a side effect. The prover reports a countermodel when a branch saturates and its literals are pairwise non-complementary by a syntactic test. That test is never checked against a model, so a countermodel answer is unattested whether or not it is right, and the scoring throughout treats refutations as the result and countermodels as a separate uncertified count.

A monomorphic logic on a managed runtime. Rank-1 polymorphic input is monomorphised during parsing, so no reasoning about type variables happens inside the calculus. The BEAM was chosen for reduction-counted preemption, per-process heaps, per-core scheduling and concurrent table access, and for supervision, which lets a failing worker be restarted without the session losing its other branches. The exchange is a constant factor against implementations in OCaml and C, accepted in Chapter 1 on the argument that parallel search would compensate for it.

9.3 Measurement Results

The ablation study of Chapter 8 tests those decisions component by component, and two of its results bear on the design rather than on the numbers.

The first confirms the partition. $\mathcal{CA}$ accounts for roughly half of all refutations, and a prover deciding closure only locally answers a weaker question than the one posed. Global closure is therefore where the search happens, which is what the calculus predicts.

The second contradicts the assumption behind it. A pool of workers performs about three times the inference of a single worker at equal wall clock and does not increase the rate at which global closure is attempted, so the parallelism lands on the side of the partition that does not decide the verdict. The follow-up probe locates the reason in $\mathcal{CA}$ as implemented: it holds one search slot, a dispatched search occupies it for about half the budget, every trigger arriving meanwhile is discarded, and the search itself enumerates the cartesian product of the per-branch candidate lists, so a wider frontier makes a dispatch less likely and a dispatched search more expensive. $\mathcal{CA}$ is in its present form too weak for the architecture built around it: the global step is not small relative to the local work, and until it is given either a better method of solving for a global pre-unifier or concurrency of its own, additional workers cannot help. This is the taxonomy of [Bon00] applied to a higher-order tableau, and the deficiency lies in the global step rather than in

the calculus, since nothing about rigidity requires the constraint to be solved by one process enumerating combinations.

The qualitative study on the structured problems of [BB05] confirms the boundary the extensionality decision predicts. Grouped by the model class of [BBK04] in which each problem is valid, the classes needing only conversion or Boolean extensionality are largely decided and those needing one of the functional principles are largely not, which locates an absent rule rather than a general weakness.

9.4 Limitations

Nothing measured was tuned. Every configuration ran at the defaults of Chapter 6 with a uniform budget, one run each, and only two numeric parameters were swept at all. No portfolio, no per-problem strategy selection, no schedule of parameters and no tuning against the corpus took place, and the prover has never been run in a competition setting. Three observations bound how the counts should be read. Switching one non-default value, eager definition unfolding, improves on the baseline by more than any component in the study contributes; the union of problems solved across the sixteen configurations exceeds the baseline by a fifth; and both swept parameters were left at their defaults because the sweep gave no reason to move them, not because the space around them was explored. The absolute counts are therefore a lower bound on what the calculus reaches, and no figure in Chapter 8 should be read as this system’s ceiling or compared against a tuned prover’s.

The performance figures measure an untuned prototype on a managed runtime. Against the reference systems on the structured set the prover is about a factor of three slower at the median and two orders of magnitude at its extreme, and the parallel search does not recover the difference, for the reason above. The constant factor of the substrate and the deficiency in the global step are separate, and only the second is a defect of the design.

The remaining limitations follow from the decisions rather than from the measurement.

- Completeness is not established. Section 3.11 of Chapter 3 states as a conjecture what a complete extension would require and names the rule the calculus lacks, and the semantic layering of [BBK04] is used to state what is assumed rather than to prove what is achieved.
- The functional principles are reachable only from a formula that is already an equality, so a branch needing one of them to separate two atoms saturates.
- Transitivity of the term order is an open question and its soundness rests on a workshop paper [NM25b], so the order is used only for pairwise orientation decisions and never as a comparator over a set; the termination argument for demodulation inherits that restriction.
- Countermodel answers are inferred from a syntactic test and never checked, so those agreeing with the reference status are unattested as well.
- The choice operator is absent, and the fragment is narrower than that of [BB11].

- The logic is monomorphic, so a proof requiring an instance the monomorphisation does not produce is out of reach.
- Corpus coverage is bounded by the parser rather than by the calculus: about one problem in six was never read.
- Concurrency is confined to a single BEAM node.
- $\mathcal{MA}$ was inert in every configuration of the study, so its cost and its contribution are unmeasured.
- Verdicts are not deterministic. Workers race, evidence reaches $\mathcal{CA}$ in a non-deterministic order, and a small proportion of problems answer differently across identical repeats.

9.5 Future Work

The extensionality boundary comes first, since it is the one gap located in the calculus rather than in the search, and Section 3.11 of Chapter 3 states the rule that would close it. Finite α -quantification shows the shape such a rule would take: it decides the examples inside its scope outright and leaves the rest of the class untouched. The rule to try admits the arguments of a disagreeing atom pair as an extensionality goal rather than only as a closure candidate, and the part needing care is its interaction with the binding-free discipline, since such a goal introduces a formula whose provability depends on a value the branch may not choose.

The global closure search comes second, since it is the one decision the measurements overturn. Three changes follow from the probe counters and are stated in Section 8.6: a per-search deadline set as a fraction of the remaining budget rather than the session deadline, retention of the most recent suppressed trigger so that it runs when the slot frees, and a bounded number of searches in flight over disjoint subsets of the open branches.

Those redistribute work the agent already performs. The larger question is how the global pre-unifier is computed. The present enumeration shares nothing between two combinations that agree on all but one candidate pair, retains no partial solution, and rejects a combination only after a unification call has failed on it. Two directions are open and they are compatible. The first keeps the search serial and makes it incremental in the sense of [Gie01]: retain the partial solutions of the combinations already tried, propagate the constraint each newly published pair imposes on the candidates of the other branches, and order the candidate lists so that a combination extending no surviving partial solution is discarded without a unification call. The second applies concurrency inside pre-unification itself, whose search branches at every flex-rigid pair into independent subtrees, so a single combination can be attempted across several schedulers. Either direction, or a different one that raises the rate at which closure is attempted, is required before a wider pool is worth its inference.

The third direction concerns the signature. Σ^{HOL} of Chapter 2 carries $\neg$ at $\alpha \rightarrow \alpha$ and four of the sixteen Boolean functions at $(\alpha, \alpha) \rightarrow \alpha$, so a problem written with any of the remaining ones reaches the calculus as a composition of those five. Since a tableau performs neither clausification nor normalisation, the signature should be maximal at both types: all four functions at $\alpha \rightarrow \alpha$ and all sixteen at $(\alpha, \alpha) \rightarrow \alpha$ as

constants in their own right, each with its own rule and its own case split. Exclusive disjunction is the clearest instance. Written as $(\varphi \wedge \neg\psi) \vee (\neg\varphi \wedge \psi)$ it reaches its two branches in three rule applications through an intermediate formula in which both arguments occur twice; as a constant of its own it is one β -rule. The interpretation conditions of Chapter 2 extend to each added constant by its truth table, so the semantics is unchanged, the general bindings of primitive substitution [And89] draw from a larger head family at the same depth, and the proof object presents the connective the problem was written with.

The preprocessing of Chapter 3 has to be adapted for that, and as it stands works against it. The diagram readback emits $\neg$, $\wedge$ and $\vee$ only, under the six short-circuit identities of Section 3.9, so a formula written with $\equiv$ or with any function outside those three is returned in a normal form over them and the connective is lost where the oracle rewrites the formula. The two extreme verdicts are unaffected, since $\top$ and $\perp$ are canonical whatever the readback emits; everything in between, which is the majority of what the oracle sees, is normalised more heavily than the calculus requires. A readback over a maximal signature would recognise the node patterns of the whole family, so that the diagram $\langle v, \langle w, \perp, \top \rangle, \langle w, \top, \perp \rangle \rangle$ is returned as an exclusive disjunction of v and w rather than as its expansion, and canonicity is kept without discarding structure. Which of the sixteen functions are worth recognising, and whether recognition should be attempted below the root, are open questions.

The fourth direction concerns the choice operator, which was excluded in Chapter 1 and should be investigated for this calculus. Backes and Brown [BB11] give analytic tableaux for higher-order logic with choice over tableau objects of the same kind used here. What this calculus adds to that setting is the deferred substitution, so a choice rule would have to record the condition under which its witness exists as evidence, in the way the renaming and instantiation rules record theirs, rather than committing to a witness on the branch that introduced it.

The last direction reads the tableau as a clausification. A branch's literal set is a conjunction of literals, and the open branches of a tableau are a disjunction of those conjunctions, so the analysis a tableau carries out on its input delivers something close to a conjunctive normal form without a clausification step. A resolution or superposition calculus can therefore be run on the literal set of a single branch, where the term order and the equations of Chapter 3 are already available. The condition such a calculus must meet is the one rigidity imposes: an inference has to return a pre-unifier in the sense of Chapter 4, which the branch publishes as evidence, rather than apply a substitution to the branch's terms. Superposition with lambdas [BBT⁺21] and the treatment of Boolean subterms in [NBTV21] state what the higher-order side of such a calculus requires; what is open here is which of its inferences survive the restriction to rewrite rules with an empty matcher, and whether a branch-local saturation repays its cost on a branch the global search may close at any moment.

Several smaller items were named where they arose. Dual instantiation is applied to instantiation candidates in Chapter 3 and not to definitions, which is where TPS applies it [BA98]: a branch could retain a defined atom alongside its expansion instead of choosing between the two, and the choice could be made per definition rather than per session, which is the third `unfold_defs` mode the eager row of Chapter 8 argues for. The evidence stream admits further consumers at negligible cost, and a lemma agent over the published clash candidates is the nearest one.

The runtime's distribution facilities are unused, and the closure search of the first direction above is the component whose cost would justify a second node. The study itself should be repeated with $\mathcal{MA}$ enabled, with repeats per configuration, and with the parameters tuned rather than left at their defaults, which is the one change that would make its absolute counts comparable with a published system's.

Bibliography

- [ABB00] Peter B. Andrews, Matthew Bishop, and Chad E. Brown. System description: TPS: A theorem proving system for type theory. In *Automated Deduction (CADE-17)*, volume 1831 of *Lecture Notes in Computer Science*, pages 164–169. Springer, 2000.
- [And81] Peter B. Andrews. Theorem proving via general matings. *Journal of the ACM*, 28(2):193–214, 1981.
- [And89] Peter B. Andrews. On connections and higher-order logic. *Journal of Automated Reasoning*, 5(3):257–291, 1989.
- [And02] Peter B. Andrews. *An Introduction to Mathematical Logic and Type Theory: To Truth Through Proof*, volume 27 of *Applied Logic Series*. Kluwer Academic Publishers, 2 edition, 2002.
- [Arm03] Joe Armstrong. *Making Reliable Distributed Systems in the Presence of Software Errors*. PhD thesis, Royal Institute of Technology (KTH), Stockholm, 2003.
- [BA98] Matthew Bishop and Peter B. Andrews. Selectively instantiating definitions. In *Automated Deduction (CADE-15)*, volume 1421 of *Lecture Notes in Computer Science*, pages 365–380. Springer, 1998.
- [BB05] Christoph Benzmüller and Chad E. Brown. A structured set of higher-order problems. In *Theorem Proving in Higher Order Logics (TPHOLs 2005)*, volume 3603 of *Lecture Notes in Computer Science*, pages 66–81. Springer, 2005.
- [BB11] Julian Backes and Chad E. Brown. Analytic tableaux for higher-order logic with choice. *Journal of Automated Reasoning*, 47(4):451–479, 2011.
- [BBK04] Christoph Benzmüller, Chad E. Brown, and Michael Kohlhase. Higher-order semantics and extensionality. *The Journal of Symbolic Logic*, 69(4):1027–1088, 2004.
- [BBT⁺21] Alexander Bentkamp, Jasmin Blanchette, Sophie Tourret, Petar Vukmirović, and Uwe Waldmann. Superposition with lambdas. *Journal of Automated Reasoning*, 65(7):893–940, 2021.
- [Ben99a] Christoph Benzmüller. *Equality and Extensionality in Automated Higher Order Theorem Proving*. PhD thesis, Universität des Saarlandes, Saarbrücken, 1999.
- [Ben99b] Christoph Benzmüller. Extensional higher-order paramodulation and RUE-resolution. In *Automated Deduction (CADE-16)*, volume 1632 of *Lecture Notes in Computer Science*, pages 399–413. Springer, 1999.
- [Ben02] Christoph Benzmüller. Comparing approaches to resolution based higher-order theorem proving. *Synthese*, 133(1–2):203–335, 2002.

- [BG94] Leo Bachmair and Harald Ganzinger. Rewrite-based equational theorem proving with selection and simplification. *Journal of Logic and Computation*, 4(3):217–247, 1994.
- [BJR15] Frédéric Blanqui, Jean-Pierre Jouannaud, and Albert Rubio. The computability path ordering. *Logical Methods in Computer Science*, 11(4:3), 2015.
- [BK98] Christoph Benzmüller and Michael Kohlhase. Extensional higher-order resolution. In *Automated Deduction (CADE-15)*, volume 1421 of *Lecture Notes in Computer Science*, pages 56–71. Springer, 1998.
- [BK22] Chad E. Brown and Cezary Kaliszyk. Lash 1.0 (system description). In *Automated Reasoning (IJCAR 2022)*, volume 13385 of *Lecture Notes in Computer Science*, pages 350–358. Springer, 2022.
- [BM14] Christoph Benzmüller and Dale Miller. Automation of higher-order logic. In *Computational Logic*, volume 9 of *Handbook of the History of Logic*, pages 215–254. Elsevier, 2014.
- [BN98] Franz Baader and Tobias Nipkow. *Term Rewriting and All That*. Cambridge University Press, 1998.
- [BN10] Jasmin Christian Blanchette and Tobias Nipkow. Nitpick: A counterexample generator for higher-order logic based on a relational model finder. In *Interactive Theorem Proving (ITP 2010)*, volume 6172 of *Lecture Notes in Computer Science*, pages 131–146. Springer, 2010.
- [Bon00] Maria Paola Bonacina. A taxonomy of parallel strategies for deduction. *Annals of Mathematics and Artificial Intelligence*, 29(1):223–257, 2000.
- [BPTF08] Christoph Benzmüller, Lawrence C. Paulson, Frank Theiss, and Arnaud Fietzke. LEO-II: A cooperative automatic theorem prover for classical higher-order logic (system description). In *Automated Reasoning (IJCAR 2008)*, volume 5195 of *Lecture Notes in Computer Science*, pages 162–170. Springer, 2008.
- [Bro12] Chad E. Brown. Satallax: An automatic higher-order prover. In *Automated Reasoning (IJCAR 2012)*, volume 7364 of *Lecture Notes in Computer Science*, pages 111–117. Springer, 2012.
- [Bry86] Randal E. Bryant. Graph-based algorithms for Boolean function manipulation. *IEEE Transactions on Computers*, C-35(8):677–691, 1986.
- [BS98] Christoph Benzmüller and Volker Sorge. A blackboard architecture for guiding interactive proofs. In *Artificial Intelligence: Methodology, Systems, and Applications (AIMSA 1998)*, volume 1480 of *Lecture Notes in Computer Science*, pages 102–114. Springer, 1998.
- [BSJK05] Christoph Benzmüller, Volker Sorge, Mateja Jamnik, and Manfred Kerber. Can a higher-order and a first-order theorem prover cooperate? In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR 2004)*, volume 3452 of *Lecture Notes in Computer Science*, pages 415–431. Springer, 2005.
- [BSJK08] Christoph Benzmüller, Volker Sorge, Mateja Jamnik, and Manfred Kerber. Combined reasoning by automated cooperation. *Journal of Applied Logic*, 6(3):318–342, 2008.

- [BSPT15] Christoph Benzmüller, Nik Sultana, Lawrence C. Paulson, and Frank Theiss. The higher-order prover Leo-II. *Journal of Automated Reasoning*, 55(4):389–404, 2015.
- [BSW17] Christoph Benzmüller, Alexander Steen, and Max Wisniewski. Leo-III version 1.1 (system description). In *LPAR-21 Workshops: Empirically Successful Automated Reasoning in Higher-Order Logic (IWIL)*, volume 1 of *Kalpa Publications in Computing*, pages 11–26. EasyChair, 2017.
- [Chu40] Alonzo Church. A formulation of the simple theory of types. *The Journal of Symbolic Logic*, 5(2):56–68, 1940.
- [CRD⁺22] Julie Cailler, Johann Rosain, David Delahaye, Simon Robillard, and Hinde Lilia Bouziane. Goéland: A concurrent tableau-based theorem prover (system description). In *Automated Reasoning (IJCAR 2022)*, volume 13385 of *Lecture Notes in Computer Science*, pages 359–368. Springer, 2022.
- [dB72] Nicolaas G. de Bruijn. Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the church-rosser theorem. *Indagationes Mathematicae*, 75(5):381–392, 1972.
- [Der82] Nachum Dershowitz. Orderings for term-rewriting systems. *Theoretical Computer Science*, 17(3):279–301, 1982.
- [Dow01] Gilles Dowek. Higher-order unification and matching. In *Handbook of Automated Reasoning*, volume 2, chapter 16, pages 1009–1062. Elsevier and MIT Press, 2001.
- [FC06] Jean-Christophe Filliâtre and Sylvain Conchon. Type-safe modular hash-consing. In *Proceedings of the 2006 Workshop on ML*, pages 12–19. ACM, 2006.
- [Fit96] Melvin Fitting. *First-Order Logic and Automated Theorem Proving*. Graduate Texts in Computer Science. Springer, 2 edition, 1996.
- [Gie01] Martin Giese. Incremental closure of free variable tableaux. In *Automated Reasoning (IJCAR 2001)*, volume 2083 of *Lecture Notes in Computer Science*, pages 545–560. Springer, 2001.
- [Gol81] Warren D. Goldfarb. The undecidability of the second-order unification problem. *Theoretical Computer Science*, 13(2):225–230, 1981.
- [Häh01] Reiner Hähnle. Tableaux and related methods. In *Handbook of Automated Reasoning*, volume 1, chapter 3, pages 100–178. Elsevier and MIT Press, 2001.
- [Hen50] Leon Henkin. Completeness in the theory of types. *The Journal of Symbolic Logic*, 15(2):81–91, 1950.
- [HS94] Reiner Hähnle and Peter H. Schmitt. The liberalized δ -rule in free variable semantic tableaux. *Journal of Automated Reasoning*, 13(2):211–221, 1994.
- [Hue75] Gérard P. Huet. A unification algorithm for typed λ -calculus. *Theoretical Computer Science*, 1(1):27–57, 1975.
- [HW03] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.

- [Joh85] Thomas Johnsson. Lambda lifting: Transforming programs to recursive equations. In *Functional Programming Languages and Computer Architecture (FPCA 1985)*, volume 201 of *Lecture Notes in Computer Science*, pages 190–203. Springer, 1985.
- [Knu84] Donald E. Knuth. Literate programming. *The Computer Journal*, 27(2):97–111, 1984.
- [Koh95] Michael Kohlhase. Higher-order tableaux. In *Theorem Proving with Analytic Tableaux and Related Methods (TABLEAUX 1995)*, volume 918 of *Lecture Notes in Computer Science*, pages 294–309. Springer, 1995.
- [Kon98] Karsten Konrad. HOT: A concurrent automated theorem prover based on higher-order tableaux. In *Theorem Proving in Higher Order Logics (TPHOLs 1998)*, volume 1479 of *Lecture Notes in Computer Science*, pages 245–261. Springer, 1998.
- [KSR16] Cezary Kaliszyk, Geoff Sutcliffe, and Florian Rabe. TH1: The TPTP typed higher-order form with rank-1 polymorphism. In *Proceedings of the 5th Workshop on Practical Aspects of Automated Reasoning (PAAR 2016)*, volume 1635 of *CEUR Workshop Proceedings*, pages 41–55, 2016.
- [Liv25] Livebook: Automate code and data workflows with interactive elixir notebooks. The Livebook Team, <https://livebook.dev>, 2025. Accessed 2026-08-15.
- [Mil87] Dale A. Miller. A compact representation of proofs. *Studia Logica*, 46(4):347–370, 1987.
- [Mil91] Dale Miller. A logic programming language with lambda-abstraction, function variables, and simple unification. *Journal of Logic and Computation*, 1(4):497–536, 1991.
- [NBTv21] Visa Nummelin, Alexander Bentkamp, Sophie Tourret, and Petar Vukmirović. Superposition with first-class booleans and inprocessing clausification. In *Automated Deduction (CADE-28)*, volume 12699 of *Lecture Notes in Computer Science*, pages 378–395. Springer, 2021.
- [NM25a] Johannes Niederhauser and Aart Middeldorp. The computability path order for beta-eta-normal higher-order rewriting. In *Automated Deduction (CADE-30)*, volume 15943 of *Lecture Notes in Computer Science*, pages 207–225. Springer, 2025.
- [NM25b] Johannes Niederhauser and Aart Middeldorp. NCPO goes beta-eta-long normal form. In *Proceedings of the 20th International Workshop on Termination (WST 2025)*, Leipzig, Germany, 2025. Informal proceedings. https://www.imn.htwk-leipzig.de/~waldmann/WST2025/proceedings/WST2025_paper_6.pdf.
- [NM26] Johannes Niederhauser and Aart Middeldorp. Unification of deterministic higher-order patterns. In *Automated Reasoning (IJCAR 2026)*, Lecture Notes in Computer Science. Springer, 2026. To appear. Preprint <https://arxiv.org/abs/2601.14211>.
- [SB10] Geoff Sutcliffe and Christoph Benzmüller. Automated reasoning in higher-order logic using the TPTP THF infrastructure. *Journal of Formalized Reasoning*, 3(1):1–27, 2010.

- [SB21] Alexander Steen and Christoph Benzmüller. Extensional higher-order paramodulation in Leo-III. *Journal of Automated Reasoning*, 65(6):775–807, 2021.
- [Smu68] Raymond M. Smullyan. *First-Order Logic*, volume 43 of *Ergebnisse der Mathematik und ihrer Grenzgebiete*. Springer, 1968.
- [SO25] 2025 developer survey: Technology. Stack Overflow, <https://survey.stackoverflow.co/2025/technology>, 2025. Accessed 2026-08-15.
- [Sor01] Volker Sorge. *Ω -ANTS: A Blackboard Architecture for the Integration of Reasoning Techniques into Proof Planning*. PhD thesis, Universität des Saarlandes, Saarbrücken, 2001.
- [Sti09] Colin Stirling. Decidability of higher-order matching. *Logical Methods in Computer Science*, 5(3), 2009.
- [Sut08] Geoff Sutcliffe. The SZS ontologies for automated reasoning software. In *LPAR Workshops: Knowledge Exchange, Automated Provers and Proof Assistants (KEAPPA) and Empirically Successful Automated Reasoning in Higher-Order Logic (IWIL)*, volume 418 of *CEUR Workshop Proceedings*, pages 38–49, 2008.
- [Sut17] Geoff Sutcliffe. The TPTP problem library and associated infrastructure: From CNF to TH0, TPTP v6.4.0. *Journal of Automated Reasoning*, 59(4):483–502, 2017.
- [Sut26] Geoff Sutcliffe. The 13th IJCAR automated theorem proving system competition: CASC-J13. <https://tptp.org/CASC/J13>, 2026. Accessed 2026-08-15.
- [SWB16] Alexander Steen, Max Wisniewski, and Christoph Benzmüller. Agent-based HOL reasoning. In *Mathematical Software (ICMS 2016)*, volume 9725 of *Lecture Notes in Computer Science*, pages 75–81. Springer, 2016.
- [VBB⁺21] Petar Vukmirović, Alexander Bentkamp, Jasmin Blanchette, Simon Cruanes, Visa Nummelin, and Sophie Turret. Making higher-order superposition work. In *Automated Deduction (CADE-28)*, volume 12699 of *Lecture Notes in Computer Science*, pages 415–432. Springer, 2021.
- [WB16] Max Wisniewski and Christoph Benzmüller. Is it reasonable to employ agents in automated theorem proving? In *Agents and Artificial Intelligence (ICAART 2016)*, volume 1, pages 281–286. SCITEPRESS, 2016.
- [WSB15] Max Wisniewski, Alexander Steen, and Christoph Benzmüller. LeoPARD: A generic platform for the implementation of higher-order reasoners. In *Intelligent Computer Mathematics (CICM 2015)*, volume 9150 of *Lecture Notes in Computer Science*, pages 325–330. Springer, 2015.

Examples

A.1 Overview

The problems collected here are drawn from the structured set of Benz Müller and Brown [BB05]. That set is meant to be applied to a calculus before its soundness and completeness are analysed formally: the problems are individually small, and each isolates one technical issue, so a system that fails one has a located defect rather than a general weakness.

What makes the set worth using in place of a sample of TPTP is that its problems are graded along two dimensions, and each problem is annotated with the weakest assumptions under which it holds. The first dimension is extensionality. Four properties are involved: η -conversion, the congruence rule ξ identifying two abstractions with identical bodies, functional extensionality $\mathfrak{f}$ identifying two functions that agree on every argument, and Boolean extensionality $\mathfrak{b}$ identifying two formulae with the same truth value. Varying them over the semantics of [BBK04], which Chapter 2 builds on, yields eight classes of models, from $\mathfrak{M}_\beta$ where only β -equal terms are identified up to $\mathfrak{M}_{\beta\mathfrak{f}\mathfrak{b}}$, which is Henkin semantics. Every example is labelled with the largest class in which it is valid, and validity in a larger class implies validity in every smaller one. A problem valid in $\mathfrak{M}_\beta$ therefore tests no extensionality principle at all, and one valid only in $\mathfrak{M}_{\beta\mathfrak{f}\mathfrak{b}}$ tests all of them.

The second dimension is set comprehension. Which sets exist in a model depends on which logical constants the signature supplies, and instantiating a set variable during proof search is what the primitive substitution of Section 3.4 does. Examples on that dimension are annotated with the signature they need rather than with a model class.

The two dimensions are what turn a run of this appendix into a statement about the calculus. SHOT's rules commit it to particular principles: the equality rules give substitutivity, the renaming and instantiation rules of Section 3.5 give $\mathfrak{b}$, the η -long β -normal representation of Chapter 2 gives η without any rule firing, and the equality-expansion rules give the functional principles. Each section below names the example, the class it belongs to, and which of those commitments it is testing.

Where a problem carries free constants, the ambient type environment is supplied explicitly rather than left to inference. Where an example is meant to exercise the rule loop rather than the preprocessing of Section 3.9, simplification is disabled for that call, since the BDD would otherwise decide the propositional examples before any tableau rule fires.

A.2 The Propositional and Quantifier Rules

The first three problems fix the shape of an entry and exercise the rules that carry no higher-order content. $p \supset p$ closes by one α -expansion and a ground clash, and

runs with simplification disabled so that the rule loop performs the closure rather than the BDD.

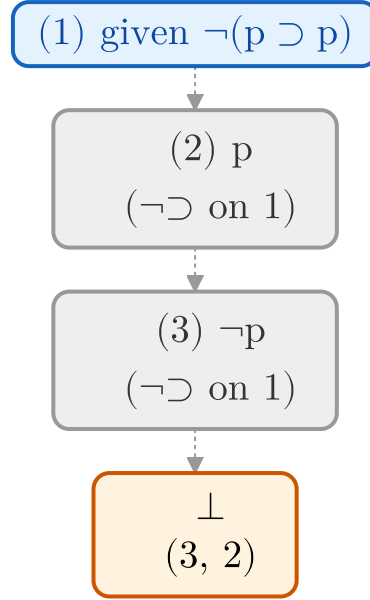


Figure A.1: THM. The proof of $p \supset p$.

Distribution of a universal quantifier over a conjunction exercises γ on the assumption side and β on the conclusion side. Each of the two conclusion branches closes under its own instance, so the global pre-unifier of Chapter 6 has two constraints to satisfy together, and the γ -rule fires twice because it is reinserted rather than consumed.

$$(\forall l. p(1) \wedge q(1)) \supset ((\forall l. p(1)) \wedge (\forall l. q(1)))$$

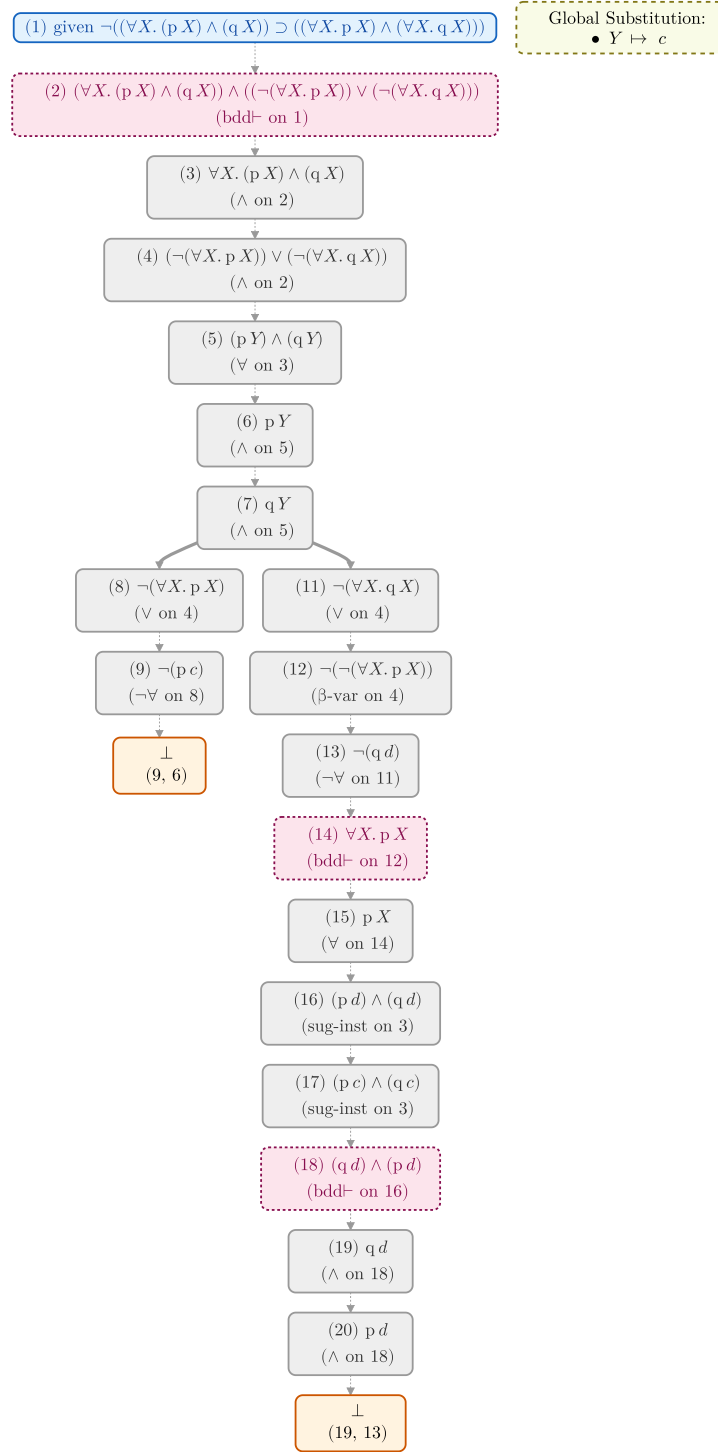


Figure A.2: THM. The proof of the distribution of a universal quantifier over a conjunction. The γ -rule fires twice, once for each conclusion branch.

The δ -rule applies where an existential is asserted or a universal denied. Here the negated conclusion introduces a parameter and the assumption's γ -variable is bound to it.

$$(\forall l. p(1)) \supset (\exists l. p(1))$$

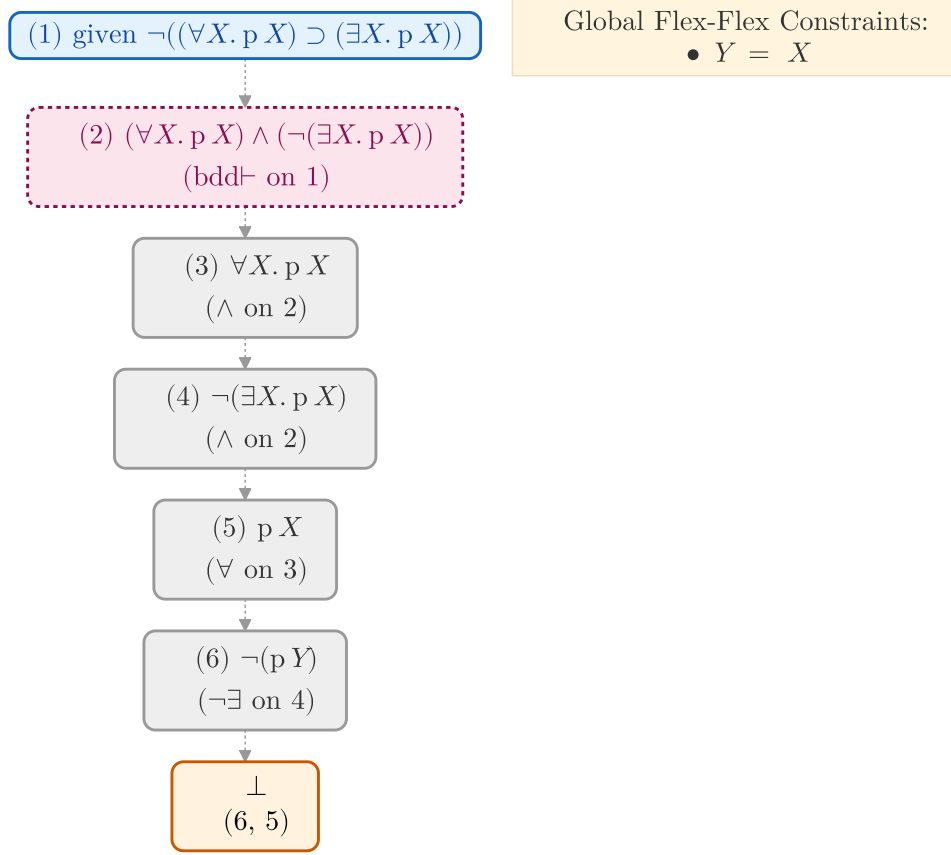


Figure A.3: THM. The proof of $(\forall l. p(1)) \supset (\exists l. p(1))$: the negated conclusion introduces the parameter to which the assumption's Y -variable is bound.

A.3 Equality

Example 6 asks for reflexivity, symmetry and transitivity of equality, and Example 7 for its congruence properties. Both are valid in $\mathfrak{M}_\beta$, so neither tests extensionality; they establish that equality reasoning works before the later examples load anything onto it. Each is stated once for primitive equality and once for each definitional encoding, so that a calculus which handles one and not the others is identified. The primitive versions are decided by the equality rules of Section 3.6 without any instantiation.

$$\forall l. 1 = 1 \quad \forall l, l. 2 = 1 \supset 1 = 2 \quad \forall l, l, l. (3 = 2 \wedge 2 = 1) \supset 3 = 1$$

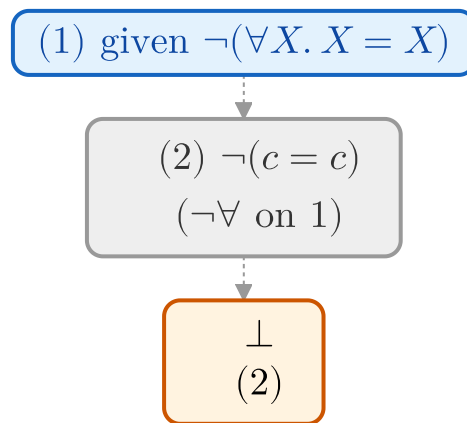


Figure A.4: THM. The proof of reflexivity.

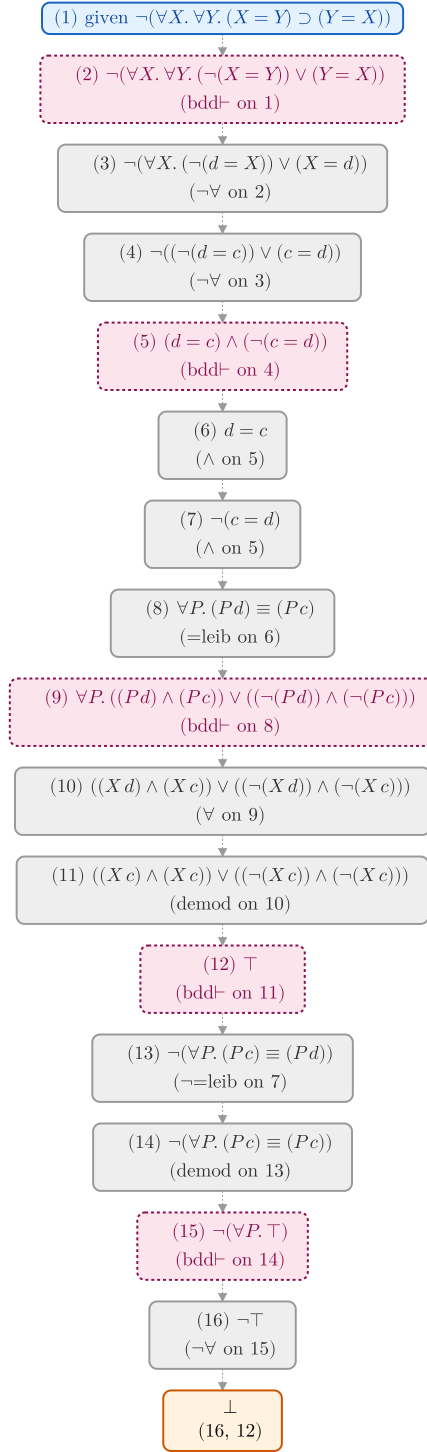


Figure A.5: THM. The proof of symmetry.

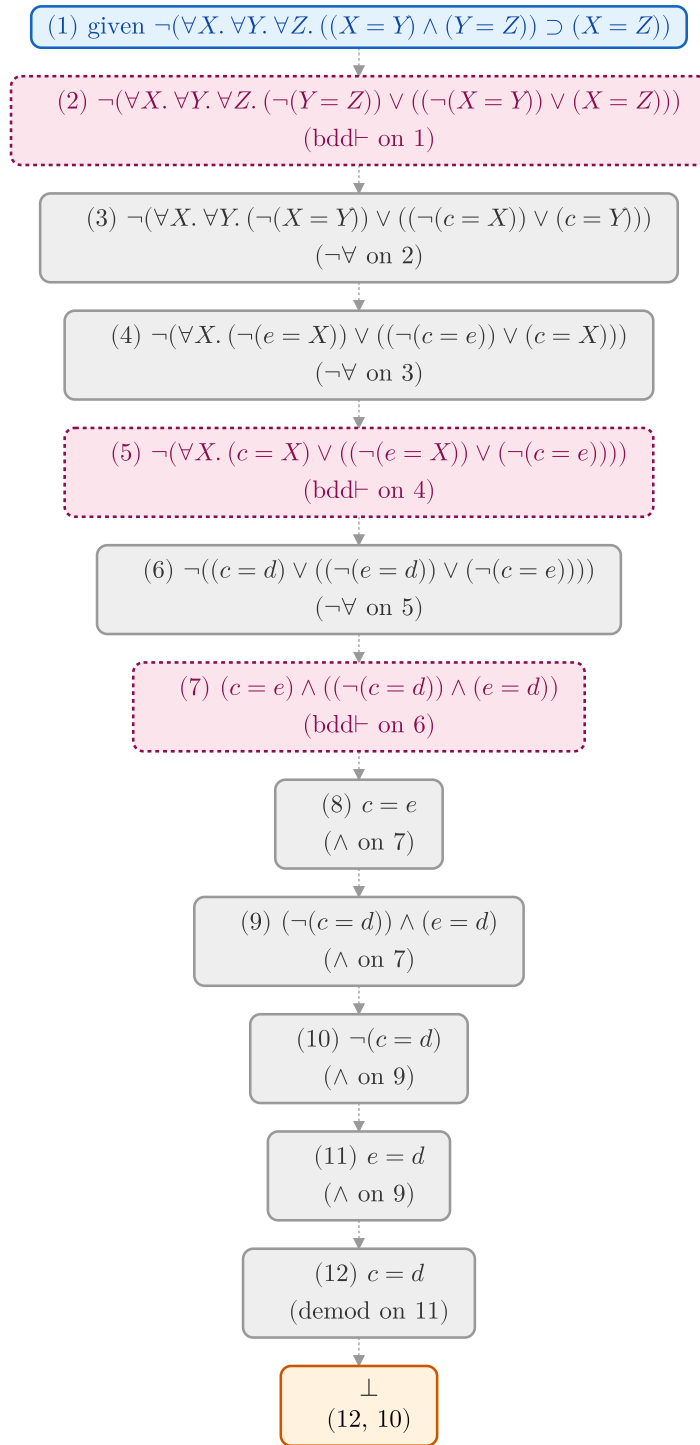


Figure A.6: THM. The proof of transitivity.

The encoded variants replace $=$ by a defined relation, which turns a problem about the equality rules into one about γ over a predicate variable. **HoLSuite** carries six variants of each part: primitive equality, the three Leibniz encodings obtained by reading the definition with $\supset$, $\subset$ and $\equiv$, the Andrews encoding, and the extensional encoding. Transitivity is where the encodings separate, and it is also the problem on

which the suggestion agent of Chapter 7 performs the most work.

A.4 Leibniz and Primitive Equality

Example 8 asks whether Leibniz equality implies primitive equality, and it is the one problem in the set whose status as a theorem is argued rather than assumed. It holds in every model satisfying property $\mathfrak{q}$, which is what makes Leibniz equality a definition of equality at all, and Henkin’s original models do not satisfy it [BB05]. Chapter 2 takes the corresponding position by interpreting equality as identity and making this direction a Proposition only in separating frames, and the tableau rule that recovers it Skolemises the separating property. The problem is a direct test of that rule, and SHOT does not close it within the bounds reached under iterative deepening.

$$(\forall l \rightarrow o. 1(a) \equiv 1(b)) \supset a = b$$

Timeout

The verdict is Timeout and not :unknown. The last section of this appendix takes up that distinction.

A.5 Extensionality: The Trivial Directions

Example 9 is the pair of directions of functional and Boolean extensionality that are valid already in $\mathfrak{M}_\beta$, the largest class. They follow from substitutivity and need no extensionality rule, which is what makes them useful: a system that fails here has an equality-reasoning defect, and a failure on Example 10 or 11 can be attributed to the extensionality rules only once Example 9 passes.

$$(\lambda l. f(1)) = (\lambda l. g(1)) \supset \forall l. f(1) = g(1)$$

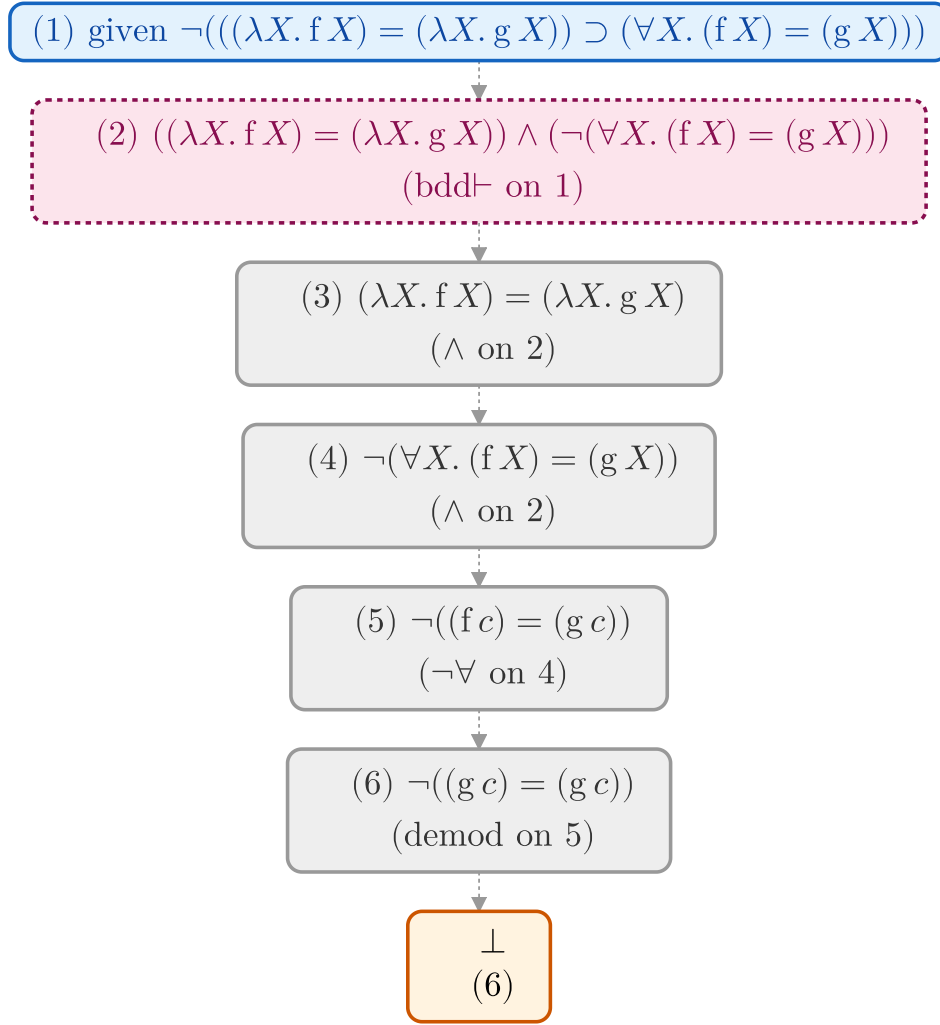


Figure A.7: THM. The proof of Example 9, the direction of functional extensionality that follows from substitutivity alone.

A.6 Boolean Extensionality: The Non-Trivial Direction

Example 10 is the direction a calculus without a Boolean extensionality rule cannot obtain. It is valid in $\mathfrak{M}_{\beta\mathfrak{b}}$ and in no larger class, so it is the sharpest single test of property $\mathfrak{b}$: two equivalent formulae must be equal at type o , which is a statement about $\mathcal{D}_o$ having two elements and does not follow from congruence.

$$\forall o, o. (2 \equiv 1) \supset 2 = 1$$

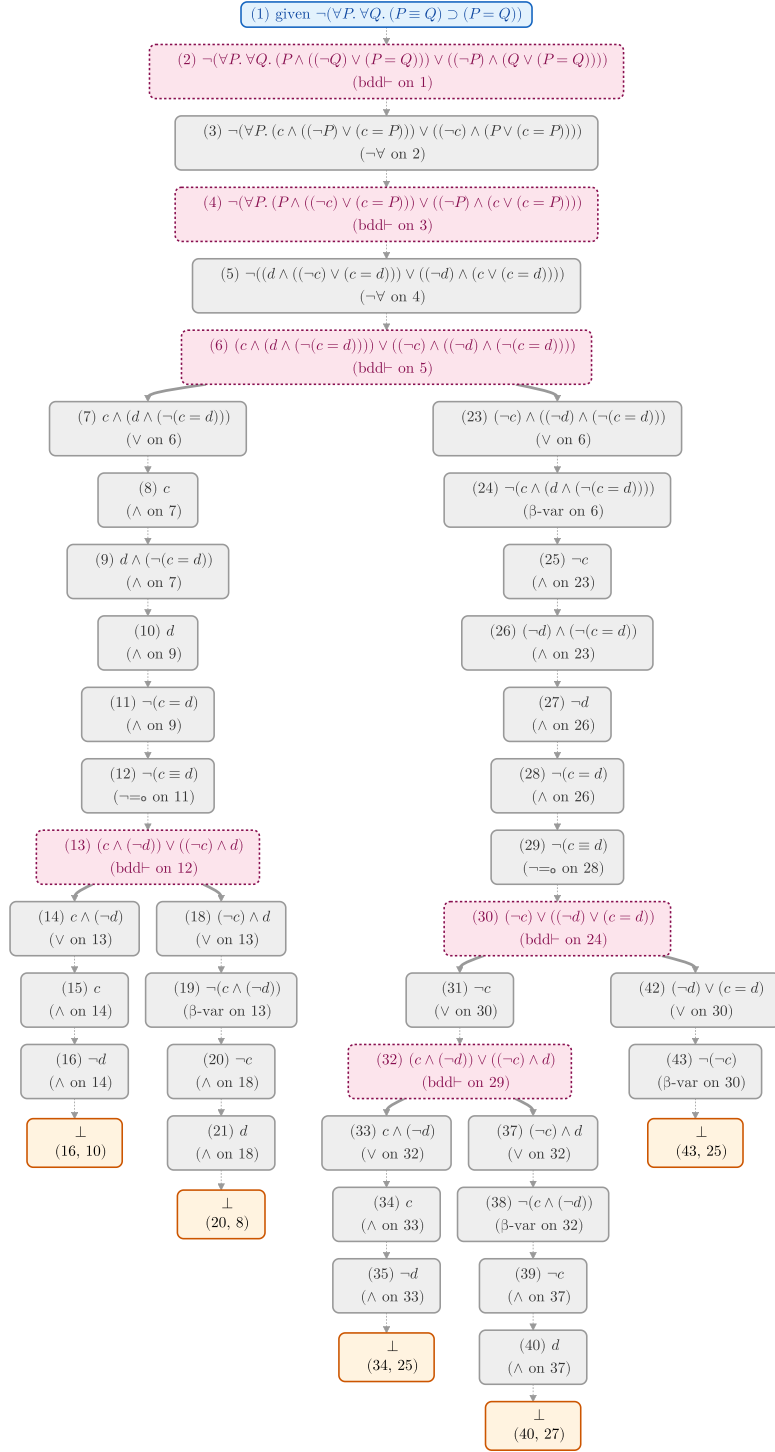


Figure A.8: THM. The proof of Example 10, the direction of Boolean extensionality that no calculus without a rule for it can obtain.

Examples 16 and 17 are also $\mathfrak{M}_{\beta\delta}$, and both place a Boolean argument where no connective governs it, which is the configuration the renaming and instantiation rules of Section 3.5 exist to reach. In the first, $a \wedge b$ occurs as an argument of p and the rules case-split on it; in the second, a formula is compared with its own negation.

$$(p(a) \wedge p(b)) \supset p(a \wedge b) \qquad \neg(a = \neg a)$$

Status: THM

Status: THM

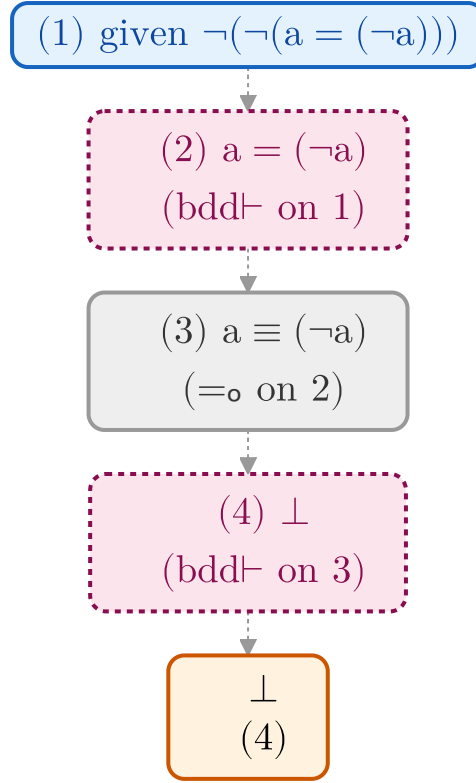
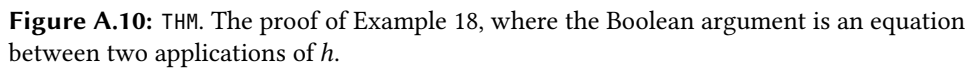


Figure A.9: The proof of Example 17, $\neg(a = \neg a)$.

Example 18 nests the Boolean argument one level further: the argument of the outer h is an equation between two applications of h . It is again $\mathfrak{M}_{\beta\delta}$, and the argument for it is a case analysis on that equation. If $h(\top)$ and $h(\perp)$ are equal the argument reduces to the assumption, and if they are not it reduces to an instance of reflexivity. The example runs with simplification disabled, since the BDD reduces it before the rule loop is reached and the rule loop is the object of the example.

$$h(h(\top) = h(\perp)) = h(\perp)$$



A.7 De Morgan by Degrees

Example 20 states one fact, De Morgan’s law, at four strengths, and each strength is valid in a different class. It is the clearest illustration in the set of what the first

dimension measures. Reading the two sides as equivalent needs nothing beyond β . Asserting them equal at type o needs $\mathfrak{b}$. Abstracting both sides and asserting the abstractions equal needs ξ as well. Replacing the left abstraction by the connective itself needs full functional extensionality, which is Henkin semantics.

$$\forall o, o. (2 \wedge 1) \equiv \neg(\neg 2 \vee \neg 1)$$

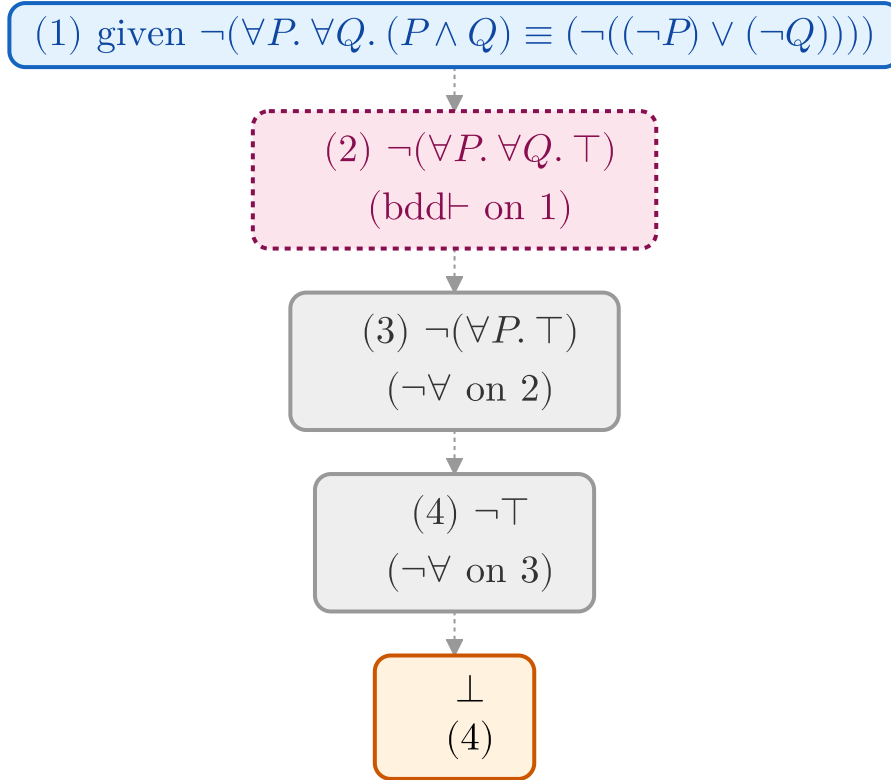


Figure A.11: THM. The proof of De Morgan's law at the first strength, read as an equivalence.

The same statement with primitive equality in place of the equivalence is valid in $\mathfrak{M}_{\beta\mathfrak{b}}$ and no larger class.

$$\forall o, o. (2 \wedge 1) = \neg(\neg 2 \vee \neg 1)$$

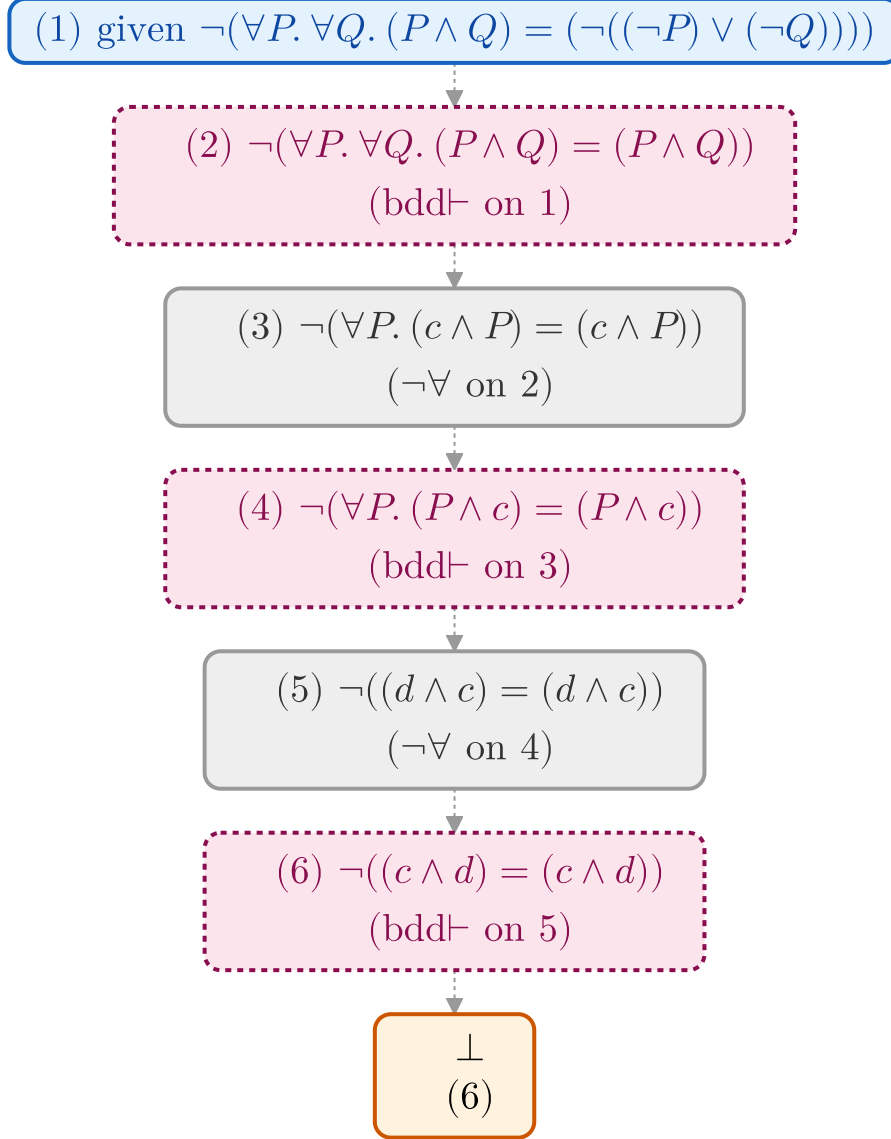


Figure A.12: THM. The proof of De Morgan's law at the second strength, asserted as an equality at type o .

Moving the binders outward turns the statement into an equation between two abstractions, valid in $\mathfrak{M}_{\beta\xi\mathfrak{b}}$. This is where the equality-expansion rules of Section 3.6 have to descend under a binder before the Boolean rules can act. Note that deep simplification would close this instantly as de Morgan rules are solved by the BDD normalisation of Section 3.9.

$$(\lambda o. o. 2 \wedge 1) = (\lambda o. o. \neg(\neg 2 \vee \neg 1))$$

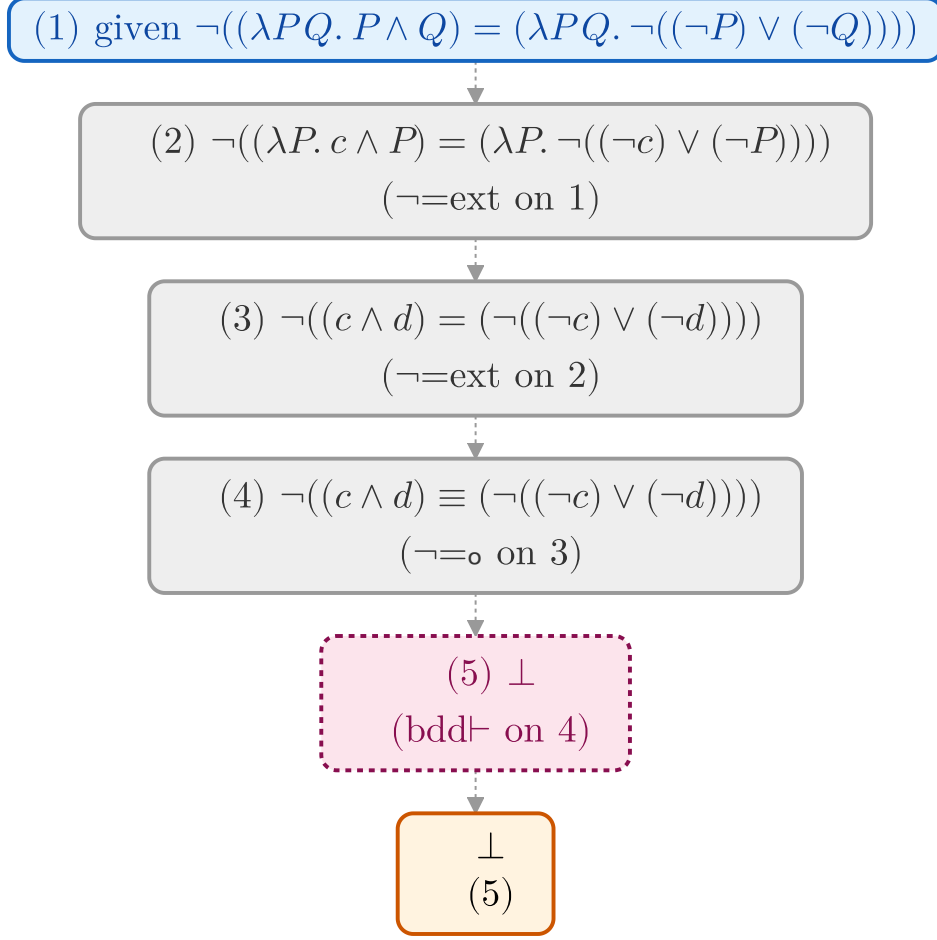


Figure A.13: THM. The proof of De Morgan's law at the third strength, as an equation between two abstractions.

The fourth strength replaces the left abstraction by $\wedge$ itself and is valid only in $\mathfrak{M}_{\beta\text{fb}}$. `HoISuite` carries all four strengths in six encodings each. The pattern across them is that the primitive-equality form is closed at every strength, while the definitional encodings are closed at the two weaker strengths and time out at the two stronger ones, which places the difficulty in the interaction between γ over a predicate variable and the extensionality rules rather than in either alone.

A.8 Primitive Substitution and Finite Domains

Example 21 is valid in $\mathfrak{M}_{\beta\text{fb}}$ and in no larger class, so it is a direct test of Henkin semantics: it states that a model has exactly four functions from truth values to truth values, and it holds only where both functional and Boolean extensionality are available. A witness for a variable of predicate type is a formula, and the (prim) rule of Section 3.4 enumerates approximations to one by fixing the outermost logical symbol [And89]. The problem supplies p on the four elements of $\mathcal{D}_{o \rightarrow o}$ and asks for p on every function of that type, so the witness has to be assembled rather than found among the branch's terms.

$$(p(\lambda o. 1) \wedge p(\lambda o. \neg 1) \wedge p(\lambda o. \perp) \wedge p(\lambda o. \top)) \supset \forall o \rightarrow o. p(\lambda o. 2(1))$$

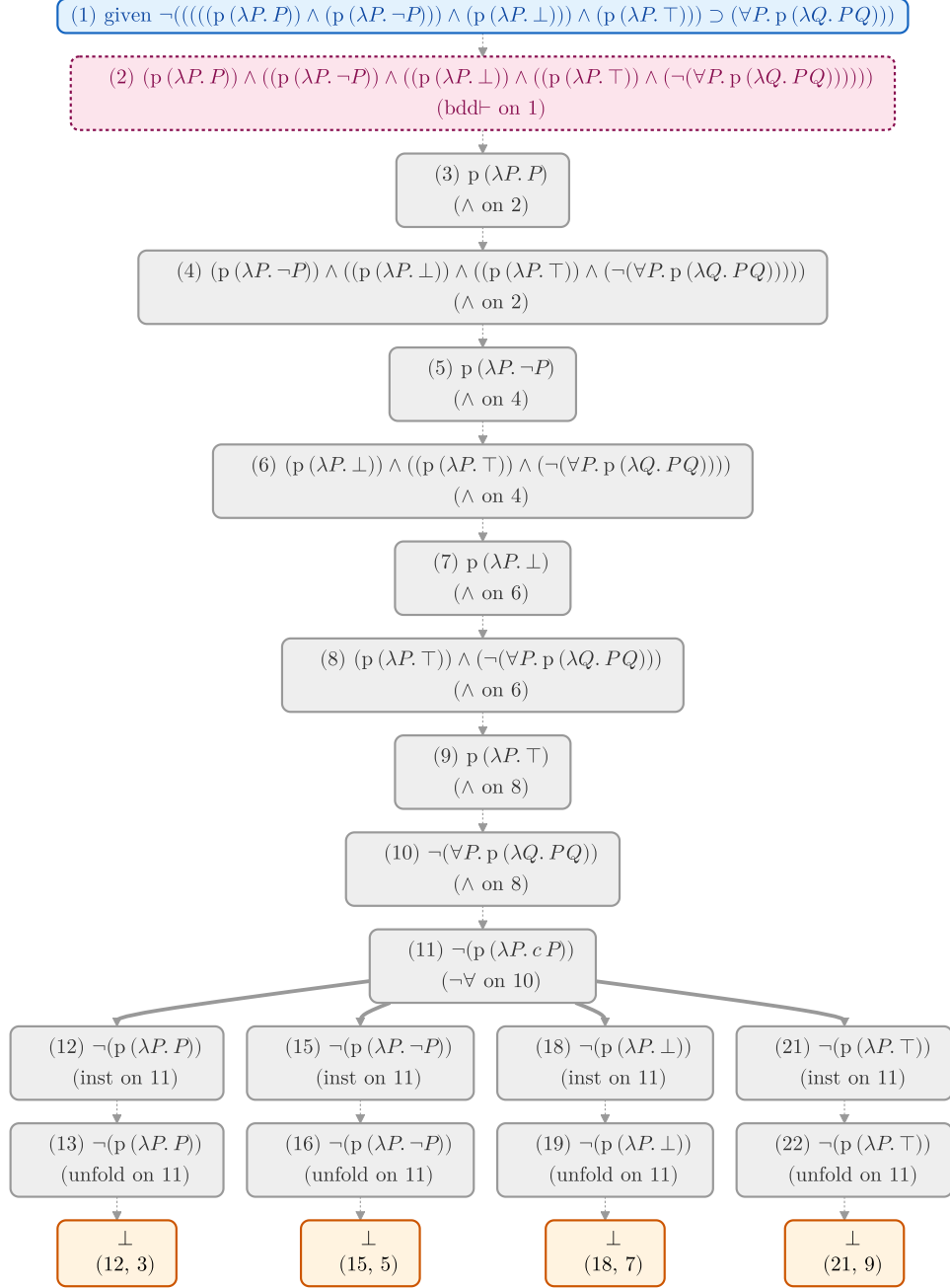


Figure A.14: THM. The proof of Example 21, the four functions of type $o \rightarrow o$.

The same problem is where $(\forall_{\text{fin}})$ applies: $o \rightarrow o$ is a pure o -type, so `Generation.gen_o/1` enumerates its four elements and the quantifier expands linearly instead of through a free variable. With `finite_o_quantification` off, the quantifier is instantiated by the ordinary γ -rule and the four instances have to be recovered by `(prim)`.

Example 12 asks for the η -expanded form of a constant to be interchangeable with the constant. It is valid in $\mathfrak{M}_{\beta\eta}$ and not in $\mathfrak{M}_{\beta}$, so a calculus that identifies only β -equal terms must fail it. SHOT passes it without applying a rule, because terms are held in η -long β -normal form and f is therefore stored η -expanded: the two sides of the implication are the same term, as the display shows. The commitment to η is made by the representation of Chapter 2 rather than by an inference rule, which is worth stating because it means the commitment cannot be ablated.

$$p(\lambda u. f(1)) \supset p(\lambda u. f(1))$$

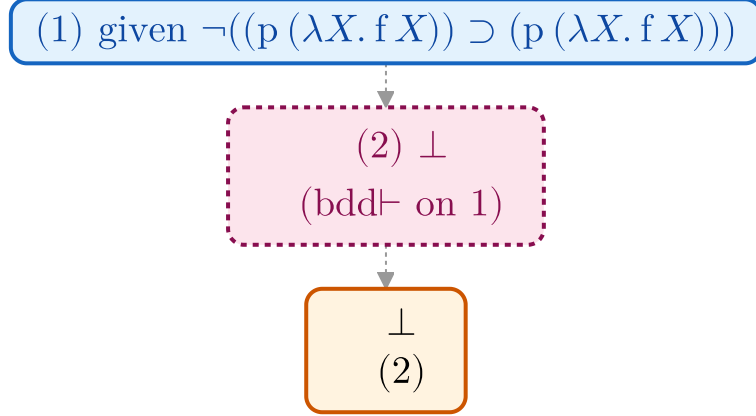


Figure A.15: THM. The proof of Example 12: both sides are the same term, so the proof closes without a rule firing on η .

A.9 Higher-Order Unification

Example 27 belongs to the second dimension. It asks for a binary connective satisfying a specification, and it is valid provided the signature supplies $\vee$, or else $\neg$ and $\wedge$ from which $\vee$ can be built; with a signature too poor to define disjunction it is not a theorem at all. The existential ranges over a predicate of type $(o, o) \rightarrow o$, so the witness has to be built by (prim) and then confirmed by unification against both sides of the equivalence, which exercises the enumeration of Chapter 4 where the flex side is genuinely flexible.

$$\exists(o, o) \rightarrow o. \forall o, o. 3(2, 1) \equiv (2 \vee 1)$$

Status: THM

A.10 Cantor

Example 31, the surjective Cantor theorem, is the largest problem in this appendix that SHOT closes. It sits on both dimensions at once: it is valid in $\mathfrak{M}_{\beta\text{fb}}$ and needs $\neg$ in the signature, since the refutation goes through the diagonal predicate and (prim) has to construct it. Its negation asserts a surjection from ι onto $\iota \rightarrow o$. This theorem has a very small tableau proof in the core calculus.

$$\neg \exists(\iota, \iota) \rightarrow o. \forall \iota \rightarrow o. \exists \iota. (\lambda \iota. 4(2, 1)) = (\lambda \iota. 3(1))$$

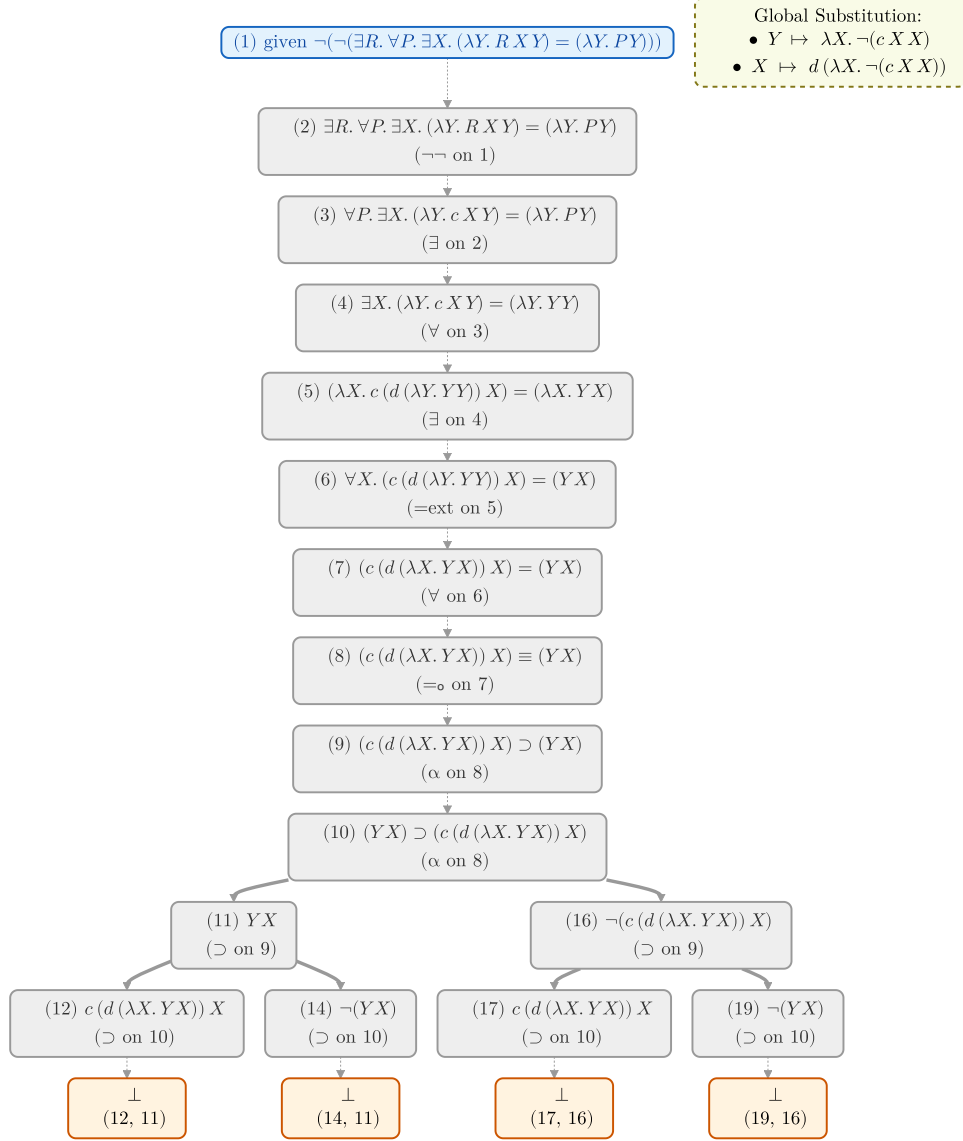


Figure A.16: THM. The proof of Example 31, the surjective Cantor theorem.

Example 32, the injective form, asserts an injection from $\iota \rightarrow o$ into ι and is not closed within the bounds reached. The problem is the hardest in the set, and its difficulty is that the needed set instantiations occur beneath other set instantiations [BB05]. Both forms are in `Holsuite`, and the pair states compactly what the search bounds cost.

A.11 Countermodels

A saturated branch that is decidable open is reported with the literals describing the model. The first problem is propositional and its countermodel is a truth assignment;

it runs with simplification disabled so that the branch rather than the BDD produces the assignment.

Status: CSA, model $p, \neg q, \neg r$

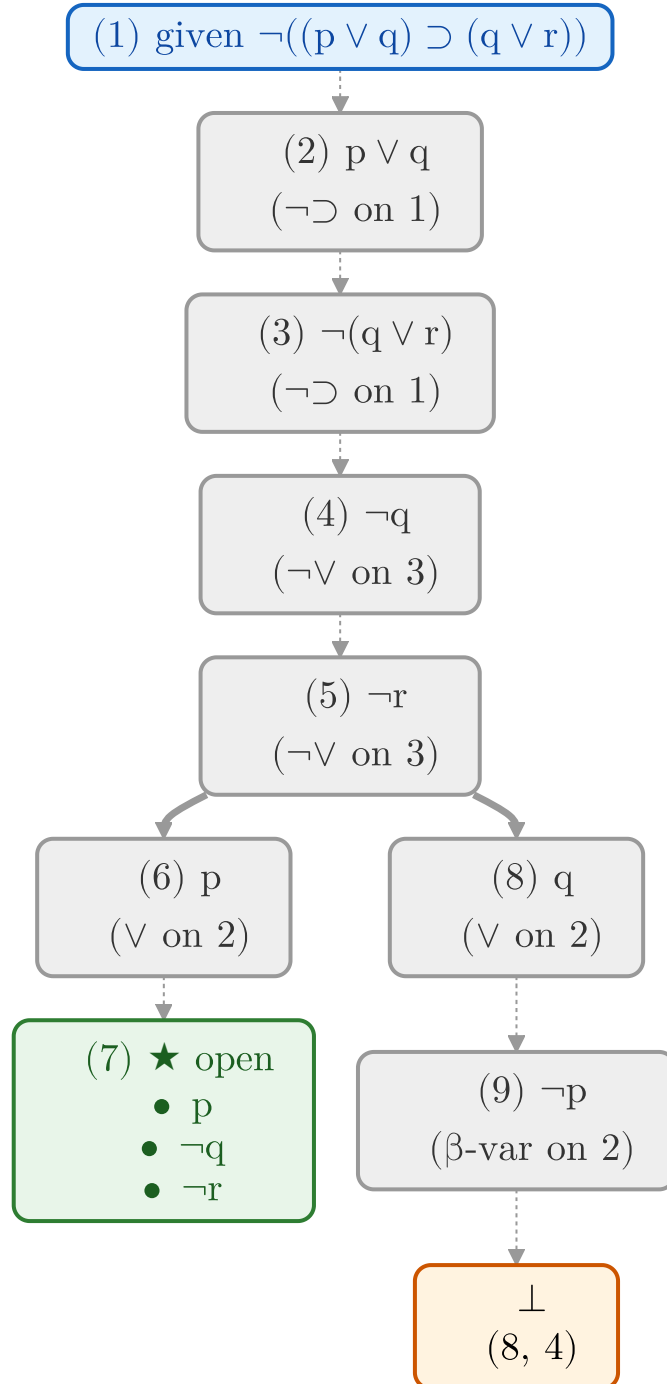


Figure A.17: The saturated branch behind the propositional countermodel.

The second is the primitive-substitution problem above with its conclusion

negated. The reported model is the branch's literal set, and the four applications of p that survive are its content.

Status: CSA, model $p(\lambda. 1), p(\lambda. \neg 1), p(\lambda. \perp), p(\lambda. \top)$

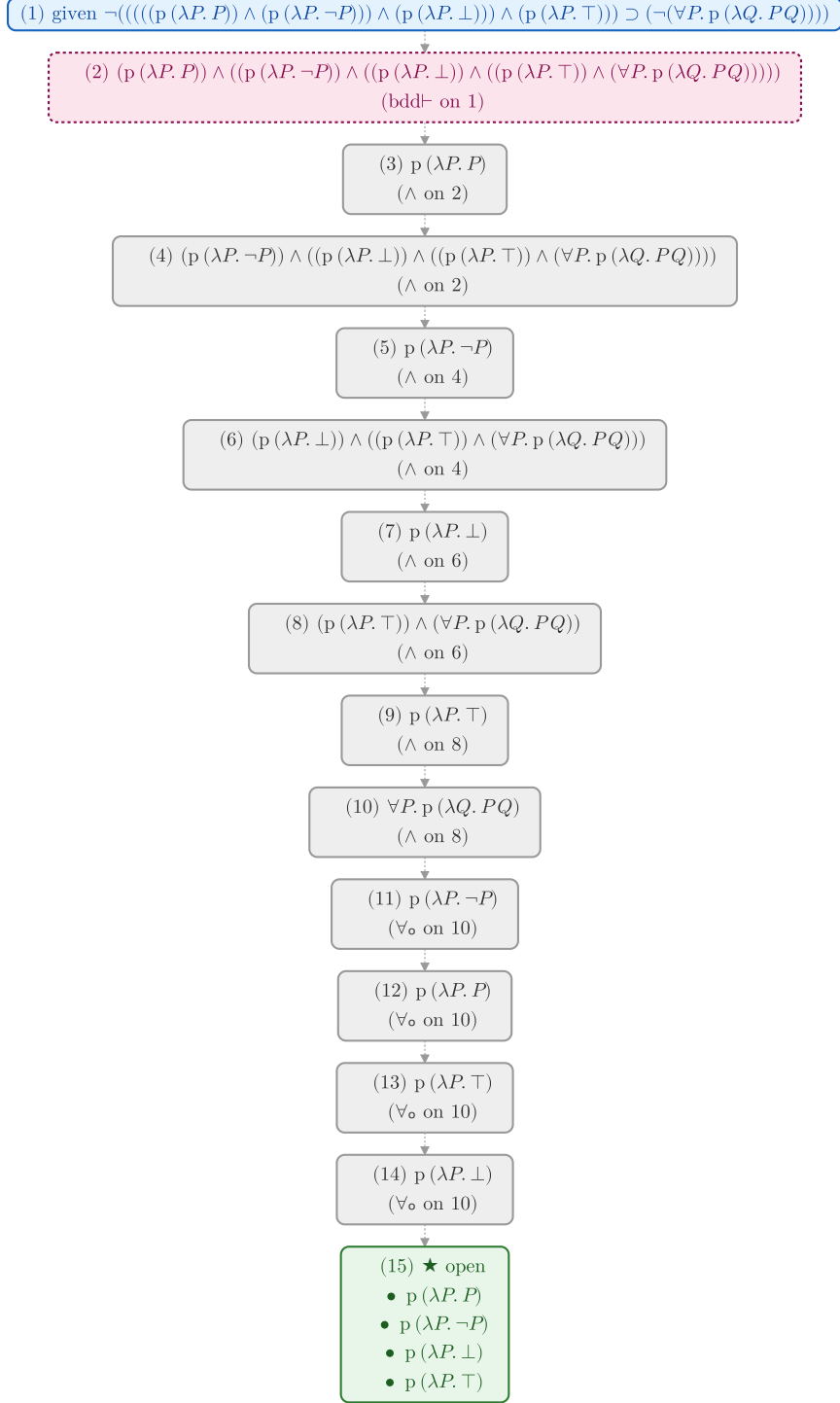


Figure A.18: The saturated branch behind the countermodel of Example 21's negation.

A.12 Bounds, Timeouts and the Unknown Verdict

Every problem of the structured set that SHOT does not close is reported as Timeout, and none is reported as :unknown. That follows from the search control rather than from the calculus. Under iterative deepening the γ -limit and the primitive-substitution depth rise whenever the pool stalls, so the search never runs out of work, and the wall clock set by timeout (5000 ms) is always the binding constraint.

The verdict :unknown is reachable with deepening disabled, which fixes the bounds and lets the search exhaust them. Example 8 is the case: at `initial_gamma_limit 1` and `initial_prim_limit 1` the search exhausts and reports :unknown, while at 3 and 2 the space is large enough that the clock runs out first and the verdict returns to Timeout.

```
[timeout: 60000] -> Timeout in 63847 ms
[timeout: 60000, iterative_deepening: false] -> UNK in 60001 ms
[timeout: 60000, iterative_deepening: false, initial_gamma_limit: 3,
 initial_prim_limit: 2] -> Timeout in 64701 ms
```

The consequence for Chapter 8 is that a run scored by verdict alone cannot distinguish a problem the calculus cannot reach from one it did not reach in time. Separating the two takes a second run with `iterative_deepening` off at fixed bounds, and the ablation reports both.

Declaration of Independent Authorship

I hereby declare to § 9 (12) APO that this thesis is the result of my own independent scholarly work and that in all cases material from the work of others is acknowledged. Quotations and paraphrases are clearly indicated and no material or tools other than listed have been used. This written work has not been submitted to any university before. I understand that the use of sources whose authenticity cannot be verified is a serious violation of the principles of good academic practice (cf. <https://www.uni-bamberg.de/studium/im-studium/studienorganisation/quellen/> and <https://www.uni-bamberg.de/en/research/research-culture/good-academic-practice/>).

Furthermore, I declare that I have used text- or otherwise content-generating artificial intelligence (AI) software (e.g. ChatGPT) only after consultation and with the permission of my instructor/examiner and that I have noted this under **Tools**. I am prepared to provide detailed documentation of my use of AI software (e.g. chat transcripts) if clarification is required.

I am aware that this digital version can be subjected to a software-supported, anonymized check for plagiarism.

Bamberg, 21.08.2026

Place, date



Signature

Tools

For planning and executing this work, I used Claude Sonnet 4.6 & 5, Claude Opus 4.7, 4.8, 5, Claude Fable 5, and Gemini 3.1 Pro for architecture planning, code and documentation generation, and drafting of chapter content. I further used Claude Sonnet 5 and Claude Opus 5 for generating a $\LaTeX$ version of this work in preparation of a printable document. After using these tools, I reviewed and edited everything generated as needed and take full responsibility for the presented content. The thesis was written and executed in Livebook, and the prover it describes is published as the packages `shot_ds`, `shot_un`, `shot_to` and `shot_tx`, whose sources are archived with this work.